Not Worth the Effort: Generative AI and Effort Opacity in Collaborative Work *

Sahiba Chopra

Haas School of Business, UC Berkeley

Abstract

Generative AI helps individuals produce more, and sometimes better, work, yet realizing the value of those contributions often depends on the effort others invest in reviewing and developing them. Because creator effort is rarely observable, reviewers infer it from the work before deciding how much review effort to reciprocate. I argue that AI creates *effort opacity* by making the traces of effort in work less informative, leading reviewers to reciprocate less through review. Departing from questions about whether AI-assisted work is judged favorably, this paper asks whether AI-assisted work can attract the review effort it needs to be developed. Using a custom AI classifier on code from top GitHub projects with over 1 million stars and 2 million downstream dependencies, I find that AI-assisted work receives 25% less detailed feedback and is 30 percentage points more likely to be rejected, even after controlling for code quality. A pre-registered study with software developers tests the effort opacity mechanism. Holding the underlying code constant, I vary AI disclosure and visible evidence of contributor effort. Making substantial creator effort legible increases review priority even when AI assistance is disclosed. Perceived contributor effort is the only measured mediator with a statistically significant indirect effect. These findings show that by making effort harder to infer from the work itself, technology can reduce the reciprocal review needed to develop and integrate valuable work.

*I am grateful to Mathijs de Vaan and Abhishek Nagaraj for their guidance and support throughout this project. I also thank Don Moore and Solène Delecourt for valuable input and guidance on the experimental designs. The code detection experiment was pre-registered on OSF (osf.io/76nrt). This mechanism experiment was preregistered on OSF (https://osf.io/dypjh). I gratefully acknowledge financial support from the Institute for Business Innovation at the UC Berkeley Haas School of Business. All errors are my own.

1 Introduction

Generative AI has been shown to produce large individual productivity gains - access to AI assistance raises customer-support productivity by 15%, with gains above 30% for novices (Brynjolfsson, Li, and Raymond, 2025); developers with an AI coding assistant complete tasks roughly twice as fast (Peng et al., 2023); similar gains appear in professional writing and consulting (Noy and Zhang, 2023; Dell'Acqua et al., 2026). These gains, however, are measured at the level of the individual task, and knowledge work rarely ends at the individual. Much knowledge work is interdependent: one person produces a contribution that another must review. Therefore, realizing the value of one person's contribution depends on the review effort it attracts from others. I argue that labor-augmenting technologies can raise individual output while weakening the reciprocity on which collaborative review depends.

Work serves two functions in collaboration: it is an output to be evaluated, and it also contains traces of the effort that was invested in creating it. I argue that labor augmenting technologies can potentially improve the quality of work while making effort harder to infer from the work itself. This matters because reviewers use inferred creator effort to decide how much effort to contribute in return. A long tradition of social exchange research argues that people reciprocate based on what they believe others have invested in an exchange (Cropanzano et al. 2017; Cropanzano and Mitchell 2005; Gouldner, 1960; Blau, 1964; Emerson, 1976). Applied to collaborative work, a reviewer may therefore invest more effort in a contribution when the creator appears to have invested more effort in producing it. Research shows that the value recipients assign to help depends on how they interpret the exchange, providing a basis for understanding why equivalent outputs may elicit different reciprocal responses (Flynn, 2006). I call this difficulty in inferring human effort from the work itself *effort opacity*. When reviewers interpret signs of assistance as evidence of little human effort, work may attract less reciprocal review. Work created with the assistance of labor-augmenting technology can consequently attract less reciprocal effort even when its creator has invested substantial effort, and the work is of high quality.

Why should inferred effort matter when reviewers ultimately care about the quality of the work? If reviewers allocated effort solely according to a contribution's expected usefulness, contributions expected to be equally useful would attract similar review, regardless of how they were produced. Yet intrinsic quality often cannot be known without first investing scarce review effort to examine the work (Criscuolo et al. 2017; Lamont, 2012). Reviewers must therefore allocate effort before quality is fully revealed (Ocasio, 1997; Piezunka and Dahlander 2015). More generally, the information legible to evaluators can shape both whether they evaluate as well as the value of the evaluations they provide (Botelho, 2024). In doing so, they often rely on more readily observable cues, including what the work appears to reveal about the effort already invested in it (Chinander and Schweitzer, 2003; Kruger et al., 2004; Morales, 2005). Perceived creator effort is therefore not only a cue to a contribution's possible quality, it also provides a basis for deciding how much effort to return. It can shape both how a contribution is assessed and whether it receives the developmental review

through which its quality can improve.

Generative AI is one form of assistance that can create *effort opacity*. It can perform a growing share of the execution involved in producing work at low and declining cost (Bick et al., 2025; Cowgill et al., 2026). Recent research has found that generative AI can produce outputs that observers have difficulty distinguishing from human-produced work (Kovács, 2024). Therefore, features that once included traces of effort can now potentially be produced by a generative model. AI therefore weakens the extent to which the work itself reveals the human effort invested in creating it. Importantly, I argue that technology reduces reciprocity by creating *effort opacity*, not simply because reviewers oppose the use of technology in producing work. The mechanism is based on information that work is expected to provide. Reviewers use traces in work to infer how much effort the creator invested, and those inferences help determine how much effort they reciprocate. When labor augmenting technologies make creator effort more difficult to infer from the work, this basis for reciprocal exchange weakens, reducing the effort reviewers invest in return.

The setting for this study is open source software development where AI coding assistants are increasingly used to write code. The setting also captures the interdependence central to the theory: a contribution must attract a maintainer's scarce review effort before it can be developed and integrated. Maintainers therefore do not merely evaluate finished work - through their review effort, they help determine what the work becomes and whether it is integrated at all. Following Yeverechyahu, Mayya, and Oestreicher-Singer (2025), I sample popular Python projects and restrict the analysis to substantive code contributions authored and reviewed by different people. The resulting dataset contains 13,614 contributions across 166 projects with more than one million stars and nearly two million downstream dependencies.

To measure AI assistance, I fine-tune a classifier to identify signatures of AI assistance and specifically, code likely produced with little human revision. I refer to this measure as *AI-predicted*, rather than verified AI use, because extensively revised AI code may appear human-generated and human-written code may sometimes resemble AI output. AI-predicted contributions receive 25% less detailed feedback, 22% more general feedback, and are approximately 30 percentage points more likely to be rejected. These estimates establish the consequential field pattern, but not its mechanism: AI-predicted contributions may differ on unobserved dimensions. The accompanying studies therefore test the perceptual and behavioral links proposed by the theory.

A preregistered study tests whether developers can detect the signatures captured by the classifier and whether perceived AI involvement changes effort attribution. Developers distinguish AI- from human-generated code above chance, although imperfectly, and attribute less human effort to code they believe is AI-generated regardless of its actual source. Effort attributions therefore follow perceived rather than actual AI use, validating a central perceptual assumption behind the observational analysis.

The observational evidence cannot observe reviewers' inferences or hold the work constant. I therefore conduct a preregistered 2×2 within-subjects experiment that holds code constant while varying AI disclosure and effort legibility. Among AI-disclosed contributions, high effort-legibility

improves review priority by 0.76 rank points ($d_z = 0.43$, $p < .001$). Reviewers therefore give greater priority to work when the creator's effort is visible, even when the use of AI remains disclosed. Under high effort legibility, the estimated disclosure difference is small and falls within the equivalence bounds reported below. Perceived contributor effort is the only measured mediator with a statistically significant indirect effect in the parallel mediation model. These findings support perceived effort as a pathway connecting high effort legibility to review priority and show that disclosed AI assistance does not prevent contributions from benefiting from making effort legible. Together, these findings support effort legibility as a basis for prioritizing contributions for review, consistent with the proposed account of reciprocal review effort.

First, the paper contributes to research on generative AI and work. Prior studies show that AI can improve the speed and quality of individual production (Noy and Zhang, 2023; Brynjolfsson, Li, and Raymond, 2025; Dell'Acqua et al., 2026; Peng et al., 2023). I examine what happens downstream, when an output must attract another person's attention before it can be accepted and used. The central insight is that AI's organizational value depends not only on what individuals produce, but also on whether their work makes effort legible so that it attracts the complementary human effort required to develop and integrate it. Organizational returns to AI therefore cannot be inferred from individual production gains alone.

Second, the paper contributes to social exchange theory. Existing research explains how reciprocity depends on exchange structure, risk, and accumulated trust (Gouldner, 1960; Blau, 1964; Emerson, 1976; Molm, 2003, 2010; Cropanzano & Mitchell, 2005; Cropanzano et. al, 2017). I extend this work by examining how labor-augmenting technologies alter the traces through which collaborators recognize one another's effort investment. Signaling theory explains how observable work can reveal a creator's otherwise unobservable investment; social exchange theory explains why that inference can motivate investment in return (Connelly et al. 2011). I connect these traditions by identifying effort legibility as a condition shaping reciprocity. When assistance weakens the traces of effort in work, collaborators lose part of the informational basis on which their reciprocity depends. Therefore, labor-augmenting technologies can consequently weaken the relationship between effort invested and effort returned without requiring creators to invest less or for the work to be poor quality.

Third, the paper contributes to research on evaluation in organizations. Recent studies show that AI disclosure can reduce evaluations of quality, competence, motivation, trust, and authenticity (Raj, Berg, and Seamans, 2025; Reif, Larrick, and Soll, 2025; Schilke and Reimann, 2025; Gai, Hou, and Tu, 2025). I conceptualize evaluation not only as a judgment of finished work, but also as a reciprocal allocation of review effort that helps develop and integrate the work. The findings further show that making human effort legible increases review priority even when AI use remains disclosed. This shifts attention from whether AI-assisted work is judged favorably to whether it attracts the complementary effort required for its development.

The paper proceeds as follows. Section 2 develops the theory of *effort opacity*. Sections 3–5 describe the open-source setting, empirical design, and observational evidence. Section 6 presents

the preregistered mechanism experiment, and Section 7 discusses the theoretical and organizational implications. Across these analyses, the central question is when AI-assisted work is judged "worth the effort" by the collaborators whose review effort is necessary for its development and integration.

2 Theory

2.1 Overview

I develop a theory of how labor-augmenting technology changes the discretionary developmental review that contributions attract in collaborative work. The central argument is that a contribution serves two roles: it advances shared work, and it provides evidence of the creator's effort in that work. I argue that technology can preserve or improve the first while weakening the second. When reviewers partly reciprocate the effort they believe the creator invested, this divergence can reduce the developmental effort returned to a contribution, which has consequences for whether the contribution is integrated into an organization or not.

The theory concerns exchanges in which a creator submits a contribution, a unit of work, for another person's evaluation and development. Creators and reviewers supply different forms of effort toward a shared goal (Bedwell et al., 2012). Review may yield a verdict, such as acceptance or rejection, but it can also provide developmental feedback that helps the creator revise and improve the work (Harvey and Kou, 2013; Harrison and Rouse, 2015). Such feedback can accompany either verdict, and its depth may remain discretionary even when evaluation is required. Review is therefore not only an assessment of what the creator has produced: it is also a further investment in what the contribution can become. The theory applies to peers and supervisors insofar as they retain discretion over this investment.

I define *creator effort* as the costly personal investment in producing a contribution (Kool and Botvinick, 2018; Inzlicht et al., 2018; Van Iddekinge et al., 2023), and *reviewer effort* as the costly investment in assessing, verifying, and developing it. Reviewers often cannot directly observe creator effort. Instead, they infer it from *traces*: features of the contribution that they treat as evidence of the activities involved in its production. Building on social exchange theory, I argue that this perceived effort provides one basis for an obligation to return effort toward the shared work (Gouldner, 1960; Blau, 1964). I term this component of developmental effort *reciprocal review*: reviewer effort motivated, at least in part, by perceived creator effort.

I use *labor-augmenting technology* to refer to technology that expands a person's productive capacity by supporting or performing activities involved in producing a contribution. A technology may automate particular activities while augmenting the creator's capacity to produce the contribution as a whole (Acemoglu and Restrepo, 2019; Raisch and Krakowski, 2021). Its use does not establish how much effort the creator actually invested: a creator may reduce effort or redirect it toward defining the problem, evaluating alternatives, and checking and adapting the result. Generative AI is the technology studied here, but the argument concerns a more general relationship

between production technology and the inference of effort.

Technology creates *effort opacity* when it reproduces traces previously associated with substantial creator effort without providing equally informative evidence of the effort actually invested. I define effort opacity as the difficulty of inferring human effort from work produced with the assistance of a labor-augmenting technology. The same observable contribution may then be compatible with substantial or little human effort. The problem is not necessarily that creators have stopped investing effort, but that the work no longer reliably contains traces of human effort.

Opacity concerns the informativeness of the traces of effort in work, whereas perceived effort concerns the judgment reviewers draw from it. A reduction in reciprocal review requires a further step. For contributions whose traces would otherwise suggest substantial investment, I argue that reviewers discount perceived creator effort when they believe that a labor augmenting technology supplied those traces and lack countervailing evidence of human effort. Through reciprocal review, this discount reduces the developmental effort they are willing to return. A creator who invested substantially can consequently receive less reciprocal review because the contribution no longer makes that investment apparent Conversely, credible evidence of substantial creator effort should increase reciprocal review even when assistance of a labor augmenting technology is known. Knowing that a technology was used is not equivalent to knowing how much effort its user invested. The argument therefore distinguishes disclosure of assistance from the legibility of effort, and it does not predict a uniform penalty for technology-assisted work.

The organizational implication is that the returns to labor-augmenting technology depend partly on whether contributions continue to elicit the human investment needed to develop them. Technology can expand a creator's capacity to produce work while weakening an informational basis on which others invest in that work. The sections that follow develop the reciprocal structure of developmental review, explain how assistance of labor augmenting technologies changes effort inferences, and specify the conditions under which this change should reduce reciprocal review.

2.2 Reciprocal Exchange in Collaborative Work

The theory rests on an assumption that may not be obvious: that the developmental feedback a contribution receives depends less on its quality than on the perceptions of creator effort. This section provides arguments that support that assumption. I first argue that developmental review is part of collaborative work, then I specify what reciprocity means within collaborative work, then I argue why quality is not the aspect that governs how reviewers allocate developmental feedback, and finally I argue why perceived creator effort is as, and sometimes more, important than quality in determining developmental feedback.

Developmental review is collaborative work in the sense that two parties engage in joint activity, toward a shared goal, in a way that makes each party's effort contingent on the other's (Bedwell et al., 2012). The activity is joint because a contribution is rarely complete when it is submitted. Assessing it requires understanding its content, rationale, and context, much of which the reviewer

acquires only in the course of review (Bacchelli and Bird, 2013; Pascarella et al., 2018), and improving it requires knowledge the creator often lacks, for example, what it would take to integrate (Sadowski et al., 2018). Research on creative collaboration shows that evaluation and feedback are where participants identify problems, examine alternatives, and revise emerging work (Harvey and Kou, 2013; Harrison and Rouse, 2015), and that realizing a contribution's potential often requires elaboration and sustained help from others (Perry-Smith and Mannucci, 2017; Fisher, Pillemer, and Amabile, 2018). The creator and the reviewer therefore supply different forms of effort toward the same contribution. The goal is shared because both parties benefit when the contribution is integrated into the project in improved form. And the effort is reciprocal because each party's effort is a response to the other's: the creator submits, the reviewer returns a verdict and, at their discretion, developmental feedback, and the creator revises in turn.

This structure holds whether the reviewer is a peer or holds formal authority over the creator (Rice, 2018). In the empirical setting I study, the reviewer also decides whether the contribution is integrated (Gousios et al., 2015), and the theory applies to that case and to peer review alike, because the features that drive it are the same in both. A verdict is required by the reviewer's role; the depth of developmental feedback exceeds what the role requires and is allocated at the reviewer's discretion (Organ, 1988). True creator effort cannot be observed by a reviewer and must be inferred from the contribution. And the relationship between a supervisor and a subordinate is itself theorized and measured as a reciprocal exchange whose quality depends in part on the supervisor's perception of the subordinate's effort (Wayne, Shore, and Liden, 1997; Liden and Maslyn, 1998; Cropanzano and Mitchell, 2005). Formal authority may raise or lower how much developmental feedback a reviewer extends at baseline; it does not change the direction in which perceived creator effort moves it.

By *reciprocity* I mean the norm that receiving a benefit creates an obligation to return one (Gouldner, 1960). Three features of the norm matter here. First, the return need not reproduce what was received. Gouldner (1960) distinguishes exchanges of similar things from exchanges of different things regarded as equivalent in value, and Blau (1964) emphasizes that social exchange leaves the precise return unspecified. Reciprocal exchange can therefore involve different forms of resource investment, rather than a return of the same resource. In developmental review, the creator supplies a contribution and the effort embodied in it; the reviewer returns review effort. The two are different in kind and need not be equal in quantity. Second, the obligation is proportionate to what has been received: a larger benefit obligates a larger return (Gouldner, 1960). Third, the obligation is triggered by receipt, not by the expectation of future exchange, so it operates in a single encounter as well as in an ongoing relationship (Gouldner, 1960; Fehr and Gächter, 2000). This matters because many collaborative exchanges, including many in my setting, involve parties who have not worked together before and may not again. What the creator works on is therefore a contribution that advances the shared goal (Pascarella et al., 2018). What the reviewer returns is the developmental feedback that improves the work. Reciprocal review, as defined above, is review effort motivated at least in part by the reviewer's perceptions of the creator's effort.

A natural objection is that a reviewer should allocate developmental feedback according to the contribution's quality, its usefulness to the project, or both, leaving no role for effort. I use *quality* to mean the degree to which a contribution meets the standards of the task; in my setting, this is defined as whether the contribution is suitable for integration (Sadowski et al., 2018). Quality and usefulness do matter. Reviewers prioritize contributions partly according to their criticality and urgency (Gousios et al., 2015), and they assess whether work is suitable for integration and what changes it requires (Sadowski et al., 2018). But three features of developmental review can prevent quality from governing the allocation of review effort. First, quality is not observable at the moment the work is received. Evaluating a contribution requires understanding its content, rationale, and context (Bacchelli and Bird, 2013; Pascarella et al., 2018), so a reviewer must sometimes decide how much effort to invest before knowing the quality of the work. Second, in collaborative work, quality is partly a product of review. Developmental feedback changes the contribution (Harvey and Kou, 2013; Harrison and Rouse, 2015; Oliveira et al., 2025), and a low-quality contribution may warrant more development, not less (Perry-Smith and Mannucci, 2017; Fisher, Pillemer, and Amabile, 2018), so initial quality cannot by itself help a reviewer gauge how much developmental review is worth supplying. Third, review effort is scarce, and its allocation responds to costs and obligations as well as to the value of the work. Reviewers allocate attention across contributions according to their importance and the effort required to assess them (Gousios et al., 2015); helping others carries costs for the helper's own work (Mueller and Kamdar, 2011; Dierdorff and Rubin, 2022).

Therefore, perceived creator effort influences reviewer effort oftentimes regardless of the quality of the work. First, traces of effort are available at submission, before any review effort has been spent, so they can inform the allocation decision at the moment it is made. People use effort as a cue when quality is difficult to assess (Kruger et al., 2004), and a reviewer facing an unassessed contribution is in exactly that position. Second, perceived creator effort is the creator's investment in the shared work, and it is the investment, not the work itself, that triggers the obligation to reciprocate. Effort shapes evaluations of a person even when it adds no quality or economic value (Celniker et al., 2023). Evidence from exchange settings connects the recognition of another's investment to gratitude and to reciprocal responses: consumers reward firms' effort even when it does not improve the product, with gratitude mediating the response (Morales, 2005); making a provider's effort visible raises perceived value through perceived effort and feelings of reciprocity (Buell and Norton, 2011; Buell, Kim, and Tsay, 2017; Hwang, Lee, and Legendre, 2024); gratitude and reciprocal transfers respond to the cost a benefactor incurred, not only to the benefit received (Yu et al., 2018); and gratitude motivates costly assistance in return (Bartlett and DeSteno, 2006; Palmatier et al., 2009). In organizations, felt obligations to reciprocate motivate discretionary contributions to coworkers (Dierdorff and Rubin, 2022). Building on these findings, I argue that recognizing a creator's effort motivates reviewers to supply developmental feedback beyond what the contribution's quality alone would warrant, and, by the same logic, that failing to infer creator effort leads to reviewers withholding developmental feedback that quality would otherwise warrant.

In summary, this subsection establishes that developmental review operates under a logic of reciprocal exchange rather than a strict transactional assessment of output quality. While quality establishes a baseline for the mandatory role-based verdict (acceptance or rejection), it is the perceptions of creator effort that are inferred through traces in the work that, I argue, govern the reviewer's willingness to supply high-effort discretionary, developmental feedback. When these informational signals are intact, high creator effort commands a high-effort review in return, sustaining the collaborative productivity loop.

2.3 Labor-Augmenting Technology and Effort Opacity

This section derives the hypotheses. It first focuses on generative AI, the labor-augmenting technology studied in this paper and identifies the property that makes some uses of such technology, and not others, a source of effort opacity: they produce the traces that reviewers use to infer creator effort. It then reviews what is known about how people evaluate work produced with AI, which motivates the prediction that contributions produced with the assistance of labor-augmenting technology receive less reciprocal review (Hypothesis 1). It closes by setting the effort opacity framework against three alternative explanations for that penalty, that AI-assisted contributions are of lower quality, and that reviewers are averse to AI, and derives the prediction that the reduced reciprocity instead runs through perceived creator effort (Hypothesis 2).

Labor-augmenting technologies vary in how much of the labor involved in producing a contribution they perform. Automation reallocates particular activities from people to technology (Acemoglu and Restrepo, 2019), while augmentation raises the productivity of the person who remains engaged in the work, and the two coexist within a single production process (Raisch and Krakowski, 2021): a creator may delegate an activity to a tool while investing effort in defining the problem, evaluating alternatives, and adapting the result. The overview used the term at the level of the contribution for this reason. Generative AI is labor-augmenting in this sense. Studies show that AI has reduced the time professionals spend on writing tasks by 40% while raising rated quality (Noy and Zhang, 2023), raised the productivity of customer-support agents by 14% on average and by more for less experienced agents (Brynjolfsson, Li, and Raymond, 2025), and increased the output of software developers in laboratory and field settings (Peng et al., 2023; Cui et al., 2025). Each of these findings provide examples of the property of labor-augmenting technologies identified in the overview: a contribution of given quality requires less creator effort than it does when a labor-augmenting technology is not used to create it.

However, not every use of a labor-augmenting technology produces effort opacity, and the amount it produces does not necessarily depend on how much of the labor the technology performs. It depends on what the technology does to the traces used to infer creator effort. A trace is informative about creator effort for the reason any signal is informative: producing it is costly, and the cost falls more heavily on those who lack the underlying attribute (Spence, 1973; Connelly et al., 2011). A technology therefore creates opacity to the extent that it lowers the cost of producing traces that

reviewers use to infer effort.

The way this plays out follows a simple pattern. Every labor-augmenting technology takes over some of the activities involved in producing a contribution and leaves the rest to the creator. The traces of the activities it takes over stop being informative about effort, because the technology can now produce them cheaply. Reviewers then have to rely on traces of the activities the creator still performs. A spell-checker takes over spelling, so correct spelling stops telling a reviewer anything about the creator's care, even though readers did draw inferences about a writer's conscientiousness from spelling and grammatical errors (Vignovic and Thompson, 2010). Once spell-checkers were universal, reviewers potentially stopped using correct spelling as evidence of effort altogether; the trace was retired rather than rendered ambiguous, as a signal is when every sender can produce it at the same cost (Spence, 1973). This caused little disruption, because the activities the spell-checker left to the creator, developing the argument and structuring the document, already left their own traces in the document. Reviewers simply read those instead.

Whether the shift is this easy depends on one thing: whether the activities the technology leaves to the creator already leave traces of their own. Generative AI is consequential because, for the activities it leaves to the creator, they usually do not. Generative AI takes over execution. It produces drafts, explanations, proposed solutions, and tests (Noy and Zhang, 2023; Peng et al., 2023; Dell'Acqua et al., 2023). What it leaves to the creator is judgment: deciding what the contribution should do and why, which alternatives to reject, and how to check that it works. Before generative AI, judgment rarely needed traces of its own, because execution was the evidence of it. For example, a working, tested piece of code could be used as evidence that someone had thought about what to build and how to verify it, since the code could not have been produced without that thinking. Generative AI severs this link. Execution can now be produced without the creator's judgment, so execution stops being evidence of judgment, and the judgment the creator did exercise leaves no trace in the contribution unless the creator adds a new trace. Unlike the spell-checker case, there is potentially nothing already in the contribution for reviewers to shift to, so one must be created anew. Stated precisely, effort opacity is created when a labor-augmenting technology makes the old traces of effort uninformative. What makes effort legible again are traces of the activities that the technology cannot perform. A creator can use generative AI to write every line and still have made every decision but whether a reviewer can infer this depends on whether those decisions left a trace.

Three features of generative AI make the technology different from many other labor augmenting technologies. The first is that the capabilities of the technology expand over time which make the exact aspects of what counts as effort dynamic. The second is that generative AI adoption and use is uneven across creators. Roughly a quarter of employed adults in the United States reported using it for work in a given week two years after its release, with adoption varying by occupation and education (Bick, Blandin, and Deming, 2025), and much use goes undisclosed (Zhang, Xu, and Alvero, 2025). Third, it leaves no cue that reliably distinguishes what the technology was and was not used for. A creator may use it to edit a sentence, to generate alternatives that are then weighed, or

to produce an initial contribution that receives little further examination (Dell'Acqua et al., 2023), and the contribution may not reveal which of these the technology was used for.

While it might be difficult for reviewers to accurately determine whether work used AI assistance or not, a growing body of research shows that when reviewers are told that work was produced using AI, they evaluate the work less favorably than otherwise equivalent work, and that the penalty attaches to the person as well as the work. People discount algorithmic judgment relative to human judgment after seeing it err (Dietvorst, Simmons, and Massey, 2015) and value creative work less when it is labeled as AI-generated (Millet et al., 2023; Bellaiche et al., 2023). Recipients feel less heard by a message once it carries an AI label, even when its content is unchanged (Yin, Jia, and Wakslak, 2024). Actors who disclose using AI are trusted less across a range of occupations, an effect that runs through reduced perceptions of legitimacy (Schilke and Reimann, 2025). Colleagues who use AI at work are judged lazier, less competent, and less diligent than colleagues who use other forms of assistance (Reif, Larrick, and Soll, 2025), and help offered with the aid of AI reduces the recipient's perception of the helper's warmth, felt obligation to reciprocate, and subsequent reciprocity (Dishop et al., 2025).

In nearly all of this research, the evaluator is told that AI assistance was used. Therefore, in these studies, the treatment is the AI label, and the question is what happens as a result of this disclosure. My argument starts one step earlier. What drives the penalty in the first study in this paper is the reviewer's own belief that the contribution was produced with AI, which the reviewer forms from the contribution itself. This has three implications that the label studies do not test. First, the penalty should appear in work where AI use is not disclosed, because AI assisted work often carries AI "smells" (Fiedler, A., & Döpke, J., 2025). Second, contributions that look AI-assisted whether or not they were, should be penalized since the reviewer is responding to an inference, not to ground truth. Third, the outcome is not an evaluation but a decision: how much discretionary effort to return to a contribution the reviewer must actually process, in an exchange where that effort has consequences for the work and for the person who submitted it. However, reviewers can become aware of a technology's use through several routes like the creator's disclosure, using an AI detection tool, or suspicion, and these routes carry different information: disclosure may signal candor, discovered concealment may raise concerns of its own, an external label says nothing about what the creator intended, and suspicion can be mistaken. I return to these distinctions below. For the first hypothesis, what matters is that each route places the reviewer in the same position, believing or suspecting that the contribution was produced with the technology's help, and that both the theory and the evidence above give reason to expect such contributions to receive less reciprocal review. In addition, reciprocal review is an allocation of the reviewer's effort, and it can be observed at two moments. At the moment the work is received for review, it is the priority a reviewer assigns to a contribution for their own review effort relative to other demands on it; this is what the experiments measure. After the reviewer reviews of the work, the reviewer can decide the depth of developmental feedback the contribution is worth, and at the limit, whether it was rejected with or without developmental feedback. Given the prior literature that AI disclosed work

is penalized and my argument that developers can detect AI use in code, above chance, the first hypothesis follows:

Hypothesis 1. *Contributions produced with the assistance of labor-augmenting technology receive less reciprocal review than contributions that do not use the technology.*

Hypothesis 1 rests on a premise that the label studies assume away: that reviewers form this belief from the contribution's features when it is not otherwise disclosed. The next section tests that premise directly, before turning to whether the belief has the consequences the hypothesis predicts.

This paper argues that the reduced reciprocity is driven by perceptions of creator effort, and not by three alternative explanations: that work produced with labor-augmenting technology is of low quality, or, in the context of this paper, that it is "AI slop"; that the use of such technology leads reviewers to see the creator as less competent; or that reviewers are averse to the technology itself, whether from distrust of its output, a sense that its use violates norms (Schilke and Reimann, 2025), or a preference for human-made work.

Knowing that AI was used does not establish how much human effort was invested; disclosure resolves uncertainty about tool use while leaving effort opaque. What it does change is what the reviewer can infer from the traces of effort in the work. Traces that would otherwise indicate substantial investment for example, a long document, or a suite of tests, become compatible with the use of the technology and therefore less effort since the technology can now produce them cheaply. This matters for review specifically because, as the previous section established, perceived creator effort is what reviewers use to decide how much developmental feedback to return. A change in what the traces reveal about effort is therefore a change in the basis of reciprocity itself. Opacity by itself makes the effort inference uncertain, and the predicted reduction in perceived effort depends on how reviewers resolve that uncertainty. I argue that they resolve it downward, for two reasons. The technology raises the share of low-effort contributions bearing high-effort traces, so that discounting the traces is a reasonable response to a changed base rate (Gartenberg et al., 2026). And observers treat AI use as having substituted for effort rather than supplemented it, judging those who use it lazier and less diligent (Reif, Larrick, and Soll, 2025). Lacking independent evidence of effort, reviewers therefore attribute less of it to the creator, and because perceived creator effort is the basis on which review effort is reciprocated, the obligation to return developmental feedback weakens. The mechanism can operate on a contribution of high quality, because perceived effort supplies a basis for reciprocity beyond the output's quality (Gouldner, 1960; Morales, 2005).

Hypothesis 2. *The effect of labor-augmenting technology use on reciprocal review is mediated by perceived creator effort.*

The experiment in this study is designed to separate these accounts. Reviewers evaluate contributions that are identical in content across conditions, which holds quality constant; whether the technology's use is disclosed and whether the creator's effort is independently legible are varied independently; and perceived creator effort, perceived quality, and perceived competence are measured as competing paths. Effort opacity is supported if the disclosure penalty runs through perceived effort with the other paths accounted for, and if it attenuates when effort is made legible.

Aversion predicts no such attenuation, and quality and competence predict that the other paths, not perceived effort, carry the effect.

To summarize, this subsection has made three arguments. It argues that generative AI is a labor-augmenting technology that creates effort opacity for a specific reason: it takes over execution, which was the evidence of judgment, and leaves the creator's judgment with no trace of its own unless one is added. It showed that reviewers who believe a contribution was produced with AI, whether use is disclosed or inferred from the contribution itself, reciprocate less developmental review (Hypothesis 1). In addition, I argue that the reduced reciprocity is mediated by perceptions of low creator effort and not quality, competence, or aversion to the technology (Hypothesis 2). The argument so far has been stated for contributions of a particular kind: those whose traces would ordinarily indicate substantial creator effort, reviewed by someone who retains discretion over developmental feedback and has no independent evidence of the effort behind the work. The next section states these scope conditions explicitly and discusses what the theory predicts when they do not hold.

2.4 Scope Conditions

The theory concerns developmental review within exchanges in which contributions advance shared work. Within this domain, four conditions determine when a labor-augmenting technology should weaken reciprocal review through effort opacity.

First, the technology must take over activities whose traces reviewers read as evidence of creator effort. A technology that makes the same activities faster without performing any of them, or that performs activities reviewers never treated as evidence of effort, does not undermine an existing basis for inference.

Second, the activities the technology leaves to the creator must lack traces of their own. When they already leave traces in the contribution, as the argument does in a spell-checked document, reviewers shift to those traces and little is lost. Opacity arises when the remaining activities are judgment whose only evidence was the execution the technology now produces. Because the technology's capabilities expand over time, the line between execution and judgment moves, and the traces that make effort legible must move with it: what counts as evidence of judgment is not fixed.

Third, reviewers must lack independent evidence of creator effort. Seeing the effort behind a contribution, rather than only its output, changes how people reciprocate it (Khubulashvili et al., 2021). Direct observation of the creator's process, knowledge acquired through prior interaction, or the expertise to tell which traces remain informative despite the technology's use each supply such evidence, and they weaken the mechanism to the extent that they establish how much the creator invested. Familiarity with the creator or with the technology does not by itself do so.

Fourth, reviewers must retain discretion over the effort they return. Even where review is mandatory, its timing, depth, and developmental content usually remain the reviewer's to decide, and

these are what perceived creator effort shapes. Where review effort is fully prescribed and enforced, inferred effort has little room to act. Formal hierarchy does not by itself place an exchange outside the theory: a supervisor deciding how much developmental feedback to give on an employee's proposal faces the same four conditions.

Effort opacity need not disappear as a labor-augmenting technology becomes familiar or widely adopted. Knowing that assistance is available, or even that it was used, does not establish the effort invested in a particular contribution. Opacity diminishes when reviewers acquire evidence that distinguishes substantial creator investment from its appearance. New conventions may supply such evidence, for example when contributions are accompanied by credible accounts of problem formulation, verification, or revision. As technological capabilities and production practices change, the traces that support this inference may also change. What becomes obsolete is not necessarily the value reviewers attach to effort, but the evidence through which they recognize it.

Persistent opacity does not, however, imply an unchanging penalty. Reviewers may revise what they regard as an appropriate contribution or place greater weight on other reasons for supplying developmental feedback. The theory therefore predicts neither inevitable adaptation nor a permanent penalty for technological assistance. It predicts reduced reciprocal review where assistance weakens informative traces of effort, reviewers consequently discount creator effort, and they retain discretion to act on that inference.

3 Empirical Setting

The theory requires a setting in which four conditions are visible. First, one person submits work. Second, another person's effort is needed for that work to become valuable. Third, the reviewer has discretion over how much effort to return. Fourth, the production process is not directly observed, so the reviewer must infer human effort from the work and its traces. GitHub pull requests provide such a setting.

GitHub is a platform for collaborative software production. Engagement with open-source communities can provide firms with access to external knowledge that supports technology development and the creation of viable products (Conti, Peukert, and Roche, 2025). Software projects are organized into repositories, and contributors propose changes through pull requests. A pull request asks project members to consider a proposed change and decide whether to respond, request revisions, reject it, or merge it into the project. It contains the submitted code, but also traces of effort around the work: the contributor's description, the sequence of commits, automated checks, discussion, review comments, and the final decision to merge or close. The production process itself remains hidden. Project members generally do not observe how the contributor wrote the code, how much of it was generated by AI, how carefully it was checked, or how much responsibility the contributor took before submitting it.

AI enters this setting by changing the relationship between the work submitted and the human

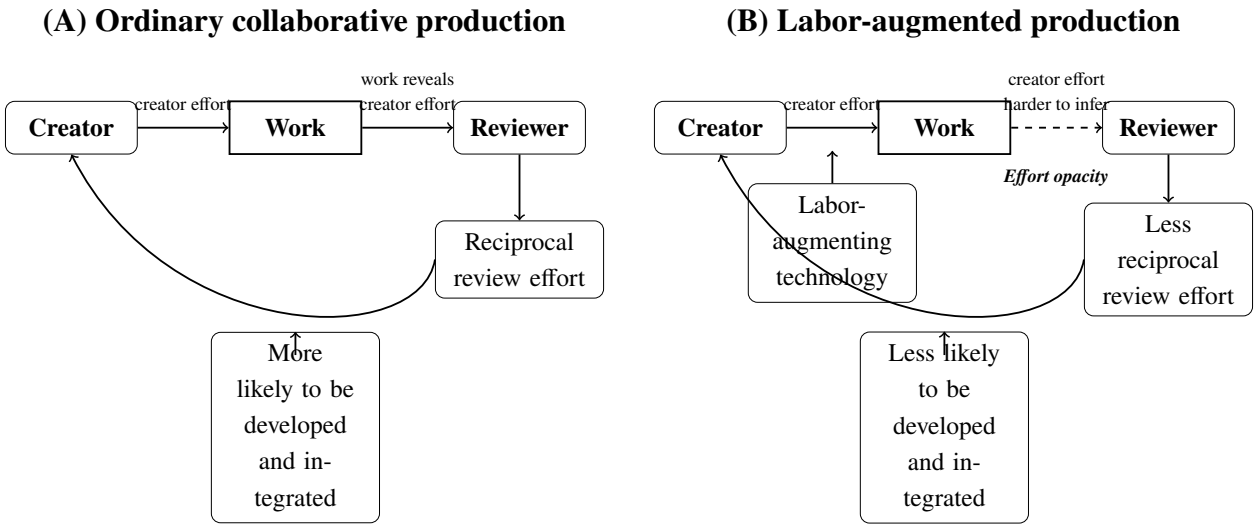


Figure 1: **How labor augmenting technologies shape reciprocity in interdependent work.** In ordinary collaborative production (Panel A), the creator contributes work and the work provides information about the creator's investment, allowing the reviewer to calibrate a reciprocal contribution of their own. In a production process that involves labor augmenting technology (Panel B), assistance changes how the creator produces the work and can make the creator's human effort harder to infer, creating effort opacity. Because creator and reviewer are interdependent, this informational change affects the reviewer's own contribution: lower inferred creator effort can reduce reciprocal review effort and make the work less likely to be developed and integrated.

effort visible from that work. Coding assistants can help contributors write code, generate tests, draft explanations, summarize changes, and respond to comments. The same submitted work can now reflect very different levels of human effort. An AI-assisted pull request may reflect high human effort: the contributor identified the problem, used AI as an aid, revised the output, checked the result, and took responsibility for the change. Or it may reflect low human effort: the contributor generated something plausible and passed the remaining work downstream. From the submitted code alone, project members may not be able to tell which case they are facing.

Maintainers and reviewers face queues of incoming work, and must decide which pull requests deserve close attention, which contributors are worth developing, and which submissions should be handled with minimal effort. Who receives that effort is not neutral for the project. The contributors most dependent on it are first-time contributors, and first-time contributors are the population from which projects recruit their future maintainers. How reciprocal effort is allocated today therefore shapes who is available to review tomorrow.

I measure suspected AI assistance from the submitted code rather than from the pull request description. Descriptions are endogenous. A detailed description may reflect contributor effort, but it may also reflect contributor experience, project norms, strategic self-presentation, anticipated scrutiny, or AI use itself. Conditioning on description detail in the field study would risk controlling

for the very channel through which contributors can make effort visible. I instead treat the description as a repair channel that can be tested experimentally. The observational study asks whether work whose code appears AI-assisted receives less reciprocal effort in the field, and the experiment holds the code constant and varies whether the description makes human effort visible.

The empirical strategy consists of two studies. Study 1 combines an observational field analysis with a preregistered developer task that validates the interpretation of the classifier signal. Study 2 is a mechanism experiment that holds the underlying code constant and manipulates AI disclosure and visible-effort cues. For Study 1, I observe 166 prominent open-source Python projects through their complete contribution histories: more than 320,000 pull requests from 27,000 contributors between 2010 and 2026. These histories provide the population frame for the analysis: who is a first-time contributor, how much standing a contributor has accumulated, and how the contributor base of each project evolves. Within them, I test the theory on 13,614 AI-era pull requests for which AI assistance can be measured. Because true AI use is not observed, I construct a classifier-based measure of AI-predicted code and interpret it as suspected AI assistance, not verified AI use.

The main outcomes capture the effort returned by reviewers. Rejection and early rejection capture withdrawal of reciprocal effort. Inline comments capture high-effort, code-specific response. General comments capture lower-effort discussion-level response. Constructive feedback captures whether the response helps the contributor improve the work. The main specifications compare AI-predicted and non-AI-predicted pull requests within repositories and months, while controlling for contribution size and observable code characteristics. The goal is not to estimate the causal effect of using AI, since contributors who use AI may differ from those who do not. It is to document whether work that appears AI-assisted receives a different pattern of reciprocal effort, and whether that pattern survives observable alternative explanations.

The preregistered developer task tests the assumption behind the field measure. If effort opacity operates through suspicion or attribution, then developers should be able to perceive AI assistance from code at least imperfectly, and those perceptions should matter for inferred effort. Software developers classify code as human-written or AI-generated and rate the effort behind it. The task connects the classifier-based field measure to human perception: it asks whether code that bears the signature of AI assistance is also the kind of work that developers associate with lower human effort.

Study 2 tests the mechanism of *effort opacity*. In a preregistered 2×2 within-subjects experiment, developers evaluate simulated pull requests in which the underlying code is held constant while AI disclosure and visible-effort cues in the pull request description are varied. Low-effort descriptions provide little evidence of what the contributor did. High-effort descriptions make human effort visible by explaining the problem, describing the reasoning, reporting checks, and taking responsibility for the submitted work. The outcome is review priority: how willing developers are to allocate scarce future attention to the contributor. If effort opacity is the mechanism, then visible effort should increase reciprocal effort toward AI-assisted work, and this effect should operate through perceived contributor effort.

Study	Question	Design	Role in the paper
Study 1: Observational evidence	Do pull requests whose code appears AI-assisted receive less reciprocal effort?	Observational analysis of 13,614 pull requests across 166 Python repositories using a classifier-based measure of AI-predicted code. The field measure is validated through held-out classifier tests and a preregistered developer task examining whether developers can distinguish AI-generated from human-written code and whether these perceptions are associated with inferred human effort.	Documents the real-world pattern: greater rejection, less code-specific developmental effort, and a shift toward discussion-level response. The developer task validates a key assumption underlying the observational measure rather than constituting a separate study.
Study 2: Mechanism experiment	Does making human effort visible increase review priority for AI-disclosed contributions?	Preregistered 2×2 within-subjects experiment crossing AI disclosure with visible-effort cues in pull request descriptions.	Tests whether visible effort increases review priority for AI-disclosed work and whether perceived contributor effort, rather than perceived quality or competence, accounts for this prioritization.

Table 1: Overview of the Two-Study Empirical Design

Table 1 summarizes the empirical design.

4 Evidence from GitHub Pull Requests

This study tests whether work that appears AI-assisted receives less reciprocal effort in a real collaborative system. The theory predicts not only that such work will be rejected more often, but that the response to it will be recomposed: high-effort responses should decline, while lower-effort responses may persist or increase. GitHub pull requests provide a setting in which this process can be observed. A contributor submits work, and another project member decides whether and how much effort to return through review, comments, revision guidance, rejection, or integration.

This study is observational. I therefore do not interpret the estimates as causal effects of actual AI use. Contributors who use AI may differ from those who do not in experience, task choice, commitment to the project, or the kinds of changes they attempt. The purpose of this study is instead to document whether pull requests whose code appears AI-assisted receive a different pattern of reciprocal effort, and whether that pattern is explained by observable features of the submitted work.

4.1 Sample Construction

The relevant empirical object is not any pull request on GitHub, but a pull request in which one person submits work and another human collaborator decides how much effort to return. This requirement rules out much of GitHub. Many public repositories are personal, inactive, or maintained by a very small core of developers (Kalliamvakou 2014, Eghbal 2016). Even in canonical open-source projects, contribution and governance are highly concentrated: core developers account for a large majority of contributions (Mockus 2002). These patterns matter because many pull-request decisions do not involve an independent responding collaborator. Large-scale research on GitHub pull requests shows that whether the submitter and integrator are the same person is one of the most important factors explaining pull-request outcomes (Zhang 2023). I therefore construct a sample in which reciprocal human response is observable.

I begin from the Python package ecosystem. Popular Python packages are useful for this purpose because they are widely used, depend on active maintenance, and receive contributions from broader communities of users and developers. They are also organizationally consequential: many downstream projects depend on them, so review and integration decisions in these repositories affect infrastructure used well beyond the focal project.

The repository universe was constructed in several steps. First, I identified prominent PyPI projects from the annual top-project lists between 2022 and 2025. I then matched each PyPI project to its source GitHub repository using package metadata, project documentation, and manual verification. I dropped projects whose source repository could not be identified, whose active development occurred outside GitHub, or whose GitHub repository was only a mirror rather than the main site of development. I then applied repository-level filters designed to retain settings in which reciprocal effort is observable: repositories had to be public, non-archived, primarily Python-based, created before 2021, and active in human pull-request review, with at least five human-handled pull requests.

The final filters are central to the design. I retain repositories with regular reciprocal pull-request activity: pull requests had to be handled by someone other than the author, and repository-months had to contain human-handled pull requests rather than only self-merges, self-closures, or bot activity. I also require variation in AI-predicted work during the AI era, so that the analysis compares AI-predicted and non-AI-predicted pull requests within comparable collaborative settings. These filters produce a final sample of 166 repositories.

This selection strategy is not intended to produce a representative sample of all GitHub repositories. For the 166 repositories, I collect pull requests created between 2019 and 2025. I restrict the pull-request sample to cases that can be interpreted as reciprocal collaboration. I exclude pull requests opened by bots or automated dependency-update systems. I also exclude pull requests whose final disposition was made by the pull-request author. A merged pull request remains in the sample only if it was merged by someone other than its author. A closed, unmerged pull request remains in the sample only if it was closed by someone other than its author. These restrictions are

Step	Criterion	Result
PyPI universe	Annual top PyPI projects during 2022–2025.	15,000/yr
GitHub matching	Matched PyPI projects to active source GitHub repositories using package metadata, documentation, and manual verification, then collected their pull-request histories.	2,080 repos
Repository & review eligibility	Public, non-archived, primarily Python repositories created before 2021 with regular human-reviewed pull-request activity (at least five pull requests, removed bot authors).	782 repos
AI variation & pull-request eligibility	Repositories retaining both AI-predicted and non-AI-predicted pull requests, where eligible pull requests are dyadic, non-bot, non-self-disposed, and contain at least one complete Python function and at least ten lines of added code.	166 repos
Final sample	Eligible pull requests in the eligible repositories.	13,614 PRs

Notes: The table summarizes the sample-construction logic. The final sample contains 13,614 pull requests submitted by 4,907 contributors across 166 Python repositories.

Table 2: Construction of the GitHub Sample

necessary because the theory concerns effort returned by another collaborator. A self-merged or self-closed pull request does not reveal whether someone else chose to reciprocate effort.

Finally, I restrict the sample to pull requests with enough substantive Python code to classify. The classifier operates on added code in pull-request diffs, so a pull request must contain at least one complete Python function and at least ten lines of added code. This restriction removes pure documentation changes, formatting-only edits, configuration-only changes, and extremely small code changes for which AI authorship cannot be measured reliably.

The final observational sample contains 13,614 pull requests submitted by 4,907 contributors across 166 Python repositories. In this sample, 64% of pull requests are authored by non-maintainers, with a repository-level mean of 76%. By construction, no merged pull request is self-merged: every merged pull request is merged by someone other than its author, and every closed, unmerged pull request is closed by someone other than its author. Seventy-seven percent of pull requests receive at least one formal review. The sample therefore captures the setting the theory requires: submitted work that must receive effort from another collaborator before it can become part of the project.

Table 3 reports descriptive statistics. The average repository contributes 82 pull requests, though the distribution is highly skewed: several large infrastructure projects account for a substantial share of activity. The average repository-level rejection rate is 17.1%, and the average repository-level AI-predicted share is 9.2%. At the pull-request level, 21.4% of pull requests are rejected, and 11.0% are classified as AI-predicted. AI-predicted pull requests differ from other pull requests in several

Panel A: Repository-Level (N = 166 repos)	Mean	Median	Std. Dev.	Min	Max
Pull requests per repository	82.01	20.00	221.87	5	2,463
Contributors per repository	30.25	11.50	99.44	3	1,237
Avg. rejection rate (repo-level)	0.171	0.137	0.155	0.000	0.813
Avg. AI-predicted PR share (repo-level)	0.092	0.057	0.122	0.000	0.720

Panel B: Pull-Request Level (N = 13,614 PRs)

	Full sample		By predicted source		
	Mean	Std. Dev.	Non-AI	AI-predicted	Difference
Outcomes (mean)					
Rejected	0.214	0.410	0.173	0.544	0.371***
Time to close (days)	42.21	127.04	40.8	55.0	14.2***
Inline comments	0.972	1.392	1.02	0.62	−0.40***
General comments	1.123	2.156	1.12	1.14	0.02
Rejection after first comment	0.057	0.232	0.048	0.129	0.081***
Constructive feedback	0.791	0.339	0.801	0.713	−0.087***
AI-predicted PR share	0.110	0.313			
Submission characteristics (median)					
Initial lines added			51.00	69.00	18.00**
Initial lines deleted			3.00	1.00	−2.00***
Initial files changed			2.00	2.00	0.00***
Cyclomatic complexity (mean)			2.79	2.66	−0.12***
Cyclomatic complexity (max)			13.00	10.00	−3.00***
Maintainability index			41.65	44.43	2.78
Observations	13,614		12,115	1,499	

Notes: Panel A reports repository-level descriptive statistics. Panel B reports pull-request-level statistics for the final observational sample. "AI-predicted" indicates that the pull request is classified by the code-based detection model as containing AI-generated code and should be interpreted as suspected rather than verified AI use. Outcomes report means. Submission characteristics report medians because their distributions are highly skewed. Significance for the difference column is from t-tests for outcomes and Wilcoxon rank-sum tests for submission characteristics: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

Table 3: Descriptive Statistics for GitHub Pull Requests

observable ways: they are rejected more often in the raw data, receive fewer inline comments, and take longer to close. They also differ in size and structure, adding more lines, deleting fewer lines, and showing slightly lower measured cyclomatic complexity. These raw differences are descriptive rather than causal; the empirical analysis below compares AI-predicted and non-AI-predicted pull requests within repositories and months while controlling for contribution size and observable code characteristics.

4.2 Timeline Reconstruction

I reconstruct each pull request from its event history. For every pull request, I collect the opening metadata, initial code diff, commits, issue comments, review comments, formal reviews, check runs where available, closing or merge event, and the identities and timestamps of actors involved in each event. I then order all events chronologically and classify each event by actor type and action type. Author actions are separated from non-author actions. Bot and automation events are separated from human events. This distinction is essential because the theory concerns reciprocal effort returned by human collaborators, not automated activity or the contributor's own follow-up work.

The reconstructed timeline allows me to measure outcomes that are not available from the pull-request record alone. First, it identifies the human collaborator responsible for final disposition: the actor who merged or closed the pull request. Second, it distinguishes early rejection from rejection after substantive review. A pull request is coded as early rejected when it is closed without being merged before formal code review begins. Third, it separates high-effort and low-effort responses. Inline comments are code-specific comments attached to changed lines and require the collaborator to engage with the submitted code. General comments occur in the conversation thread and can be made without line-level review. Fourth, the timeline identifies whether feedback is returned before or after the contributor has an opportunity to revise, which helps avoid conditioning on post-response behavior.

This reconstruction is important for interpretation. If I observed only whether a pull request was eventually merged or closed, I would miss the process through which reciprocal effort is allocated. A final rejection may occur after extensive back-and-forth, or it may occur before review begins. These cases have different meanings for the theory. The first may involve substantial reciprocal effort that ultimately fails to produce integration. The second indicates that the submitted work did not attract high-effort review in the first place. Timeline data allow me to distinguish these cases and to observe the composition of the collaborator's response.

4.3 Measuring AI-Predicted Work

True AI use is not observed in GitHub pull requests. Contributors are not required to disclose whether they used a coding assistant, and a pull request does not contain a verified field for AI assistance. I therefore measure suspected AI assistance from the submitted code. Specifically, I apply a fine-tuned ModernBERT classifier to added Python functions in the initial pull-request diff.

The classifier returns, for each added function f in pull request i, a probability $\hat{p}_f = \Pr(\text{machine-generated})$. I summarize each pull request by its line-weighted AI density:

$$\text{AIscore}_i = \frac{\sum_{f \in i} \hat{p}_f \ell_f}{\sum_{f \in i} \ell_f},$$

where ℓ_f is the number of added lines in function f. The line-weighted measure captures how much

Validation sample	N	Correct classification rate
AI-generated code		
AI generators represented in training distribution	58,209	95.5%
AI generators not represented in training distribution	35,447	81.6%
All held-out AI-generated code	93,656	90.2%
Human-written code		
Human code, v4 test set	83,680	98.1%
Human code, January 2026 validation set	19,378	98.8%

Notes: The table summarizes held-out classifier performance. For AI-generated samples, the correct classification rate is the percentage classified as AI-generated. For human-written samples, the correct classification rate is the percentage classified as human-written. "Represented in training distribution" indicates whether the model family was present in the v4 training distribution. Full model-by-model validation results are reported in the Appendix.

Table 4: Classifier Validation: Detection of AI-Generated Code and Specificity on Human Code

of the submitted code reads as AI-generated, while avoiding the problem that a single high-scoring function in a large diff can mechanically classify the entire pull request as AI-assisted. The main analysis uses a binary indicator, AI_{ijt}, equal to one when at least 30% of the added code lines fall in functions the classifier labels machine-generated. Robustness checks use the continuous density measure AIscore_i.

The classifier is trained on human and synthetic AI examples. Human training examples are drawn from GitHub commits before June 2021, before the widespread availability of AI coding assistants. AI examples are generated using a describe-and-generate pipeline repeated across a diverse set of language models, reducing the risk that the classifier learns the style of only one model. Consistent with recent guidance on using language models to annotate management data, I define the construct in advance, keep the classifier separate from the focal outcomes, and validate its interpretation against independent evidence (Carlson and Burbano, 2026). Full details of the classifier architecture, training data, generation pipeline, prompting procedure, and validation are reported in the Appendix.

Table 4 summarizes the classifier's held-out performance. The classifier detects 95.5% of AI-generated code from model families represented in the training distribution and 81.6% of AI-generated code from model families not represented in the training distribution. Across all held-out AI-generated samples, the weighted detection rate is 90.2%. Specificity on human-written code is also high: the classifier correctly classifies 98.1% of human code in the v4 test set and 98.8% of human code in an independent January 2026 human-code validation set.

This validation has two implications for the field analysis. First, the low false-positive rate on human code reduces the concern that the classifier is simply labeling ordinary human code as AI-assisted. Second, the lower detection rate on some out-of-distribution models means that the measure

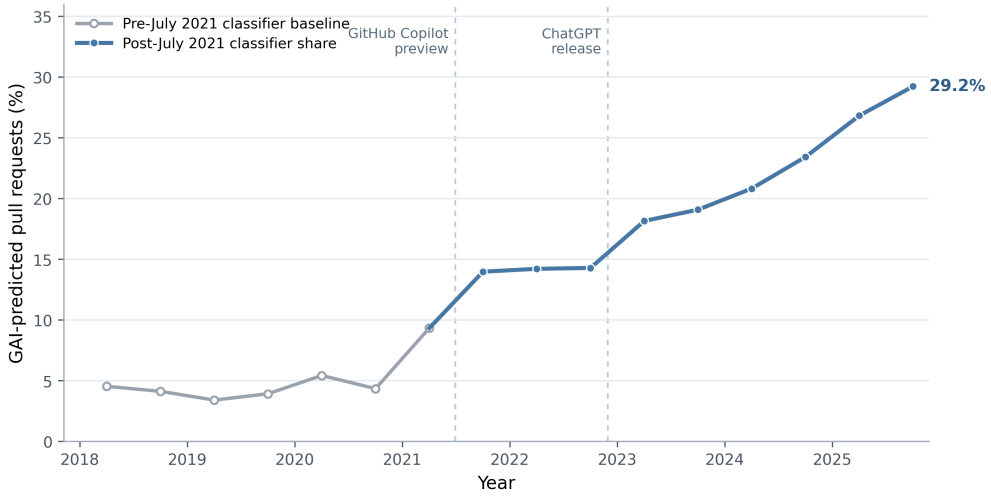


Figure 2: Growth in the share of AI-predicted pull requests. Points show half-year averages of the classifier-based series. Values before July 2021 are shown in gray as an estimate of the classifier's baseline false-positive behavior and are recoded as non-AI in the empirical analysis. The post-July 2021 series is shown in blue.

is likely to miss some genuinely AI-assisted code, especially from newer or stylistically different generators. I therefore interpret the field measure cautiously as *AI-predicted* or *AI-suspected* work, not verified AI use. The measure identifies pull requests whose code bears detectable signatures of AI assistance.

Figure 2 plots the monthly share of AI-predicted pull requests. The raw classifier output is low before the release of widely available AI coding tools and rises sharply after the release of ChatGPT in late 2022, reaching approximately 30% by 2025. In the analysis sample, pull requests classified as AI-predicted before July 2021 are recoded as non-AI. The pre-2021 raw positives are retained in the figure to show the classifier's baseline false-positive behavior.

4.4 Outcomes: Reciprocal Effort

The outcomes measure the effort returned by project collaborators.

The first is the decision. *Rejected* equals one when a pull request is closed without being merged. Rejection is the most consequential response because the submitted work does not become part of the project. Rejection alone, however, does not fully reveal how much effort was returned. A pull request may be rejected after extensive back-and-forth, or it may be rejected before formal review begins. I therefore also measure *early rejection*: rejection before formal code review. Early rejection is a sharper measure of low-effort disposition because it occurs before the contributor has an opportunity to make additional effort visible through revision, dialogue, or response to review.

The second is the type of feedback. I distinguish inline comments from general comments. Inline comments are line-level comments attached to specific changed code. They are the closest

observable proxy for high-effort response because they require the responding collaborator to engage with the submitted code at a specific location. General comments occur in the pull-request discussion thread. They may be useful, but they are less tightly tied to code-level review and can be used to ask broad questions, delay, redirect, or dispose of the work without line-level review. I also measure the constructiveness of feedback, defined as the share of non-author comments classified as constructive using the comment-level coding procedure described in the appendix.

This outcome structure follows directly from the theory. If effort opacity produces simple disengagement, then all forms of response should decline. If it produces recomposition, high-effort responses should decline while lower-effort responses persist or increase. The distinction between inline comments and general comments allows me to test this prediction in the field.

5 Results

5.1 Rejection and Early Rejection

I estimate the following specification:

$$Y_{ijt} = \beta \, \mathrm{AI}_{ijt} + \gamma X_{ijt} + \alpha_j + \delta_t + \epsilon_{ijt}, \tag{1}$$

where Y_{ijt} is an outcome for pull request i submitted to repository j in month t. AI_{ijt} is an indicator equal to one when the submitted code is classified as AI-predicted. The coefficient β captures the conditional association between AI-predicted status and the response the pull request receives from other project members.

The vector X_{ijt} contains controls measured at initial submission, before subsequent review or revision. These include the size and scope of the pull request: initial additions, initial deletions, and changed files. They also include observable code characteristics: mean cyclomatic complexity, maximum cyclomatic complexity, and the mean maintainability index. Cyclomatic complexity captures the number of linearly independent paths through code (McCabe, 1976). The maintainability index captures maintainability-related features of the code (Oman and Hagemeister, 1992). These controls address the most direct alternative explanation: that AI-predicted pull requests receive different responses because they are larger, more complex, or less maintainable.

Repository fixed effects, α_j, absorb time-invariant differences across projects, including baseline strictness, coding conventions, governance norms, and review culture. Month fixed effects, δ_t, absorb common time trends, including the diffusion of AI coding tools and platform-wide changes in reviewing behavior. The preferred specifications therefore compare AI-predicted and non-AI-predicted pull requests submitted to the same repository in the same calendar period. Standard errors are clustered by contributor to allow arbitrary correlation across pull requests submitted by the same author.

Binary outcomes are estimated using linear probability models, and count outcomes are estimated using OLS. I use linear models so coefficients are directly interpretable in levels and because

the specifications include high-dimensional fixed effects. They show whether, among observably similar pull requests submitted to the same project in the same month, those whose code appears AI-assisted receive a different pattern of reciprocal effort. The key empirical question is whether the predicted pattern appears in the field and persists after conditioning on observable features of the work.

Table 5 reports the results. Panel A reports rejection outcomes, and Panel B the composition of the response. Column 1 includes size controls. Column 2 adds repository and month fixed effects. Column 3 adds observable code-quality controls, allowing the reader to see whether the AI-predicted gap is explained by contribution size, complexity, or maintainability.

Hypothesis 1 makes two claims: that AI-attributed work receives less developmental review effort, and that it is more likely to be rejected. Panel A tests the second, and Panel B the first.

The association in Panel A is large and consistent across both measures. With repository and month fixed effects, AI-predicted pull requests are 32.2 percentage points more likely to be rejected. This raises the predicted rejection rate from the 17.3% baseline for non-AI-predicted pull requests to roughly one in two. The magnitude is substantial, and the analyses below examine how much of it observable differences in the work can account for. The same pattern appears at first contact: AI-predicted pull requests are 4.6 percentage points more likely to be rejected before formal review begins, nearly double the 4.9% baseline for non-AI-predicted submissions. Early rejection is the outcome the theory ties most directly to screening, since it disposes of a contribution before the reviewer has incurred the cost of learning more about it. It is therefore evidence on the effort side of Hypothesis 1 as well as on the decision side.

Adding observable code-quality controls barely changes the estimates. The overall rejection coefficient moves from 32.2 to 31.8 percentage points, and the early-rejection coefficient moves from 4.6 to 4.5 percentage points. The rejection gap is therefore not explained by AI-predicted pull requests being larger, more complex, or less maintainable on the measured dimensions. This does not establish that AI-predicted and non-AI-predicted work are identical on unobserved dimensions of quality, and the theory does not require it. The claim is that AI-attributed work is less likely to receive the process through which uncertain work becomes acceptable, not that reviewers reject work they know to be equally good.

5.2 Results: Reviewer Effort

Rejection shows that AI-predicted work is less likely to receive integration. The stronger test of the theory concerns the composition of response, since effort opacity predicts not simply less response but a shift in the effort level of the response. Panel B tests this developmental-effort clause, and the pattern matches the recomposition the theory predicts rather than uniform withdrawal. With repository and month fixed effects, inline comments fall by 0.25, a 25% reduction relative to the baseline of 0.97. These are the costliest form of response, because they require the reviewer to understand the submitted code. The share of feedback that is constructive falls by 4.1 percentage

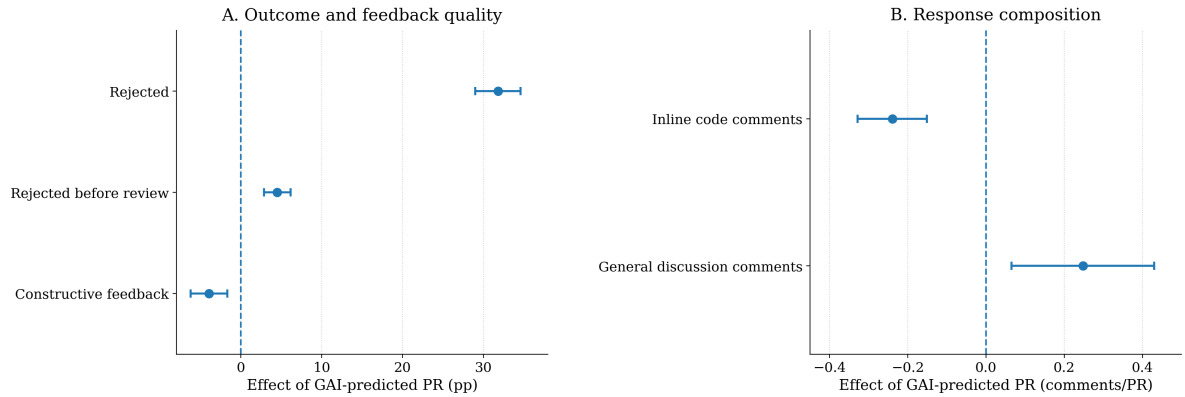


Figure 3: AI-predicted pull requests receive less high-effort reciprocity. Panel A plots percentage-point estimates for rejection, early rejection, and the share of feedback classified as constructive. Panel B plots estimates for the number of inline code comments and general discussion comments. Estimates come from the preferred specifications with repository fixed effects, month fixed effects, contribution-size controls, and observable code-quality controls. Error bars show 95% confidence intervals. Standard errors are clustered by author.

points. General comments, which a reviewer can produce without forming that understanding, rise by 0.25, a 22% increase. Adding observable code-quality controls leaves all three estimates essentially unchanged. AI-predicted pull requests therefore receive fewer of the high-effort review comments that help a contribution develop.

5.3 Is AI bias driving rejection?

Maintainers may respond to disclosed AI use, to characteristics of the submitted code, or to both. I examine these possibilities by comparing responses to AI disclosure and the code-based classifier prediction. This is a useful test even though AI coding is increasingly commonplace: 92% of developers at large U.S. companies use AI coding tools, 62% of Stack Overflow respondents used them in 2024, and 76% were using or planning to (GitHub, 2023a; Stack Overflow, 2024), and by early 2026 Google reported that AI wrote about half of its code (Alphabet, 2026). Still, open-source communities have their own norms, so I test the idea directly by asking a simple question: are pull requests that visibly carry an AI credit rejected more often?

To do this, I use a sample of 21,563 more recent pull requests from the same repositories, none of which appear in the main analysis. The sample is more recent compared to the main observational study since AI coding tools that append a co-author credit to the commits they produce by default are relatively new. In the sample repositories they first appear in mid-2025 and remain below 2% of pull requests through the end of that year, before surging to over 20% by spring 2026. If maintainers penalized AI use based on a simple AI aversion account, these AI labeled contributions are exactly where the penalty should appear. However, AI-labeled pull requests are

	Dependent variable indicated by row		
	(1)	(2)	(3)
Panel A: Rejection			
Rejected			
AI-predicted PR	0.3708***	0.3216***	0.3181***
	(0.0178)	(0.0144)	(0.0143)
R^2	0.081	0.233	0.235
Rejected before formal review			
AI-predicted PR	0.0614***	0.0460***	0.0449***
	(0.0090)	(0.0084)	(0.0084)
R^2	0.008	0.083	0.084
Panel B: Composition of reciprocal effort			
Inline code comments			
AI-predicted PR	−0.3961***	−0.2454***	−0.2395***
	(0.0473)	(0.0448)	(0.0451)
R^2	0.008	0.239	0.239
General discussion comments			
AI-predicted PR	0.0138	0.2514***	0.2477***
	(0.0874)	(0.0940)	(0.0930)
R^2	0.000	0.112	0.112
Share of feedback constructive			
AI-predicted PR	−0.0874***	−0.0406***	−0.0397***
	(0.0162)	(0.0117)	(0.0116)
R^2	0.007	0.183	0.184
Repository fixed effects	No	Yes	Yes
Month fixed effects	No	Yes	Yes
Size controls	Yes	Yes	Yes
Quality controls	No	No	Yes
Observations	13,614	13,614	13,614

Notes: Each coefficient comes from a separate regression. Rejected and rejected before formal review are estimated using linear probability models. Inline and general comments are estimated using OLS count models. The constructive-feedback outcome is the share of comments classified as constructive. Size controls include initial additions, deletions, and files changed. Quality controls include mean and maximum cyclomatic complexity and mean maintainability. Standard errors clustered by contributor are reported in parentheses. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

Table 5: AI-Predicted Pull Requests and Reciprocal Effort

not rejected at higher rates compared to pull requests without the AI label (Table 13). The estimate is negative in every specification, and when I compare the same contributor's AI-labelled and non-AI-labelled pull requests (column 4), the estimate is almost exactly zero. The pattern holds for first-time contributors as well (column 3), the group facing the highest rejection in the main observational study.

In an additional analysis in the Appendix, I show that AI disclosure itself might not be driving the rejections. I find that code disclosed as AI-assisted is not rejected at higher rates that code not disclosed as AI-assisted. To account for the fact that the AI label can be removed by contributors, I leverage a natural experiment, detailed in the Appendix (Section A.2.3). A software bug in April 2026 briefly created an "AI-assisted" label to be stamped on pull requests automatically, and whether the label appeared depended on when a contributor's editor happened to update to the new version, not on the contributor opting in to add the trailer or even using AI to write the code. This natural experiment affected a different set of repositories from the main sample, making it an independent test. The results show that contributions with an AI-coauthor trailer were not rejected any more often than contributions that did not have the AI-coauthor trailer. Taken together, the evidence suggests that AI aversion might not be driving the rejections.

5.4 Heterogeneity

The previous sections show that AI-predicted pull requests receive less high-effort reciprocity on average. This section provides evidence for where the penalty is largest. This heterogeneity analysis is informed by the predictions that the theory makes about where the effects of effort opacity might be strongest. First, effort opacity should have a stronger effect when collaborators face a larger queue of work since they have stronger reasons to reserve high effort for work that is worth it. Second, effort opacity should matter more when collaborators have little prior information about the contributor. If a collaborator already knows that a contributor usually invests effort, the current work does not have to carry the full burden of showing effort. But when the contributor is unknown, the submitted work becomes the main basis for deciding how much effort to return.

I measure repository activity based on whether the repository is in the top decile based on the distribution of the total pull-request volume across repositories. This is a proxy for settings in which maintainers face many competing claims on their effort. The second is contributor status, measured in several ways: whether the pull request is the contributor's first pull request to the repository, whether the contributor has a prior merged pull request in the repository, and whether the maintainer has previously merged work from the same contributor.

5.4.1 Repository Size

Effort opacity should be most costly when collaborators must triage. In high-activity repositories, incoming work creates a larger queue. Collaborators cannot return high effort to every pull request, so they must decide which work deserves close reading, specific feedback, and integration effort.

	(1) Rejected	(2) Any review comment	(3) General comments	(4) Share constructive
AI-predicted PR	0.2314***	−0.2424***	0.0786	−0.0200
	(0.0218)	(0.0617)	(0.1476)	(0.0180)
AI-predicted PR × High-activity repository	0.1290***	−0.0218	0.3495†	−0.0418†
	(0.0305)	(0.0939)	(0.1905)	(0.0240)
Repository FE	Yes	Yes	Yes	Yes
Month FE	Yes	Yes	Yes	Yes
Controls	Yes	Yes	Yes	Yes
Observations	13,614	13,614	13,614	13,614
R^2	0.237	0.239	0.115	0.184

Notes: Each column reports a separate regression with repository and month fixed effects. "High-activity repository" is a time-invariant indicator for repositories in the top decile by total pull requests. Its main effect is absorbed by repository fixed effects. The interaction is identified because AI-predicted status varies across pull requests within repositories. Controls are initial additions, deletions, changed files, mean and maximum cyclomatic complexity, and mean maintainability index. Standard errors clustered by author in parentheses.

Significance: $^{\dagger}p < 0.10$, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

Table 6: Heterogeneous Association of AI-Predicted Work by Repository Activity

If AI-predicted work makes human effort harder to infer, then the penalty should be larger in these high-activity settings.

Table 6 reports the results. The interaction between AI-predicted status and high repository activity is positive and large for rejection. In lower-activity repositories, AI-predicted pull requests are 23.1 percentage points more likely to be rejected. In high-activity repositories, the penalty is an additional 12.9 percentage points larger. The total rejection penalty in high-activity repositories is therefore approximately 36.0 percentage points.

The response-composition results point in the same direction. The interaction is positive for general comments and negative for the share of constructive feedback, though both are estimated less precisely. The interaction for any review comment is small and not statistically distinguishable from zero. Thus, the clearest evidence is on disposition: when reciprocal effort is scarce, AI-predicted work is especially likely to be rejected. The broader pattern is consistent with effort recomposition, with suggestive evidence that high-activity repositories also shift more strongly toward lower-effort general response and away from constructive feedback.

These results support the scarcity prediction. The AI-predicted penalty is not evenly distributed across collaborative settings. It is amplified where responding collaborators face larger queues of work and must ration high effort more aggressively.

5.4.2 Contributor Status

The second prediction concerns what collaborators already know about the contributor's effort. When a contributor is unknown, the submitted work carries most of the burden of showing effort. If that work appears AI-assisted, collaborators have little alternative information to rely on. The penalty should therefore be largest for contributors with no observed track record. Prior standing can also supply recognition cues that affect how audiences evaluate otherwise comparable work (Azoulay, Stuart, and Wang, 2014), making the absence of a track record especially consequential when effort is opaque.

Table 14 tests this prediction using three measures of contributor standing. Column 1 uses an indicator for whether the pull request is the contributor's first observed pull request to the repository. The interaction between AI-predicted status and first-time contributor status is positive: first-time contributors face an additional 4.5 percentage point rejection penalty beyond the baseline AI-predicted penalty. The estimate is modest and statistically weaker than the repository-activity result, but it is consistent with the theory. The penalty is largest where the collaborator has no prior information about the contributor's effort.

The first-time measure is conservative. It is based on the contributor's first observed eligible pull request in the sample, not necessarily the contributor's first-ever interaction with the repository. Contributors may have prior documentation changes, issues, comments, or pull requests outside the eligible code sample. This measurement error should attenuate the interaction, making the estimate best interpreted as a lower bound on the penalty for truly unknown contributors.

Columns 2–4 examine whether additional forms of standing reduce the penalty. Column 2 tests whether the contributor has a prior merged pull request in the repository. The interaction is close to zero and statistically insignificant. This suggests that the extensive margin of being entirely new matters more than accumulated repository standing, at least as measured here. Column 3 tests collaborator-specific familiarity: whether the responding collaborator has previously merged work from the same contributor. The interaction is large and negative, suggesting that familiarity may reduce the penalty. However, Column 4 adds responding-collaborator fixed effects, absorbing stable differences in collaborator leniency. Once this stricter comparison is used, the moderation disappears. The familiarity result therefore appears to reflect which collaborators handle familiar contributors, rather than a clean familiarity effect.

Taken together, the results are consistent with the theory but more nuanced than a simple reputation story. The cleanest evidence is at the boundary between no prior information and some prior information: the reciprocity penalty for AI-predicted contributions is larger for first-time contributors. Once a contributor has any observed history, additional measures of status do not consistently reduce the penalty. This pattern fits the effort-opacity logic. Prior status can partly substitute for effort visible in the submitted work, but the largest effects are borne by contributors whose effort cannot be inferred from either the focal work or a prior track record.

The heterogeneity results are consistent with the boundary implications of the theory. AI-

predicted work is penalized most strongly where collaborators must ration scarce effort and, more modestly, where they have the least prior information about the contributor. The repository-activity result is especially strong: the rejection penalty is substantially larger in high-activity repositories. The contributor-standing result is weaker but directionally consistent: first-time contributors face an additional AI-predicted penalty, while deeper familiarity measures are less robust.

5.5 Robustness Checks

Because the GitHub study is observational, the estimates may reflect differences other than AI assistance. I focus on three central concerns. First, AI-predicted contributions may simply be lower quality. Second, the classifier may detect code patterns that human reviewers cannot notice. Third, reviewers may penalize any disclosed use of AI, regardless of the code itself. I address each concern below and report the complete analyses in Appendix A.2.

The main models already account for contribution size, code complexity, and maintainability. I also examine the 635 pull requests for which automated test results are available. AI-predicted pull requests are somewhat less likely to pass every test: 42.7% pass, compared with 51.9% of other pull requests. This confirms that the two groups are not identical in observed quality. However, adding test results to the model changes the estimated rejection difference by only 0.5 percentage points. Even among pull requests that pass every observed test, AI-predicted contributions are rejected 57.0% of the time, compared with 29.3% of other contributions. Automated tests cannot measure architectural fit, long-term maintainability, or every other aspect of quality. Nevertheless, the results show that observed test failures explain little of the rejection difference.

5.5.1 Does the AI classifier reflect a software developer's judgment?

The observational study also assumes that the classifier detects code features that developers could notice. I examine this assumption in a preregistered study with software developers. Participants evaluated five code snippets drawn from matched human-written and AI-generated examples and classified each as human- or AI-generated. After applying the preregistered data-quality rules, the analysis contains 126 developers and 630 decisions.

As Figure 4 shows, developers correctly identified the source of the code 64.0% of the time, above the 50% expected by chance ($p < .001$). They correctly identified 72.3% of human-written snippets and 56.6% of AI-generated snippets. Detection was therefore meaningful but imperfect. More importantly, developers were more likely to classify code as low effort when they believed it was AI-assisted. The classifier is therefore not capturing a signal that is entirely invisible to human evaluators; the same broad distinction can be detected by developers and is connected to their judgments of human effort.

This study does not show that the maintainers in the observational study consciously identified particular pull requests as AI-generated. It establishes the narrower point needed for the study: code produced with AI can contain signals that developers notice, and those signals are associated with

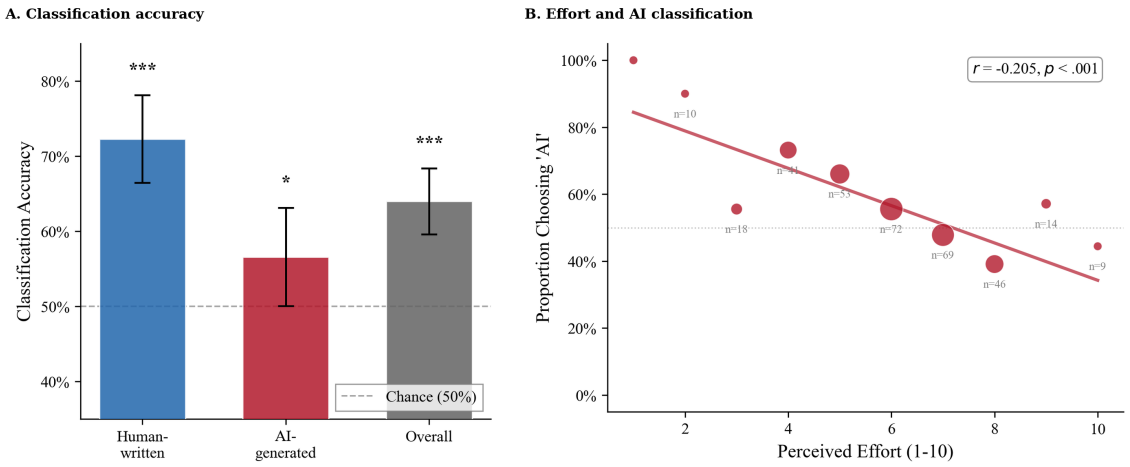


Figure 4: **Developers can partly identify AI-generated code, and their judgments are related to perceived human effort.** *Panel (a)* reports classification accuracy. The dashed line marks the 50% expected by chance. *Panel (b)* shows, among AI-generated snippets, how often developers classified the code as AI-generated at different levels of perceived human effort. Larger points represent more observations.

lower perceived human effort. Full details of the stimuli, participant screening, and analysis are reported in the Appendix.

5.5.2 Additional checks

The Appendix provides additional robustness tests. Across specifications accounting for contributor experience, project and reviewer strictness, and repeated observations, AI-predicted contributions remain 11 to 16 percentage points more likely to be rejected. The pattern also appears among employees contributing to their employers' public repositories, suggesting that it extends beyond unpaid collaboration.

The accompanying descriptions provide a further check on effort cues. AI-predicted contributions have comparatively longer descriptions than non-AI-predicted contributions, but contain less substantive evidence of effort, such as reasons for decisions and testing evidence. This distinction matters for effort opacity: visible features such as length and mentions of testing need not reveal the effort behind the work. Accounting for description quality leaves the rejection gap essentially unchanged, while descriptions containing more substantive evidence of effort are themselves associated with lower rejection.

Finally, I examine whether code from rejected contributions subsequently appears in the project, hinting at the rejection of potentially useful work. Lines newly proposed in rejected pull requests later appear verbatim in the codebase for 7.7% of AI-predicted rejections and 9.7% of non-AI-predicted rejections ($p = .18$). This identifies rejected AI-predicted contributions containing code that the project later uses. It does not establish equal quality across the two groups, but suggests

that rejection can occur even when a contribution contains code useful enough to be used in the future.

Together, these checks show that the rejection gap persists after accounting for several observable differences in contributions and contributors. Unobserved differences may still contribute to the pattern.

6 Mechanism: Effort Opacity

The observational study documents the pattern anticipated by *effort opacity*: pull requests whose code appears AI-assisted receive less high-effort reciprocal review. That study cannot, however, provide evidence for a clear mechanism driving this reduced reciprocity. The experiment examines an implication of the proposed mechanism: whether making substantial creator effort legible increases review priority even when AI assistance is disclosed. I mirror the context of open source software to create a custom code review platform where I hold the code quality constant and vary AI disclosure and effort legibility. This 2x2 design is implemented in a within-subjects study where each subject sees 4 code contributions.

The experiment tests whether making contributor effort legible increases review priority while AI assistance remains disclosed. The code within each stimulus is unchanged across conditions. However, the accompanying description may change perceptions of the code's quality or the contributor's competence. I therefore measure these perceptions alongside perceived contributor effort to examine the pathways associated with review priority.

6.0.1 Study Design

Participants with programming experience evaluated four simulated pull requests in a within-subjects 2×2 design crossing AI disclosure (disclosed against not disclosed) with effort legibility (high against low). A 4×4 Latin square counterbalanced stimuli across conditions. An initial pilot was run to test the design and the results form the confirmatory study are reported in this section. This confirmatory study uses the preregistered standard-cleaned sample of 83 participants, yielding 332 contribution-level observations. In the high-effort legibility conditions the description made the contributor's effort legible by reporting the problem diagnosed, the reasoning behind the change, the checks performed, and the contributor's ownership of the result. In the low-effort legibility conditions the description supplied comparatively little evidence of human effort, as in "Fixed type handling. Tests pass."

The primary dependent variable is a forced ranking over the four contributions the participant has just reviewed. The four pull requests remain on screen, each shown with its title, repository, contributor, and description, and the participant places these four objects in order of the review priority each will receive. The instruction reads, "Rank the 4 pull requests you just reviewed by

priority. A higher-priority PR is one you find more WORTH your time and review effort as a maintainer. A lower-priority PR is one you find less worth your effort."

This is a deliberate choice, motivated by the constraints that reviewers face. The outcome used here asks the reviewer to divide her own scarce attention among four submissions that require review. This is the correct outcome to test the mechanism since review effort draws on limited capacity, and effort spent on one contribution is unavailable to another (Ocasio, 1997; Piezunka, 2015), so the reviewer facing a queue must decide which of the contributions are worth her reciprocal review effort.

The forced-choice format provides a better test for the theory compared to a rating scale since the latter would permit a participant to assign high priority to all four contributions, which makes it less relevant to the observational study where a maintainer's effort is budgeted and effort given to one submission is withdrawn from another. Ranking also removes between-participant differences in scale use by construction, and it yields a within-person comparison in which the reviewer is held constant, and the stimuli are balanced across conditions by the Latin square.

Additionally, the results show that within participants, the ranking given to contributions is associated with merge acceptance ($\rho = -.30$), with willingness to work with the contributor ($\rho = -.42$), and with perceived contributor effort ($\rho = -.40$), all $p < 10^{-7}$. Secondary outcomes, collected per trial on rating scales, comprise perceived code quality, perceived contributor effort, perceived contributor competence, willingness to work with the contributor, and merge acceptance likelihood. Two of these, willingness to work with the contributor and merge acceptance, are rated freely for each pull request and so carry none of the ranking's ipsative constraint.

Because the design is fully within-subjects with one trial per cell, every contrast has a paired form. I report the difference in rank points alongside Cohen's d_z, the standardized paired difference, and where informative d_{av}, which standardizes by the average of the two cell standard deviations and is less sensitive to the within-person correlation. Lower rank values indicate higher review priority; figures are signed so that positive values indicate a gain in priority.

A sample of 83 may appear small, and it is worth being explicit about what a design of this form can and cannot resolve. Because each participant contributes all four cells, the relevant unit for power is the paired difference rather than the participant, and the 83 participants yield 332 contribution-level observations. At $N = 83$, $\alpha = .05$ two-sided, the design detects a paired effect of $d_z = 0.31$ with 80% power and $d_z = 0.36$ with 90% power; the sample size was set in advance at 80 for 90% power against the effect anticipated from earlier work. Achieved power for the effects actually observed is accordingly high for the tests the argument rests on, at .97 for the contrast among AI-disclosed contributions and .74 for the corresponding contrast among undisclosed contributions, and low for effects near $d_z = 0.2$, at .49 for the contrast that crosses both factors. This is worth stating plainly, since it means the design is well powered for precisely the comparisons on which conclusions are drawn and poorly powered for those reported below as suggestive. To put the efficiency of the within-subjects arrangement in perspective, detecting the same effect on willingness to work with the contributor in a between-subjects design, given the

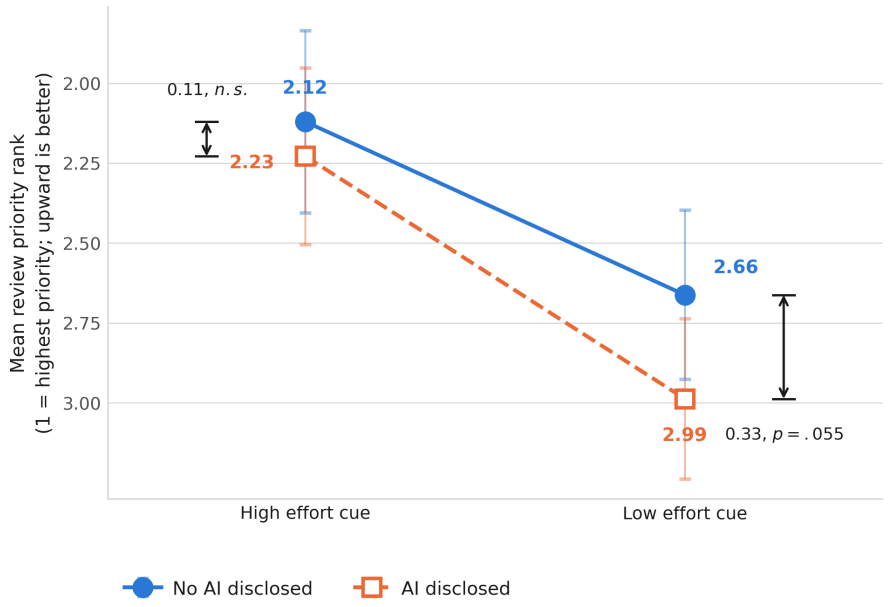


Figure 5: Mean review priority by AI disclosure and effort legibility. The vertical axis is inverted so that upward indicates higher review priority. Brackets report the difference associated with disclosure at high or low effort legibility. Error bars are within-subject confidence intervals; because the design is fully within-subjects and the tests reported in the text are paired, between-subject intervals would overstate the separation of conditions relative to the comparisons that bear on them.

observed within-person correlation of $r = +.27$, would require roughly 40 participants per cell, or about 159 in total.

6.0.2 Effort Legibility and the Allocation of Review Effort

The manipulation appears to have operated as intended: participants rated high-effort descriptions as substantially more detailed than low-effort descriptions ($b = 2.42$, $d_z = 1.03$, $p < .001$), while AI disclosure had a considerably smaller negative effect on perceived description detail ($b = -0.28$, $d_z = 0.28$, $p = .012$).

Figure 5 displays the four cell means. High-effort-legibility contributions received higher mean review priority under both disclosure conditions, while AI-disclosed contributions with low effort legibility received the lowest priority. Making contributor effort legible increased review priority even when AI use remained disclosed. AI-disclosed contributions with high effort legibility received a mean rank of 2.23, compared with 2.99 under low effort legibility (difference = −0.76 rank points; $d_z = 0.43$, $p < .001$, 95% CI $[-1.15, -0.37]$).

Figure 6 shows how these mean differences translate into the distribution of review priority. Low-effort-legibility, AI-disclosed contributions were given the lowest priority more frequently than any other condition - 36.1% of participants placed them fourth, compared with 25.3% for low-effort-

Equivalence bound	p_{TOST}	Conclusion
$d_z = \pm 0.20$	.106	not supported
$d_z = \pm 0.25$	.045	supported
$d_z = \pm 0.30$	.016	supported
$d_z = \pm 0.35$	.005	supported

Table 7: Equivalence tests on the parity contrast (high-effort AI-disclosed against high-effort undisclosed) across candidate bounds. Observed difference: $d_z = +0.06$.

legibility contributions without AI disclosure and 19.3% for each high-effort-legibility condition. More broadly, 74.7% of participants placed low-effort-legibility, AI-disclosed contributions in the bottom two positions, compared with 34.9% for high-effort-legibility, AI-disclosed contributions. These descriptive distributions illustrate the allocation pattern underlying the mechanism: AI-disclosed work with low effort legibility is concentrated near the bottom of the reciprocal review-priority ordering.

In addition, an AI-disclosed contribution with high legible effort received somewhat higher review priority than a non AI-disclosed contribution with low legible effort, with mean ranks of 2.23 and 2.66 respectively, a difference of −0.43 rank points ($d_z = 0.24$, $d_{av} = 0.40$, $t(82) = -2.23$, $p = .029$).[1] Reviewers do not appear to have ranked contributions by AI use alone. Offered a choice between AI-assisted work with high effort legibility and work with low effort legibility but created without AI assistance, participants were more likely to reciprocate their scarce review effort toward the AI-assisted work.

Therefore, where work has high effort legibility, disclosed AI-assisted work receives review effort similar to high effort legible work that is not AI-assisted. Among high-effort legibility contributions, the difference between AI-assisted and non AI-assisted work is +0.11 rank points ($d_z = 0.06$), with a 90% confidence interval of [−0.21, +0.43] rank points, or $d_z \in [-0.12, +0.24]$. Taking the smallest effect size of interest to be $d_z = 0.30$, the conventional threshold for a small effect, equivalence is supported ($p_{\text{TOST}} = .016$). Since the choice of bound governs the conclusion, Table 7 reports a range of candidate bounds rather than a single figure. Equivalence obtains at any bound of $d_z \geq 0.25$ ($p_{\text{TOST}} = .045$) and is not supported at $d_z = 0.20$ ($p_{\text{TOST}} = .106$); the tightest symmetric bound the data reject at $\alpha = .05$ is $d_z = 0.24$, or 0.43 rank points.

6.0.3 Perceived Effort and Reduced Reciprocity

The theory argues that reviewers reciprocate effort in response to the effort they can infer from the work. Two alternatives suggest different carriers: if effort legibility makes the code seem better, perceived quality ought to carry the effect; if effort legibility makes the contributor seem

[1] This contrast crosses both experimental factors and was not preregistered. I report it as a planned theoretical contrast rather than as a confirmatory test.

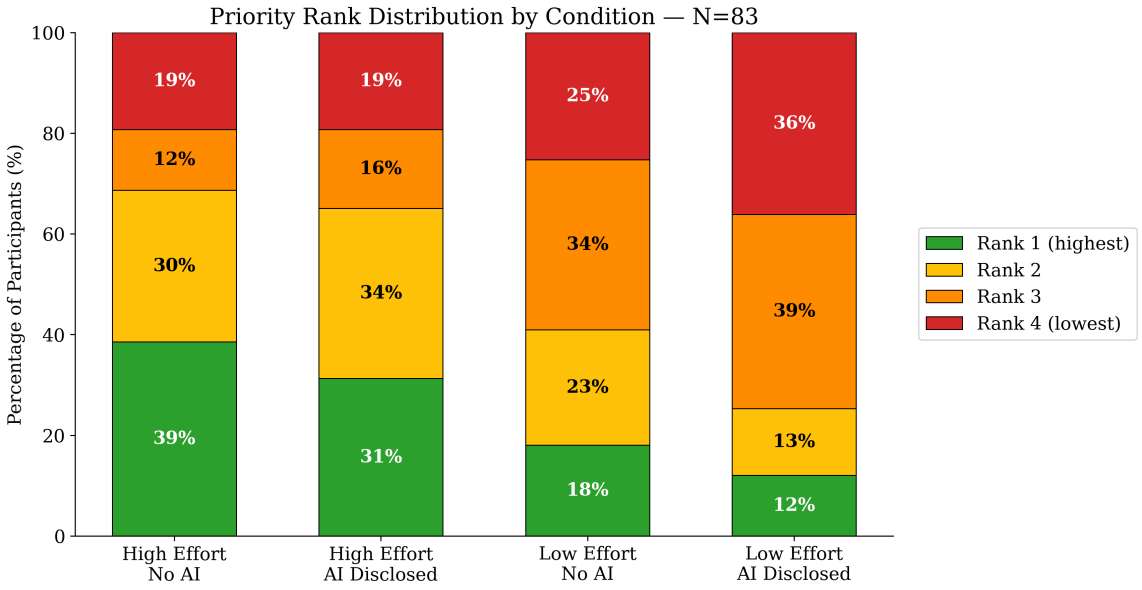


Figure 6: Distribution of Review Priority by AI Disclosure and Effort Legibility. Each bar reports the percentage of participants assigning each rank to a contribution in that condition. Rank 1 indicates the highest review priority and rank 4 the lowest. Each participant assigned ranks 1 through 4 once across the four contributions. Segment labels are rounded to the nearest percentage point.

more capable, perceived competence ought to carry it. Section 2 raises a third possibility, namely that effort legibility makes the work easier to review, in which case the effect might run through anticipated review cost rather than through perceived contributor effort. I consider first which perception covaries with the allocation at all. Because the ranking is ipsative, all three perceptions are centered within participant and standardized, and standard errors are clustered by participant. Within AI-disclosed trials, perceived contributor effort predicts review priority ($b = -0.42$, $SE = 0.12$, $p < .001$); perceived competence is a weaker and only marginal predictor ($b = -0.24$, $SE = 0.12$, $p = .053$); and perceived code quality does not predict it ($b = +0.12$, $SE = 0.12$, $p = .32$). Pooling all trials, the coefficient on perceived code quality is close to zero ($b = -0.01$, $SE = 0.10$, $p = .91$). Perceived effort remains a significant predictor after conditioning on AI disclosure ($b = -0.37$, $SE = 0.12$, $p = .001$).

Formal mediation points in the same direction. In a parallel model entering all three candidate mediators jointly, with participant fixed effects and 95% confidence intervals obtained by participant-cluster bootstrap, only perceived contributor effort carries the effect of visible effort cues on review priority (Table 8); the indirect paths through perceived competence and perceived code quality are not statistically distinguishable from zero. That perceived quality appears inert here is worth remarking on, since a quality account is probably the one reviewers themselves would name first. AI disclosure did not significantly reduce perceived code quality ($b = -0.19$, $d_z = 0.16$, $p = .15$), and perceived code quality does not predict the priority ranking decision. Perceived competence behaves in something like the mirror image: AI disclosure does appear to lower it

Mediator	a path	b path	Indirect effect	d_z	95% bootstrap CI
Perceived contributor effort	0.89	−0.23	−0.208	−0.15	[−0.385, −0.065]
Perceived contributor competence	0.60	−0.09	−0.052	−0.04	[−0.145, 0.045]
Perceived code quality	0.38	−0.05	−0.017	−0.01	[−0.093, 0.063]

Table 8: Parallel mediation of effort legibility on review priority. The a path estimates the effect of visible effort cues on the mediator with participant fixed effects and AI disclosure included as a covariate. The b path estimates the association between the mediator and review priority in a model including legible effort, AI disclosure, all three mediators, and participant fixed effects. Indirect effects are given in rank points and, in the d_z column, standardized by the standard deviation of the paired rank difference (1.37). Confidence intervals are from a participant cluster bootstrap. The total effect of the effort cue is −0.65 rank points ($p < .001$) and the direct effect net of the three mediators is −0.37 ($p = .035$). Negative indirect effects indicate higher review priority.

($b = -0.28$, $d_z = 0.25$, $p = .023$), making it the one social attribution disclosure reliably moves, yet it does not appear to be the attribution reviewers use to decide which contribution is worth their effort.

Within AI-disclosed work, the parallel mediation analysis supports an indirect effect of effort legibility on review priority through perceived creator effort (indirect −0.20, 95% CI [−0.39, −0.04]). Effort legibility also increases review priority for contributions without AI use (−0.54 rank points, $d_z = 0.29$, p = .010). These results support the prediction that high effort legibility increases review priority when AI assistance is disclosed. The benefit of high effort legibility in the no AI assistance condition is also consistent with effort reciprocity operating more generally. A common mechanism does not require identical treatment effects across AI use conditions.

After ranking the four pull requests, participants explained what had driven their choices. These responses are descriptive rather than a separate test, having been written after the ranking decision, but they are informative about what the cues appear to have meant. Participants often treated the description as evidence that the contributor had done their part of the collaboration. One wrote that "the biggest factor was how well contributors explained their thinking. Good code matters, but a clear description shows the person actually understands the problem" (P10); another that "I deprioritized PRs with vague 'tests pass' descriptions even if the code approach seemed correct, because I can't review what I don't understand" (P91). Disclosure produced a division that maps loosely onto the experimental pattern. Some participants penalized disclosed AI use as such, one noting that "bottom positions were assigned to the AI-assisted contributions due to a lack of individual oversight" (P21). Others stated a conditional: "Using AI is fine, but you need to know when it is wrong. It's a tool, not a short-cut" (P81); "AI assistance did not determine my ranking. I focused more on the quality of the implementation" (P58). A number of participants also emphasized reviewability, referring to scope discipline, merge risk, and testing. These remarks are

a reminder that the two considerations travel together in reviewers' own accounts of their choices. In addition, inspection of the ranking explanations suggests that few participants explicitly framed their choices in terms of saving their own review time. Some nevertheless discussed whether the descriptions made a contribution understandable or reviewable. Because these explanations were elicited after ranking and do not measure either anticipated review cost or felt obligation, they cannot distinguish prudent triage from reciprocity.

The experiment holds the code within each stimulus constant across conditions while varying the accompanying description. Accounting for the code stimulus and presentation order leaves the estimated increase in priority for AI-disclosed work largely unchanged (-0.74 rank points, compared with -0.76 without these controls). Both effort conditions disclose AI assistance, so the difference in priority cannot be explained by the disclosure alone.

Holding the code constant does not, however, hold perceptions of quality or competence constant. The parallel mediation analysis therefore includes perceived creator effort, code quality, and contributor competence together. Within AI-disclosed work, perceived effort is the only measured mediator with a statistically significant indirect effect. This supports perceived effort as a pathway connecting effort legibility to review priority, while allowing quality and competence to influence reviewers' responses.

The increase in review priority for AI-disclosed work also holds under participant-level bootstrap and sign-flip tests. The outcome measures relative priority among contributions, however, rather than the time or depth of review participants would provide. Making substantive creator effort legible increases review priority even when AI assistance remains disclosed. The significant indirect effect through perceived creator effort provides support for the mechanism proposed in Hypothesis 2: reviewers' willingness to reciprocate depends on the human investment they infer from the work.

Two limitations qualify the interpretation of the mechanism. First, the descriptions provide task information alongside evidence of effort and differ in length. Reviewers may therefore prioritize a contribution partly because its description makes the work easier to understand, and therefore makes the review process easier. If the description made it easier to review work, I would expect that this would have an impact on how good the code is perceived to be and how likely it is that the code will be merged. However, effort legibility moves perceived code quality and merge acceptance far less compared to review priority. In the parallel mediation, the indirect path from the effort legibility to priority through perceived code quality is not significant ($d_z = +0.01$, 95% CI $[-0.09, 0.06]$), while the path through perceived effort is significant. Additionally, the features of a pull request that most directly govern review labor were held constant across effort conditions: the same commits, the same commit messages, and the same passing checks. The description is in any case a summary of what work was done rather than a substitute for reviewing it. Therefore, a reviewer who intends to act on the change must still read the code and verify that the checks were the right ones.

Second, the three perceptions (perceived effort, quality, and competence) were measured after the manipulation and are correlated with one another ($r = .64$ to $.77$), so the mediation analysis shows what moves together with reviewers' choices, not what causes them. This limits what can be

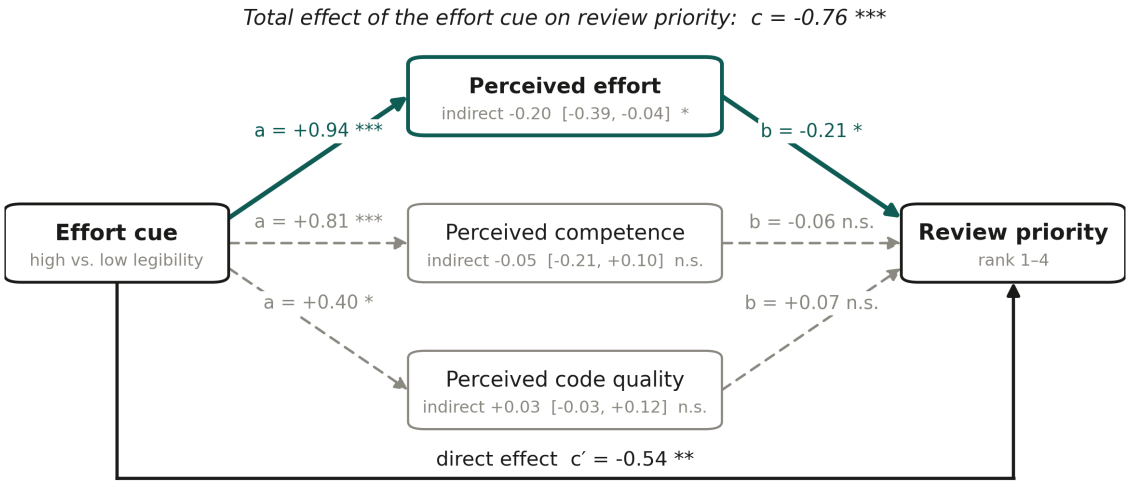


Figure 7: Parallel mediation of the effort-cue effect on review priority within AI-disclosed trials ($N = 83$ participants, 166 trials). The effort cue raises all three perceptions (a-paths: perceived effort +0.94, $p < .001$; competence +0.81, $p < .001$; code quality +0.40, $p = .043$), but when the three compete in the same model only perceived effort predicts priority ($b = -0.21$, $p = .011$; competence $b = -0.06$, $p = .53$; quality $b = +0.07$, $p = .30$). Only the indirect effect through perceived effort excludes zero (−0.20, 95% CI [−0.39, −0.04]); the direct effect remains ($c' = -0.54$, $p = .006$; total $c = -0.76$, $p < .001$), so mediation is partial. Negative coefficients on review priority indicate higher priority (rank 1 = first). Standard errors are clustered by participant; indirect-effect intervals are from a participant cluster bootstrap (5,000 draws). Mediators are post-treatment and measured alongside the outcome, so the estimates describe covariation with the allocation rather than an identified causal chain. Solid teal paths are significant; dashed grey paths are not.

said about the pathway, not about the effect itself: legible effort raises review priority for AI-assisted contributions in a direct within-participant comparison that uses no mediator ($d_z = 0.43$, $p < .001$).

In conclusion, making substantive creator effort legible increases review priority for work disclosed as AI-assisted. The reported mediation model supports perceived creator effort as one pathway explaining this increase. These findings are consistent with the proposed role of perceived creator effort, although the reported mediation does not directly test the indirect effect of labor-augmenting technology use specified in Hypothesis 2.

	Review priority (forced ranking)		Want on team (free rating)		Merge acceptance (free rating)	
Contrast	d_z	p	d_z	p	d_z	p
High-effort AI vs low-effort AI	0.43	$< .001$	0.53	$< .001$	0.19	.086
High-effort no-AI vs. low-effort no-AI	0.29	.010	0.41	$< .001$	0.11	.319
High-effort AI vs. low-effort no-AI	0.24	.029	0.35	.002	0.03	.805
High-effort AI vs. high-effort no-AI	0.06	.576	0.08	.444	0.08	.443

Table 9: The four principal contrasts estimated on the ipsative ranking and on two independently rated per-trial measures. Cohen's d_z is reported as an absolute value; all estimates run in the direction the theory anticipates except the parity contrast, which is near zero on all three measures.

7 Discussion

This paper began with a puzzle about generative AI and collaborative work. If AI helps individuals produce work faster and more easily, why might organizations still struggle to convert that output? I argue that the answer lies in how work is created. Work becomes valuable through a collaborative process that requires reviewers to invest effort in providing feedback and integrating the work.

The paper develops and tests a theory of *effort opacity*: the difficulty collaborators face in inferring effort from work when AI can help produce polished output with uncertain human involvement. The central claim is that AI changes not only the cost of producing work, but also the information that work carries about the effort behind it. When human effort becomes harder to read, collaborators become less willing to return high effort of their own.

The evidence presented in this paper supports this argument. The observational study shows that pull requests whose code appears AI-assisted are more likely to be rejected and receive less high-effort response in the field. The response is not uniform disengagement. Instead, reciprocal effort is recomposed: code-specific inline comments decline, general comments increase, and feedback becomes less constructive. These patterns persist after controlling for contribution size, code complexity, maintainability, automated test outcomes, newcomer status, abandonment, and richer project and collaborator controls. The penalty is also strongest where the theory predicts: in high-activity repositories where collaborators must ration scarce effort, and more modestly for first-time contributors whose effort cannot be inferred from prior history.

I also run a pre-registered between-subjects study to validate the key assumption behind the observational evidence. Developers can identify AI-generated code above chance but imperfectly, and perceived AI involvement is associated with lower attributed human effort. This matters because the observational study does not observe actual reviewer perception. The between-subjects study shows that code bearing AI-like signatures is behaviorally meaningful: it changes what developers infer about the human effort behind the work.

The within-subjects experiment then examines whether making contributor effort legible

changes review allocation. Holding code constant, I vary AI disclosure and effort legibility in the pull request description. Effort-legible descriptions substantially increase priority for AI-disclosed work, and perceived effort is the principal measured pathway connecting the manipulation to review priority. Legible effort raises review priority for non-AI assisted work as well, which is what the theory supports. Effort legibility drives reciprocal review generally, and generative AI degrades that legibility at scale, not because it changes what reviewers are responding to but because it makes effort less legible.

Together, the studies identify a paradox. AI can make work easier for individuals to produce while making human effort harder for others to infer. This weakens the effort-for-effort reciprocity through which individual contributions become collective value.

7.1 Contributions

The paper makes three contributions.

First, it contributes to research on AI and productivity in organizations. Prior studies demonstrate that AI can improve the speed and quality of individual work while documenting important variation across workers and tasks (Noy & Zhang, 2023; Brynjolfsson, 2025; DellAcqua et al., 2026; Peng et al., 2023). This research has primarily measured productivity at the point of production. I examine what happens downstream, when an output must receive another person's attention before it can be accepted and used. A central insight in strategy is that the value of a technology depends not only on the technology itself, but on the complementary actors, assets, and routines through which it is used (Teece 1986). I identify an evaluative microfoundation for this problem. AI may increase the supply of individual output while degrading the information collaborators use to decide whether that output deserves their own effort. Work does not integrate itself; it must attract scarce human effort. The organizational return to AI therefore cannot be inferred from production gains alone. It depends on whether AI-assisted outputs still elicit the reviewing and integration effort needed to realize their value. Research on software development shows that coordination practices shape the balance between knowledge integration and the creation of novel knowledge (Ghosh and Wu, 2023). Research on markets for ideas shows that reducing information costs can facilitate the licensing of inventions (Hegde and Luo, 2018). My findings identify an informational barrier within collaborative work: uncertainty about a contributor's effort can discourage the reciprocal effort needed to develop and use that contribution.

Second, the paper contributes to social exchange theory by identifying the observability of effort as a condition shaping reciprocity. Social exchange theory holds that cooperation is sustained when actors respond to one another's contributions with contributions of their own (Gouldner 1960, Blau 1964, Emerson 1976, Molm 2010). What sustains the exchange is not only the benefit each party receives, but each party's ability to read the other's investment from what is exchanged. Signaling theory explains why work has served that purpose: observable features are informative when they are difficult to produce without investing in the underlying task (Spence 1973). I introduce effort

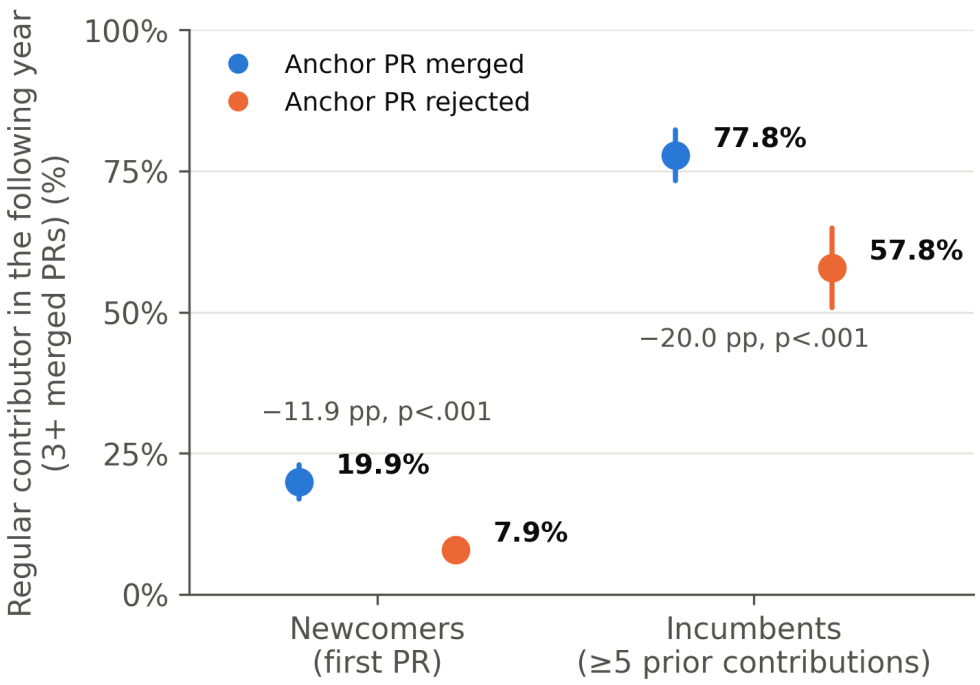
opacity to capture what happens when technology weakens that relationship. The findings suggest that reviewers calibrate their effort not only to the apparent quality of the work, but also to the effort they believe its contributor invested, and that visible-effort cues increase willingness to engage with AI-assisted work. Reciprocity therefore depends not only on the exchange of benefits, but on the legibility of effort in what is exchanged.

Third, the paper contributes to research on evaluation in organizations. Prior work examines how evaluators interpret signals about the merit of a contribution and the capabilities of its creator, with consequences for which ideas receive resources and advance (Elsbach & Kramer2003, Boudreau 2016, Criscuolo 2017). Recent studies further show that AI use can itself become an evaluative signal, affecting judgments of competence, motivation, and trust (Reif 2025, Schilke & Reimann 2025, Gai 2025). I conceptualize evaluation not only as a judgment rendered on finished work, but as an allocation of costly collaborative effort. Reviewers decide which contributions to examine, how closely to scrutinize them, and how much developmental feedback to provide, and these decisions shape what the work becomes. The findings show how production technology can alter these behavioral investments by changing what evaluators infer about the effort of the person behind the work.

7.2 Organizational Stakes: Contributor Progression and Retention

Whether elevated rejection of newcomer work matters for the organization depends on whether it falls at a consequential margin. We examine two: whether the outcome of an early contribution predicts a contributor's subsequent progression, and whether newcomers' retention has deteriorated relative to incumbents since the arrival of AI coding assistants. The focus on progression follows evidence that newcomers' early trajectories predict their later retention and advancement in organizations (Srivastava et al., 2018). Figure 8 shows that early outcomes and subsequent progression are tightly linked. Among newcomers, 19.9% of those whose first pull request was accepted become regular contributors in the following year, against 7.9% of those whose first pull request was rejected—a difference of twelve percentage points at the very margin where AI-predicted work is most likely to be rejected. Rejection is associated with lower subsequent participation among incumbents as well (77.8% versus 57.8%), but the meaning of the loss differs: a rejected incumbent still continues more often than not, while a rejected newcomer almost never becomes a regular contributor. These estimates are adjusted for year and repository, but the comparison remains associative: accepted and rejected contributions may differ in project fit, contributor commitment, and other characteristics we do not observe.

Figure 9 provides sharper timing evidence on the retention margin itself. It reports difference-in-differences event-study estimates comparing the one-year retention of experienced-outsider newcomers—contributors making their first pull request to a repository but arriving with a track record elsewhere in the sample—with that of incumbents in the same repository and year. The gap is statistically indistinguishable from its 2022 level in every pre-period year, supporting the

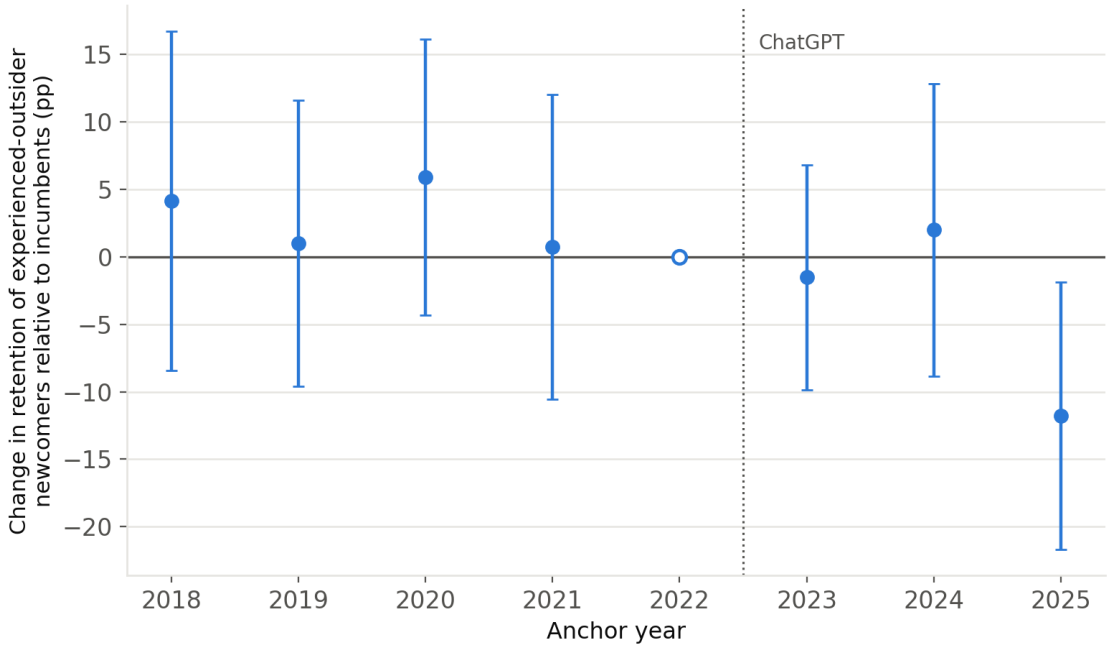


Notes: The figure shows the regression-adjusted probability of being a regular contributor in the year after a decided pull request, by the outcome of that pull request. A regular contributor is someone with three or more merged PRs in the repository within the following year, not counting the anchor PR. Newcomers are contributors whose anchor is their first PR to the repository (n = 17,963, 2019–2025); incumbents have at least five prior contributions, with one anchor per year. Estimates include year and repository fixed effects; whiskers are 95% confidence intervals from repository-clustered standard errors.

Figure 8: Probability of becoming a regular contributor, by the outcome of the anchor pull request and contributor standing.

parallel-trends assumption, and remains so through 2023 and 2024. In 2025, however, the retention of experienced outsiders falls by 11.8 percentage points relative to incumbents (p = .020). The timing is consistent with a gradually accumulating effect of AI diffusion rather than a discrete break at the release of ChatGPT, and the affected group is notable: these are not naive first-time contributors but developers whose demonstrated experience elsewhere would previously have transferred across project boundaries.

Taken together, the results identify organizational stakes rather than a causal downstream effect. AI-predicted work is disproportionately rejected when it comes from newcomers; rejected newcomers rarely progress; and the relative retention of experienced newcomers deteriorated sharply in 2025. These losses land in communities whose senior core is no longer replenishing itself—net entry into the core turned negative in 2024–2025 (Appendix Figure 10)—so elevated rejection of newcomer work operates at a margin that is consequential for the continued growth of collaborative communities.



Notes: The sample is one anchor per author–repository–year: the author's first pull request to the repository in that year. Newcomers are making their first-ever pull request to the repository but have prior pull request activity elsewhere in the 166-repository sample ("experienced outsiders"); incumbents have prior pull requests in the repository. The outcome is retention, defined as two or more further pull requests to the same repository within 365 days of the anchor; anchors with fewer than 365 days of observation before the June 2026 collection date are dropped, so every estimate—including 2025—reflects a complete one-year window. Points are coefficients on newcomer × year interactions from a linear probability model with repository and year fixed effects, with 2022 (hollow marker) as the reference year; whiskers are 95% confidence intervals from repository-clustered standard errors. The 2025 estimate is −11.8 percentage points ($p = .020$).

Figure 9: Event-study estimates of the retention gap between experienced-outsider newcomers and incumbents.

7.3 Implications for AI Governance

The findings have direct implications for AI governance. The central governance problem is not simply whether workers disclose that they used AI. A binary disclosure can tell collaborators that a tool was involved, but it does not tell them what the human did. It does not reveal whether the contributor diagnosed the problem, checked the output, revised the work, understood the implications, or took responsibility for the result.

The experiment shows that increasing effort legibility increases review priority for AI-disclosed work. Organizations should therefore avoid treating AI governance as a simple disclosure problem. Disclosure may be necessary in some settings, but it is not sufficient. The more important design problem is how to preserve credible traces of human effort around AI-assisted work.

In practice, this means that organizations may need new routines for documenting human engagement with AI-assisted output. For code, this may include pull request descriptions that explain

the problem, the reasoning behind the change, the tests performed, and the contributor's responsibility for the final code. For other forms of knowledge work, it may include annotated drafts, decision logs, audit trails, revision histories, assumptions checklists, or explicit statements of verification. The goal is not to create a bureaucracy around work—a relevant concern because responsible AI practices can impose meaningful organizational costs, particularly for smaller organizations (Bessen, Impink, and Seamans, 2022). The goal is to keep the human contribution legible when AI makes potentially polished work cheap to produce.

The paper also cautions against a purely output-centered view of AI productivity. If organizations evaluate AI adoption only by the amount or apparent quality of output produced, they may miss downstream costs in review, feedback, coordination, trust, and integration. The organizational value of AI depends on the whole collaborative system, not only on the worker using the tool.

7.4 Limitations and Future Research

Several limitations qualify the interpretation of the findings.

First, the observational study measures AI-predicted code rather than verified AI use. GitHub pull requests do not contain ground-truth information about whether contributors used AI. I therefore use a code-based classifier to identify pull requests whose submitted code appears AI-assisted. This is theoretically appropriate because effort opacity concerns what collaborators can infer from the work, not the unobserved production process. Still, the measure is imperfect. Some truly AI-assisted work will not be detected, especially if it is heavily edited or produced by models whose style differs from the training distribution. Some human-written work may also be misclassified. For this reason, I interpret the field measure as AI-predicted appearance, not verified AI use.

Second, contributors who produce AI-predicted pull requests may differ from other contributors in unobserved ways. They may choose harder tasks, understand the codebase less well, submit more speculative work, or rely on AI when they are less committed to follow-through. The robustness checks address many observable alternatives: size, complexity, maintainability, automated test outcomes, abandonment, newcomer status, project selectivity, collaborator selectivity, and author experience. But unobserved differences in task difficulty, architectural fit, contributor motivation, or true production process may still contribute to the field pattern. The observational study should therefore be interpreted as field-pattern evidence, not as a causal estimate of actual AI use.

Third, the quality alternative cannot be ruled out completely.The paper shows that the AI-predicted penalty is not absorbed by observable code characteristics or automated test outcomes, that the mechanism test experiment holds the underlying code constant, and that rejected AI-predicted work is subsequently adopted at a rate indistinguishable from rejected non-AI-predicted work. But quality in knowledge work is difficult to measure exhaustively. Automated tests do not capture architectural fit, long-run maintainability, project priorities, or whether the contributor can explain and maintain the change. The strongest claim is therefore not that AI-predicted and non-AI-predicted work are identical on every quality dimension. The reduced reciprocity for AI-

predicted work in the observational study persists after accounting for the observable quality indicators measured here. The experiment additionally shows that high effort legibility increases review priority when the quality is held constant, with perceived contributor effort emerging as a mechanism consistent with the theory.

Fourth, the observational study cannot show that the actual GitHub collaborators consciously perceived the focal pull requests as AI-assisted. I help bridge this gap by showing that developers detect AI-generated code above chance and associate perceived AI involvement with lower human effort. The two studies should therefore be read as converging evidence: the observational study documents the pattern, and the mechanism experiment tests the mechanism and remedy.

Finally, the empirical setting is open-source software. This setting is ideal for observing work, traces of effort, and reciprocal response, but it is not the only setting in which effort opacity should matter. Open-source software may amplify the mechanism because reciprocity, mentorship, and voluntary effort are especially salient. The magnitude of the effects may be smaller in settings with stronger formal authority, richer monitoring, repeated interaction, or direct observation of the work process. At the same time, the paid-employee analysis suggests that the pattern is not confined to external volunteers: AI-predicted penalties also appear among employees contributing to company-owned public repositories. Future research should examine effort opacity in private firms, consulting teams, legal work, scientific collaboration, peer review, education, and other settings where AI-assisted work must attract effort from others.

Future work can test the mechanism in other work contexts. For example, research shows that learning about AI advances can change which attributes people regard as essential to being human (Santoro and Monin, 2023). Future research could examine whether such changes in how work is perceived also reshape what collaborators regard as a valuable human contribution. Future work should also study contributor adaptation to reduced reciprocity. This paper focuses on how collaborators respond to work that appears AI-assisted. It does not observe whether contributors subsequently abandon AI tools, improve their disclosure practices, learn to document their effort, exit projects, or shift to different forms of contribution. The theory predicts that contributors will eventually develop new ways to make human effort visible around AI-assisted work. Understanding those adaptation strategies is central to understanding how organizations will rebuild collaboration in the AI era.

Generative AI is often evaluated as a technology for individual productivity. This paper shows why that view is incomplete. Organizations do not create value simply because individuals produce more work. They create value when work attracts the reciprocal effort needed for review, feedback, revision, trust, and integration. AI can disrupt that process by creating effort opacity. When technology can produce polished work with uncertain human involvement, collaborators may struggle to infer how much effort the human contributed. The work may be useful. It may even be technically strong. But if the human effort behind it is difficult to read, collaborators have less basis for returning high effort of their own.

The evidence shows that this problem is consequential. In the observational data, code con-

tributions that appear AI-assisted are more likely to be rejected and receive less code-specific developmental effort. In the experiment, making contributor effort legible substantially increases priority for AI-disclosed work, with perceived contributor effort emerging as the principal measured pathway. The evidence therefore supports effort legibility as an important basis of review allocation.

The implication is not that organizations should avoid AI, nor that AI-assisted work is necessarily low quality. The implication is that organizations must govern the human traces around AI-assisted work. The returns to AI will depend not only on the capability of the tool, but on whether collaborative systems preserve enough evidence of human effort, judgment, checking, and responsibility for others to reciprocate. AI changes what people can produce. It also changes what others can infer from what they produce. The organizations that benefit most from AI will be those that solve both problems.

References

Acemoglu, D., & Restrepo, P. (2019). Automation and new tasks: How technology displaces and reinstates labor. *Journal of Economic Perspectives*, *33*(2), 3–30. https://doi.org/10.1257/jep.33.2.3

Adams, J. S. (1963). Toward an understanding of inequity. *Journal of Abnormal and Social Psychology*, *67*(5), 422–436. https://doi.org/10.1037/h0040968

Adams, J. S. (1965). Inequity in social exchange. In L. Berkowitz (Ed.), *Advances in Experimental Social Psychology* (Vol. 2, pp. 267–299). Academic Press.

Alphabet. (2026). 2025 fourth quarter earnings call. *Alphabet Investor Relations*. https://abc.xyz/investor/events/event-details/2026/2025-Q4-Earnings-Call-2026-Dr_C033hS6/default.aspx

Ames, D. R., Flynn, F. J., & Weber, E. U. (2004). It's the thought that counts: On perceiving how helpers decide to lend a hand. *Personality and Social Psychology Bulletin*, *30*(4), 461–474. https://doi.org/10.1177/0146167203261890

Avelino, G., Passos, L., Hora, A., & Valente, M. T. (2016). A novel approach for estimating truck factors. In *Proceedings of the 24th IEEE International Conference on Program Comprehension (ICPC)* (pp. 1–10). IEEE. https://doi.org/10.1109/ICPC.2016.7503718

Azoulay, P., Stuart, T., & Wang, Y. (2014). Matthew: Effect or fable? *Management Science*, *60*(1), 92–109. https://doi.org/10.1287/mnsc.2013.1755

Bacchelli, A., & Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 2013 International Conference on Software Engineering (ICSE)* (pp. 712–721). IEEE. https://doi.org/10.1109/ICSE.2013.6606617

Baltes, S., Cheong, H. H., & Treude, C. (2026). AI slop and the software commons. *arXiv preprint arXiv:2604.16754*. https://arxiv.org/abs/2604.16754

Bartlett, M. Y., & DeSteno, D. (2006). Gratitude and prosocial behavior: Helping when it costs you. *Psychological Science*, *17*(4), 319–325. https://doi.org/10.1111/j.1467-9280.2006.01705.x

Bauer, J. (2024, November 18). Does GitHub Copilot improve code quality? Here's what the data says. *The GitHub Blog*. https://github.blog/news-insights/research/does-github-copilot-improve-code-quality-heres-what-the-data-says/

Bechky, B. A. (2003). Sharing meaning across occupational communities: The transformation of understanding on a production floor. *Organization Science*, *14*(3), 312–330. https://doi.org/10.1287/orsc.14.3.312.15162

Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. *arXiv preprint arXiv:2507.09089*. https://doi.org/10.48550/arXiv.2507.09089

Bedwell, W. L., Wildman, J. L., DiazGranados, D., Salazar, M., Kramer, W. S., & Salas, E. (2012). Collaboration at work: An integrative multilevel conceptualization. *Human Resource Management Review*, *22*(2), 128–145. https://doi.org/10.1016/j.hrmr.2011.11.007

Bessen, J., Impink, S. M., & Seamans, R. (2022). The cost of ethical AI development for AI startups. In *Proceedings of the 2022 AAAI/ACM Conference on AI, Ethics, and Society* (pp. 92–106). Association for Computing Machinery. https://doi.org/10.1145/3514094.3534195

Bick, A., Blandin, A., & Deming, D. J. (2026). The rapid adoption of generative AI. *Management Science*. Advance online publication. https://doi.org/10.1287/mnsc.2025.02523

Birtolo, D. (2026, February 18). What to expect for open source in 2026. *The GitHub Blog*. https://github.blog/open-source/maintainers/what-to-expect-for-open-source-in-2026/

Blau, P. M. (1964). *Exchange and power in social life*. Wiley.

Botelho, T. L. (2024). From audience to evaluator: When visibility into prior evaluations leads to convergence or divergence in subsequent evaluations among professionals. *Organization Science*, *35*(5), 1682–1703. https://doi.org/10.1287/orsc.2017.11285

Boudreau, K. J., Guinan, E. C., Lakhani, K. R., & Riedl, C. (2016). Looking across and looking beyond the knowledge frontier: Intellectual distance, novelty, and resource allocation in science. *Management Science*, *62*(10), 2765–2783. https://doi.org/10.1287/mnsc.2015.2285

Boudreau, K. J., & Jeppesen, L. B. (2015). Unpaid crowd complementors: The platform network effect mirage. *Strategic Management Journal*, *36*(12), 1761–1777.

Bridoux, F., & Stoelhorst, J. W. (2014). Microfoundations for stakeholder theory: Managing stakeholders with heterogeneous motives. *Strategic Management Journal*, 35(1), 107–125.

Brynjolfsson, E., Li, D., & Raymond, L. (2025). Generative AI at work. *The Quarterly Journal of Economics*, *140*(2), 889–942.

Brynjolfsson, E., Rock, D., & Syverson, C. (2018). Artificial intelligence and the modern productivity paradox: A clash of expectations and statistics. *NBER Working Paper No. 24001*.

Buell, R. W., & Norton, M. I. (2011). The labor illusion: How operational transparency increases perceived value. *Management Science*, *57*(9), 1564–1579. https://doi.org/10.1287/mnsc.1110.1376

Buell, R. W., Kim, T., & Tsay, C.-J. (2017). Creating reciprocal value through operational transparency. *Management Science*, *63*(6), 1673–1695. https://doi.org/10.1287/mnsc.2015.2411

Bukhari, S., Tan, B., & De Carli, L. (2023). Distinguishing AI- and human-generated code: A case study. *Proceedings of the 2023 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED '23)*, 23–32.

Carlson, N. A., & Burbano, V. (2026). The use of LLMs to annotate data in management research: Foundational guidelines and warnings. *Strategic Management Journal*, *47*(3), 699–725. https://doi.org/10.1002/smj.70023

Celniker, J. B., Gregory, A., Koo, H. J., Piff, P. K., Ditto, P. H., & Shariff, A. F. (2023). The moralization of effort. *Journal of Experimental Psychology: General*, *152*(1), 60–79. https://doi.org/10.1037/xge0001259

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., ... Zaremba, W. (2021). *Evaluating large language models trained on code*. arXiv preprint arXiv:2107.03374.

Chinander, K. R., & Schweitzer, M. E. (2003). The input bias: The misuse of input information in judgments of outcomes. *Organizational Behavior and Human Decision Processes*, *91*(2), 243–253. https://doi.org/10.1016/S0749-5978(03)00025-6

Cihan, U., Haratian, V., İcöz, A., Gül, M. K., Devran, Ö., Bayendur, E. F., Uçar, B. M., & Tüzün, E. (2024). Automated code review in practice. *arXiv preprint arXiv:2412.18531*. https://doi.org/10.48550/arXiv.2412.18531

Claessens, S., Veitch, P., & Everett, J. A. C. (2026). Negative perceptions of outsourcing to artificial intelligence. *Computers in Human Behavior*, *177*, 108894. https://doi.org/10.1016/j.chb.2025.108894

Connelly, B. L., Certo, S. T., Ireland, R. D., & Reutzel, C. R. (2011). Signaling theory: A review and assessment. *Journal of Management*, *37*(1), 39–67. https://doi.org/10.1177/0149206310388419

Connelly, B. L., Certo, S. T., Reutzel, C. R., DesJardine, M. R., & Zhou, Y. S. (2025). Signaling theory: State of the theory and its future. *Journal of Management*, *51*(1), 24–61. https://doi.org/10.1177/01492063241268459

Conti, A., Peukert, C., & Roche, M. (2025). Beefing IT up for your investor? Engagement with open source communities, innovation, and startup funding: Evidence from GitHub. *Organization Science*, *36*(4), 1551–1573. https://doi.org/10.1287/orsc.2023.18348

Cowgill, B., Hernández-Lagos, P., & Wright, N. L. (2026). Does AI cheapen talk? Theory and evidence from global entrepreneurship and hiring. *Management Science*. Advance online publication. https://doi.org/10.1287/mnsc.2024.07027

Cranney, K., Delecourt, S., & Koning, R. (2024). Global evidence on gender gaps and generative AI. Harvard Business School Working Paper No. 25-023.

Criscuolo, P., Dahlander, L., Grohsjean, T., & Salter, A. (2017). Evaluating novelty: The role of panels in the selection of R&D projects. *Academy of Management Journal*, *60*(2), 433–460. https://doi.org/10.5465/amj.2014.0861

Cropanzano, R., & Mitchell, M. S. (2005). Social exchange theory: An interdisciplinary review. *Journal of Management*, *31*(6), 874–900. https://doi.org/10.1177/0149206305279602

Cropanzano, R., Anthony, E. L., Daniels, S. R., & Hall, A. V. (2017). Social exchange theory: A critical review with theoretical remedies. *Academy of Management Annals*, *11*(1), 479–516. https://doi.org/10.5465/annals.2015.0099

Cui, K. Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2025). The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. *SSRN working paper*. https://doi.org/10.2139/ssrn.4945566

Dabbish, L., Stuart, C., Tsay, J., & Herbsleb, J. (2012). Social coding in GitHub: Transparency and collaboration in an open software repository. In *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work*, 1277–1286.

Daniotti, S., Wachs, J., Feng, X., & Neffke, F. M. H. (2026). Who is using AI to code? Global diffusion and impact of generative AI. *Science*, 391(6787), 831–835.

de Vaan, M., Stark, D., & Vedres, B. (2015). Game changer: The topology of creativity. *American Journal of Sociology*, 120(4), 1144–1194.

Dell'Acqua, F., Ayoubi, C., Lifshitz, H., Sadun, R., Mollick, E., Mollick, L., Han, Y., Goldman, J., Nair, H., Taub, S., & Lakhani, K. R. (2026a). The cybernetic teammate: A field experiment on generative AI and teamwork. *Organization Science*, Articles in Advance. https://doi.org/10.1287/orsc.2025.20702

Dell'Acqua, F., McFowland, E., III, Mollick, E., Lifshitz, H., Kellogg, K. C., Rajendran, S., Krayer, L., Candelon, F., & Lakhani, K. R. (2026b). Navigating the jagged technological frontier: Field experimental evidence of the effects of artificial intelligence on knowledge worker productivity and quality. *Organization Science*, *37*(2), 403–423. https://doi.org/10.1287/orsc.2025.21838

Demirer, M., Cui, Z., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2026). Production and adoption of generative AI: Evidence from 1.2 million developers. *Management Science* (forthcoming).

Dierdorff, E. C., & Rubin, R. S. (2022). Revisiting reciprocity: How accountability, proactivity, and interpersonal skills shape obligations to reciprocate citizenship behavior. *Journal of Business and Psychology*, *37*(2), 263–281. https://doi.org/10.1007/s10869-021-09743-6

Dietvorst, B. J., Simmons, J. P., & Massey, C. (2015). Algorithm aversion: People erroneously avoid algorithms after seeing them err. *Journal of Experimental Psychology: General*, 144(1), 114–126.

Dishop, C. R., Brown, A. S., Chao, P.-Y., Kuznetsov, A., & Woolley, A. W. (2025). Machines in the middle: Using artificial intelligence (AI) while offering help affects warmth, felt obligations, and reciprocity. *Journal of Business and Psychology*. Advance online publication. https://doi.org/10.1007/s10869-025-10068-x

Eghbal, N. (2016). *Roads and bridges: The unseen labor behind our digital infrastructure*. Ford Foundation.

Elsbach, K. D., & Kramer, R. M. (2003). Assessing creativity in Hollywood pitch meetings: Evidence for a dual-process model of creativity judgments. *Academy of Management Journal*, *46*(3), 283–301. https://doi.org/10.5465/30040623

Eltabakh, T. M., Soudi, N. N., & Shawky, D. (2024, December). Quality of AI-generated vs. human-generated code. In *2024 34th International Conference on Computer Theory and Applications (ICCTA)* (pp. 200–205). IEEE.

Emerson, R. M. (1976). Social exchange theory. *Annual Review of Sociology*, *2*, 335–362. https://doi.org/10.1146/annurev.so.02.080176.002003

Fisher, C. M., Pillemer, J., & Amabile, T. M. (2018). Deep help in complex project work: Guiding and path-clearing across difficult terrain. *Academy of Management Journal*, *61*(4), 1524–1553. https://doi.org/10.5465/amj.2016.0207

Flynn, F. J. (2006). How much is it worth to you? Subjective evaluations of help in organizations. *Research in Organizational Behavior*, *27*, 133–174. https://doi.org/10.1016/S0191-3085(06)27004-7

Fradkin, A., Grewal, E., & Holtz, D. (2021). Reciprocity and unveiling in two-sided reputation systems: Evidence from an experiment on Airbnb. *Marketing Science*, *40*(6), 1013–1029. https://doi.org/10.1287/mksc.2021.1311

Fradkin, A., & Holtz, D. (2023). Do incentives to review help the market? Evidence from a field experiment on Airbnb. *Marketing Science*, *42*(5), 853–865. https://doi.org/10.1287/mksc.2023.1439

Fu, Y., Liang, P., Tahir, A., Li, Z., Shahin, M., Yu, J., & Chen, J. (2025). Security weaknesses of Copilot-generated code in GitHub projects: An empirical study. *ACM Transactions on Software Engineering and Methodology*, *34*(8), Article 218. https://doi.org/10.1145/3716848

Gai, P. J., Hou, J., & Tu, Y. (2025). *Competence penalty is a barrier to the adoption of new technology*. SSRN Working Paper No. 5255039. https://doi.org/10.2139/ssrn.5255039

Gerosa, M. A., Wiese, I. S., Trinkenreich, B., Link, G. J. P., Robles, G., Treude, C., Steinmacher, I., & Sarma, A. (2021). The shifting sands of motivation: Revisiting what drives contributors in open source. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)* (pp. 1046–1058). IEEE. https://doi.org/10.1109/ICSE43902.2021.00098

Ghosh, S., & Wu, A. (2023). Iterative coordination and innovation: Prioritizing value over novelty. *Organization Science*, *34*(6), 2182–2206. https://doi.org/10.1287/orsc.2021.1499

GitHub Docs. (n.d.-a). About repositories. *GitHub Docs*. Retrieved June 25, 2026, from https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

GitHub Docs. (n.d.-b). GitHub glossary. *GitHub Docs*. Retrieved June 25, 2026, from https://docs.github.com/articles/github-glossary

GitHub Docs. (n.d.-c). About pull requests. *GitHub Docs*. Retrieved June 25, 2026, from https://docs.github.com/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

GitHub Docs. (n.d.-d). About pull request reviews. *GitHub Docs*. Retrieved June 25, 2026, from https://docs.github.com/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews

GitHub Docs. (n.d.-e). GitHub Actions documentation. *GitHub Docs*. Retrieved June 25, 2026, from https://docs.github.com/actions

GitHub Docs. (n.d.-f). About GitHub Copilot code review. *GitHub Docs*. Retrieved June 25, 2026, from https://docs.github.com/en/copilot/concepts/agents/code-review

GitHub Staff. (2025, October 28). Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1. *The GitHub Blog*. https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/

GitHub. (2023a). Survey reveals AI's impact on the developer experience. *The GitHub Blog*. https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/

GitHub. (2023b). Octoverse: The state of open source and rise of AI in 2023. *The GitHub Blog*.

GitHub. (2024). *Octoverse 2024: AI leads Python to top language as the number of global developers surges*. The GitHub Blog. https://github.blog/news-insights/octoverse/octoverse-2024/

Goffman, E. (1961). *Encounters: Two studies in the sociology of interaction*. Indianapolis, IN: Bobbs-Merrill.

Gortmaker, J. (2024). Open source software policy in industry equilibrium. *Working Paper*.

Gorwa, R., & Veale, M. (2024). Moderating model marketplaces: Platform governance puzzles for AI intermediaries. *Law, Innovation and Technology*, 1–51.

Gouldner, A. W. (1960). The norm of reciprocity: A preliminary statement. *American Sociological Review*, 25(2), 161–178.

Gousios, G., Storey, M.-A., & Bacchelli, A. (2016). Work practices and challenges in pull-based development: The contributor's perspective. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)* (pp. 285–296). ACM. https://doi.org/10.1145/2884781.2884826

Gousios, G., Zaidman, A., Storey, M.-A., & van Deursen, A. (2015). Work practices and challenges in pull-based development: The integrator's perspective. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)* (pp. 358–368). IEEE. https://doi.org/10.1109/ICSE.2015.55

Grant, R. M. (1996). Toward a knowledge-based theory of the firm. *Strategic Management Journal*, *17*(S2), 109–122. https://doi.org/10.1002/smj.4250171110

Guilbeault, D., & Centola, D. (2020). Networked collective intelligence improves dissemination of scientific information regarding smoking risks. *PLOS ONE*, *15*(2), e0227813. https://doi.org/10.1371/journal.pone.0227813

Harrison, S. H., & Rouse, E. D. (2015). An inductive study of feedback interactions over the course of creative projects. *Academy of Management Journal*, *58*(2), 375–404. https://doi.org/10.5465/amj.2012.0737

Harvey, S., & Kou, C.-Y. (2013). Collective engagement in creative tasks: The role of evaluation in the creative process in groups. *Administrative Science Quarterly*, *58*(3), 346–386. https://doi.org/10.1177/0001839213498591

Hegde, D., & Luo, H. (2018). Patent publication and the market for ideas. *Management Science*, *64*(2), 652–672. https://doi.org/10.1287/mnsc.2016.2622

Hutto, C., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the International AAAI Conference on Web and Social Media*, *8*(1), 216–225.

Hwang, E. M., Lee, S., & Legendre, T. S. (2024). Operational transparency as a service design: An investigation on labor/effort observation effect. *Journal of Hospitality & Tourism Research*, *48*(8), 1525–1540. https://doi.org/10.1177/10963480231223146

Inzlicht, M., Shenhav, A., & Olivola, C. Y. (2018). The effort paradox: Effort is both costly and valued. *Trends in Cognitive Sciences*, *22*(4), 337–349. https://doi.org/10.1016/j.tics.2018.01.007

Jakesch, M., Hancock, J. T., & Naaman, M. (2023). Human heuristics for AI-generated language are flawed. *Proceedings of the National Academy of Sciences*, *120*(11), e2208839120. https://doi.org/10.1073/pnas.2208839120

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., & Damian, D. (2014). The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories* (pp. 92–101). Association for Computing Machinery. https://doi.org/10.1145/2597073.2597074

Khubulashvili, R., Linardi, S., & Wang, X. (2021). *The role of effort cost perception in outcome bias*. Working paper. https://doi.org/10.2139/ssrn.3789963

Kogut, B., & Zander, U. (1992). Knowledge of the firm, combinative capabilities, and the replication of technology. *Organization Science*, *3*(3), 383–397. https://doi.org/10.1287/orsc.3.3.383

Kollock, P. (1999). The economies of online cooperation: Gifts and public goods in cyberspace. In M. A. Smith & P. Kollock (Eds.), *Communities in Cyberspace* (pp. 220–239). Routledge.

Kool, W., & Botvinick, M. (2018). Mental labour. *Nature Human Behaviour*, *2*(12), 899–908. https://doi.org/10.1038/s41562-018-0401-9

Kovács, B. (2024). The Turing test of online reviews: Can we tell the difference between human-written and GPT-4-written online reviews? *Marketing Letters*, *35*(4), 651–666. https://doi.org/10.1007/s11002-024-09729-3

Kruger, J., Wirtz, D., Van Boven, L., & Altermatt, T. W. (2004). The effort heuristic. *Journal of Experimental Social Psychology*, *40*(1), 91–98. https://doi.org/10.1016/S0022-1031(03)00065-9

Kwon, M., Lee Cunningham, J., & Jachimowicz, J. M. (2023). Discerning saints: Moralization of intrinsic motivation and selective prosociality at work. *Academy of Management Journal*, *66*(6), 1625–1650. https://doi.org/10.5465/amj.2020.1761

Lakhani, K. R., & Wolf, R. G. (2005). Why hackers do what they do: Understanding motivation and effort in free/open source software projects. In J. Feller, B. Fitzgerald, S. Hissam, & K. R. Lakhani (Eds.), *Perspectives on free and open source software* (pp. 3–22). MIT Press.

Lamont, M. (2012). Toward a comparative sociology of valuation and evaluation. *Annual Review of Sociology*, *38*, 201–221. https://doi.org/10.1146/annurev-soc-070308-120022

Lawrence, P. R., & Lorsch, J. W. (1967). *Organization and environment: Managing differentiation and integration*. Division of Research, Graduate School of Business Administration, Harvard University.

Legay, D., Decan, A., & Mens, T. (2018). On the impact of pull request decisions on future contributions. *arXiv preprint arXiv:1812.06269*. https://arxiv.org/abs/1812.06269

Lerner, J., & Tirole, J. (2002). Some simple economics of open source. *The Journal of Industrial Economics*, *50*(2), 197–234.

Lin, N. (2001). *Social capital: A theory of social structure and action*. Cambridge University Press.

LLVM Project. (2025, December 17). [RFC] LLVM AI tool policy: Human in the loop. *LLVM Discourse*. https://discourse.llvm.org/t/rfc-llvm-ai-tool-policy-human-in-the-loop/89159

March, J. G., & Simon, H. A. (1958). *Organizations*. Wiley.

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.

McElheran, K., Li, J. F., Brynjolfsson, E., Kroff, Z., Dinlersoz, E., Foster, L., & Zolas, N. (2024). AI adoption in America: Who, what, and where. *Journal of Economics & Management Strategy*, 33(2), 375–415.

Microsoft. (2025, July 30). Microsoft FY25 fourth quarter earnings conference call. *Microsoft Investor Relations*. https://www.microsoft.com/en-us/investor/events/fy-2025/earnings-fy-2025-q4

Mockus, A., Fielding, R. T., & Herbsleb, J. D. (2002). Two case studies of open source software development: Apache and Mozilla. *ACM Transactions on Software Engineering and Methodology*, *11*(3), 309–346. https://doi.org/10.1145/567793.567795

Molm, L. D. (2003). Theoretical comparisons of forms of exchange. *Sociological Theory*, *21*(1), 1–17. https://doi.org/10.1111/1467-9558.00171

Molm, L. D. (2010). The structure of reciprocity. *Social Psychology Quarterly*, *73*(2), 119–131. https://doi.org/10.1177/0190272510369079

Moraes, C., & Wolf, A. (2026, June 18). How pull request limits are cutting down the noise. *The GitHub Blog*. https://github.blog/open-source/maintainers/how-pull-request-limits-are-cutting-down-the-noise/

Morales, A. C. (2005). Giving firms an "E" for effort: Consumer responses to high-effort firms. *Journal of Consumer Research*, *31*(4), 806–812. https://doi.org/10.1086/426615

Mueller, J. S., & Kamdar, D. (2011). Why seeking help from teammates is a blessing and a curse: A theory of help seeking and individual creativity in team contexts. *Journal of Applied Psychology*, *96*(2), 263–276. https://doi.org/10.1037/a0021574

Nagaraj, A. (2021). Information seeding and knowledge production in online communities: Evidence from OpenStreetMap. *Management Science*, 67(8), 4908–4934.

Nagle, F. (2019). Open source software and firm productivity. *Management Science*, 65(3), 1191–1215.

Nature. (2023). Tools such as ChatGPT threaten transparent science; here are our ground rules for their use. *Nature*, 613(7945), 612.

Nembhard, I. M., & Edmondson, A. C. (2006). Making it safe: The effects of leader inclusiveness and professional status on psychological safety and improvement efforts in health care teams. *Journal of Organizational Behavior*, *27*(7), 941–966. https://doi.org/10.1002/job.413

Noy, S., & Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, *381*(6654), 187–192.

Oliveira, D., Santos, R., de Oliveira, B., Monperrus, M., Castor, F., & Madeiral, F. (2025). Understanding code understandability improvements in code reviews. *IEEE Transactions on Software Engineering*, *51*(1), 14–37. https://doi.org/10.1109/TSE.2024.3453783

O'Mahony, S., & Ferraro, F. (2007). The emergence of governance in an open source community. *Academy of Management Journal*, *50*(5), 1079–1106.

Ocasio, W. (1997). Towards an attention-based view of the firm. *Strategic Management Journal*, *18*(S1), 187–206. https://doi.org/10.1002/(SICI)1097-0266(199707)18:1+<187::AID-SMJ936>3.0.CO;2-K

Okhuysen, G. A., & Bechky, B. A. (2009). Coordination in organizations: An integrative perspective. *Academy of Management Annals*, *3*(1), 463–502. https://doi.org/10.1080/19416520903047533

Oman, P., & Hagemeister, J. (1992). Metrics for assessing a software system's maintainability. In *Proceedings of the IEEE Conference on Software Maintenance* (pp. 337–344). IEEE.

Oster, E. (2019). Unobservable selection and coefficient stability: Theory and evidence. *Journal of Business & Economic Statistics*, 37(2), 187–204.

Otis, N. G., Clarke, R., Delecourt, S., Holtz, D., & Koning, R. (2025). The uneven impact of generative AI on entrepreneurial performance. Harvard Business School Working Paper No. 24-042.

Padhye, R., Mani, S., & Sinha, V. S. (2014). A study of external community contribution to open-source projects on GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, 332–335.

Palmatier, R. W., Jarvis, C. B., Bechkoff, J. R., & Kardes, F. R. (2009). The role of customer gratitude in relationship marketing. *Journal of Marketing*, *73*(5), 1–18. https://doi.org/10.1509/jmkg.73.5.1

Pascarella, L., Spadini, D., Palomba, F., Bruntink, M., & Bacchelli, A. (2018). Information needs in contemporary code review. *Proceedings of the ACM on Human-Computer Interaction*, *2*(CSCW), Article 135, 1–27. https://doi.org/10.1145/3274404

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*. https://doi.org/10.48550/arXiv.2302.06590

Perry-Smith, J. E., & Mannucci, P. V. (2017). From creativity to innovation: The social network drivers of the four phases of the idea journey. *Academy of Management Review*, *42*(1), 53–79. https://doi.org/10.5465/amr.2014.0462

Phan, T., et al. (2025). AI-generated code quality: A comparative analysis of human and machine-written software. *Working Paper*.

Piezunka, H., & Dahlander, L. (2015). Distant search, narrow attention: How crowding alters organizations' filtering of suggestions in crowdsourcing. *Academy of Management Journal*, *58*(3), 856–880. https://doi.org/10.5465/amj.2012.0458

Piezunka, H., & Dahlander, L. (2019). Idea rejected, tie formed: Organizations' feedback on crowdsourced ideas. *Academy of Management Journal*, *62*(2), 503–530. https://doi.org/10.5465/amj.2016.0703

Piskorski, M. J., & Toffel, M. W. (2013). *A social strategy: How we profit from social media*. Harvard Business Review Press.

Podolny, J. M., & Baron, J. N. (1997). Resources and relationships: Social networks and mobility in the workplace. *American Sociological Review*, *62*(5), 673–693. https://doi.org/10.2307/2657354

Raisch, S., & Krakowski, S. (2021). Artificial intelligence and management: The automation–augmentation paradox. *Academy of Management Review*, *46*(1), 192–210. https://doi.org/10.5465/amr.2018.0072

Raj, M., Berg, J. M., & Seamans, R. (2026). The artificial intelligence disclosure penalty: Humans persistently devalue AI-generated creative writing. *Journal of Experimental Psychology: General*, *155*(4), 896–915. https://doi.org/10.1037/xge0001889

Ram, A., Sawant, A. A., Castelluccio, M., & Bacchelli, A. (2018). What makes a code change easier to review? An empirical investigation on code change reviewability. In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)* (pp. 201–212). ACM. https://doi.org/10.1145/3236024.3236080

Reif, J. A., Larrick, R. P., & Soll, J. B. (2025). Evidence of a social evaluation penalty for using AI. *Proceedings of the National Academy of Sciences*, *122*(19), e2426766122. https://doi.org/10.1073/pnas.2426766122

Rice, R. M. (2018). When hierarchy becomes collaborative: Collaboration as sensemaking frame in high reliability organizing. *Corporate Communications: An International Journal*, *23*(4), 599–613. https://doi.org/10.1108/CCIJ-04-2017-0032

Rigby, P. C., & Bird, C. (2013). Convergent contemporary software peer review practices. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)* (pp. 202–212). ACM. https://doi.org/10.1145/2491411.2491444

Russell, J., Karpinska, M., & Iyyer, M. (2025). People who frequently use ChatGPT for writing tasks are accurate and robust detectors of AI-generated text. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 5342–5373). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.267

Sadowski, C., Söderberg, E., Church, L., Sipko, M., & Bacchelli, A. (2018). Modern code review: A case study at Google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice* (pp. 181–190). ACM. https://doi.org/10.1145/3183519.3183525

Santoro, E., & Monin, B. (2023). The AI effect: People rate distinctively human attributes as more essential to being human after learning about artificial intelligence advances. *Journal of Experimental Social Psychology*, *107*, 104464. https://doi.org/10.1016/j.jesp.2023.104464

Sarkar, S. K. (2025). *AI agents, productivity, and higher-order thinking: Early evidence from software development*. SSRN Working Paper No. 5713646.

Schilke, O., & Reimann, M. (2025). The transparency dilemma: How AI disclosure erodes trust. *Organizational Behavior and Human Decision Processes*, *188*, 104405. https://doi.org/10.1016/j.obhdp.2025.104405

Song, F., Agarwal, A., & Wen, W. (2024). The impact of generative AI on collaborative open-source software development: Evidence from GitHub Copilot. *arXiv preprint arXiv:2410.02091*. https://doi.org/10.48550/arXiv.2410.02091

Spence, M. (1973). Job market signaling. *The Quarterly Journal of Economics*, 87(3), 355–374.

Srivastava, S. B., Goldberg, A., Manian, V. G., & Potts, C. (2018). Enculturation trajectories: Language, cultural adaptation, and individual outcomes in organizations. *Management Science*, *64*(3), 1348–1364. https://doi.org/10.1287/mnsc.2016.2671

Stack Overflow. (2024). 2024 Stack Overflow developer survey. https://survey.stackoverflow.co/2024/

Teece, D. J. (1986). Profiting from technological innovation: Implications for integration, collaboration, licensing and public policy. *Research Policy*, 15(6), 285–305.

Thompson, J. D. (1967). *Organizations in action: Social science bases of administrative theory*. McGraw-Hill.

Tidelift. (2024). *The 2024 Tidelift state of the open source maintainer report*. https://tidelift.com/open-source-maintainer-survey-2024

Török, G., Pomiechowska, B., Csibra, G., & Sebök, M. (2021). Rationality in joint action: Maximizing coefficiency in coordination. *Psychological Science*, 32(10), 1624–1636.

Tsay, J., Dabbish, L., & Herbsleb, J. (2014). Influence of social and technical factors for evaluating contribution in GitHub. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)* (pp. 356–366). ACM. https://doi.org/10.1145/2568225.2568315

Uhl-Bien, M., & Maslyn, J. M. (2003). Reciprocity in manager-subordinate relationships: Components, configurations, and outcomes. *Journal of Management*, *29*(4), 511–532. https://doi.org/10.1016/S0149-2063(03)00023-0

von Krogh, G., Spaeth, S., & Lakhani, K. R. (2003). Community, joining, and specialization in open source software innovation: A case study. *Research Policy*, 32(7), 1217–1241.

Watanabe, M., Li, H., Kashiwa, Y., Reid, B., Iida, H., & Hassan, A. E. (2026). On the use of agentic coding: An empirical study of pull requests on GitHub. *ACM Transactions on Software Engineering and Methodology*. https://doi.org/10.1145/3798166

Weiner, B. (1980). A cognitive (attribution)-emotion-action model of motivated behavior: An analysis of judgments of help-giving. *Journal of Personality and Social Psychology*, *39*(2), 186–200. https://doi.org/10.1037/0022-3514.39.2.186

Wessel, M., Serebrenik, A., Wiese, I. S., Steinmacher, I., & Gerosa, M. A. (2022). Quality gatekeepers: Investigating the effects of code review bots on pull request activities and dynamics. *Empirical Software Engineering*, *27*, Article 108. https://doi.org/10.1007/s10664-022-10130-9

Xiang, Y., Landy, J., Cushman, F. A., Vélez, N., & Gershman, S. J. (2025). People reward others based on their willingness to exert effort. *Journal of Experimental Social Psychology*, *116*, 104699. https://doi.org/10.1016/j.jesp.2024.104699

Yeverechyahu, D., Mayya, R., & Oestreicher-Singer, G. (2025). *The impact of large language models on open-source innovation: Evidence from GitHub Copilot*. arXiv preprint arXiv:2409.08379, version 3 (June 10, 2025). https://arxiv.org/abs/2409.08379v3

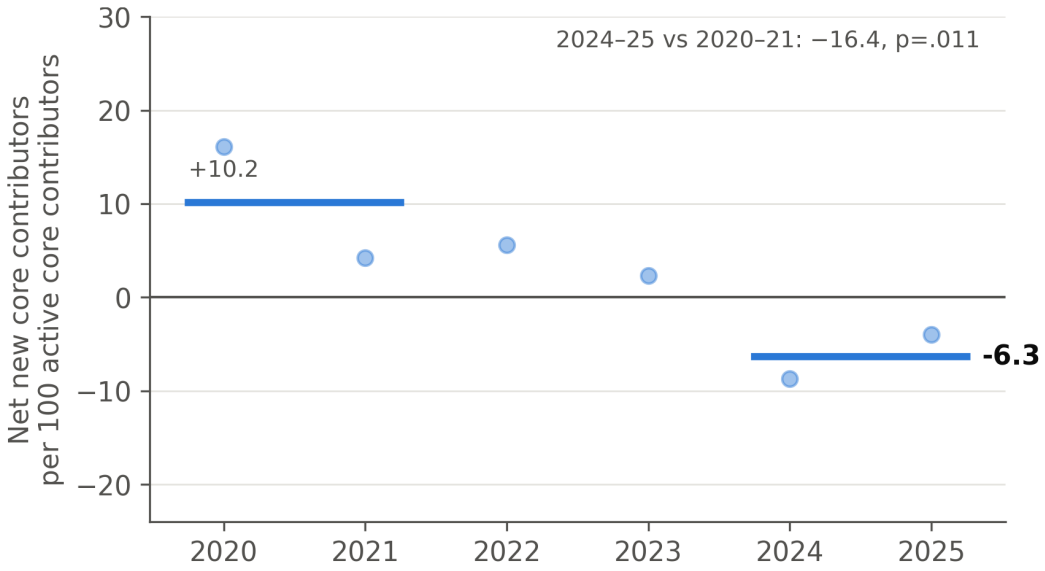
Yu, H., Gao, X., Zhou, Y., & Zhou, X. (2018). Decomposing gratitude: Representation and integration of cognitive antecedents of gratitude in the brain. *The Journal of Neuroscience*, *38*(21), 4886–4898. https://doi.org/10.1523/JNEUROSCI.2944-17.2018

Zhang, X., Yu, Y., Gousios, G., & Rastogi, A. (2023). Pull request decisions explained: An empirical overview. *IEEE Transactions on Software Engineering*, *49*(2), 849–871. https://doi.org/10.1109/TSE.2022.3165056

Zhang, X., Yu, Y., Wang, T., Rastogi, A., & Wang, H. (2022). Pull request latency explained: An empirical overview. *Empirical Software Engineering*, *27*, Article 126. https://doi.org/10.1007/s10664-022-10143-4

Zhang, S., Xu, J., & Alvero, A. J. (2025). Generative AI meets open-ended survey responses: Research participant use of AI and homogenization. *Sociological Methods & Research*, *54*(3), 1197–1242. https://doi.org/10.1177/00491241251327130

Zhou, M., & Mockus, A. (2012). What makes long-term contributors: Willingness and opportunity in OSS communities. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)* (pp. 518–528). IEEE. https://doi.org/10.1109/ICSE.2012.6227164



Notes: The figure shows the net flow into the senior core—contributors crossing their tenth merged PR minus previously active core contributors with no PRs in the calendar year—per 100 active core contributors, adjusted for repository age. Faded points are yearly estimates; the horizontal segments are the 2020–21 and 2024–25 averages. The difference between the two averages is −16.4 per 100 core contributors (p = .011, repository-clustered). Part of the decline predates the diffusion of GAI coding assistants, so we present it as context on the state of core renewal rather than as an effect of GAI.

Figure 10: Net flow into the senior contributor core, 2020–2025.

Theory concept	**GitHub operationalization**
Work submitted by contributor	Pull request code change
Responding collaborator	Maintainer or reviewer responding to the pull request
High-effort response	Inline comments, constructive feedback
Low-effort response	General comments, non-constructive feedback, rejection
Effort opacity	Pull request code appears AI-assisted, making human effort harder to infer
Scarcity of reciprocal effort	High-activity repositories / large review queues
Alternative information about effort	Contributor's prior history with the project or reviewer
Repair channel	Pull request description that makes human effort visible

Table 10: Mapping the theory to the GitHub setting.

A Appendix

A.1 Is the low effort visible in the write-up?

Pull requests classified as GAI-predicted have longer descriptions and more frequently mention motivation and testing. Length and surface structure, however, need not indicate substantive en-

Theme	Illustrative response
Description and explanation	"The biggest factor was how well contributors explained their thinking. Good code matters, but a clear description shows the person actually understands the problem" (P10).
Scope discipline	Participants down-ranked pull requests that added unrequested behavior or changed more than needed for the bug fix.
Safety and correctness	"Least risk of causing any PROD issues is how I ranked it" (P44).
AI as low ownership	"Bottom positions were assigned to the AI-assisted contributions due to a lack of individual oversight" (P21).
AI acceptable with judgment	"Using AI is fine, but you need to know when it is wrong. It's a tool, not a short-cut" (P81).

Table 11: Open-Ended Reasons for Review Priority

gagement. This section examines whether descriptions accompanying GAI-predicted code contain concrete evidence of reasoning, verification, and design. The code classifier does not establish actual GAI use or the amount of editing performed by a contributor; it identifies pull requests whose submitted code exhibits features associated with GAI generation.

To conduct this analysis, I recovered each description as it appeared when the pull request was first submitted and measured two features. First, I rated overall description effort from 0, indicating empty or boilerplate text, to 2, indicating substantive explanation. Second, I recorded whether the description contained a concrete reason for the change, actual testing evidence, and an explanation of the design. A language model applied fixed coding rules while remaining blind to the code-based GAI classification. I benchmarked these ratings against 5,914 hand-coded descriptions; the two sets of ratings agreed within one point 99.7

Finding 1: longer, but thinner. On the surface AI-flagged descriptions look more effortful—they are longer (median 505 versus 332 characters; Figure 11, left). But length is not substance. When we compare descriptions of the *same length*, AI-flagged ones are more often thin (Figure 11, right; Table 12). A raw comparison shows no gap—because AI users simply write *more*. The gap only appears once we compare like-for-like: then AI-flagged descriptions are about 5 percentage points more likely to be low-effort for their length.

Finding 2: more of the signals of effort, less of the substance. Figure 12 makes the point sharply. AI-flagged authors show *more* of every surface signal of effort: they write longer descriptions

(+12pp), use more motivation language (+10pp), and mention testing more often. But for the *same length*, their descriptions contain *less* real substance: they are 8 points less likely to give a genuine reason, 5 points less likely to show real testing evidence, and 3 points less likely to explain the design (all statistically significant). In short, AI amplifies the *signals* of effort while delivering *less* of the thing itself. The box below shows a typical case: a rejected, AI-flagged pull request whose description has every section heading of a careful write-up and no content in any of them.

> ***A real (rejected, AI-flagged) description — "Feat/run status notification":***
> *## Description*
> *## Development notes*
> *## QA notes*
> *## Checklist*
> *□ Read the contributing guidelines □ Opened this PR as a Draft □ Updated the documentation □ Added new entries to RELEASE.md □ Added tests to cover my changes*
>
> It has the shape of a thorough description—four labelled sections and a checklist—and none of the substance: every section is empty and every box unchecked.

Finding 3: a good description does help—matching the experiment. A more effortful description is linked to a lower chance of rejection (about 2 percentage points lower per effort point, holding length and project fixed), with no sign that this help is smaller for AI-flagged work. This pattern is consistent with the mechanism experiment in Section 6, where adding genuine effort cues to the description improved how reviewers treated AI-disclosed work. Putting real effort into the write-up is worth it.

Finding 4: the code penalty is a separate problem. Even after we account for how good the description is, the penalty on AI-flagged code barely moves (from +28.8 to +28.5 percentage points). The two act on their own: a better description lowers rejection a little for everyone, while the much larger AI-code penalty sits on top and is not explained away by the write-up.

Why this matters. Effort is hard to see. AI-flagged authors produce descriptions that carry *more* of the visible marks of diligence—length, structure, the language of motivation and testing—while carrying *less* of the substance behind them. The most obvious cue therefore points the wrong way. A genuinely effortful description does help, as the experiment shows; but there is also a low-effort signal in the code that a polished-looking write-up does not cover up. The tell is in the work itself, not in the way it is packaged.

A.2 Detailed Robustness Checks

The observational study documents a strong association between AI-predicted code and the reciprocal effort returned by project collaborators. Because the study is observational, the main empirical

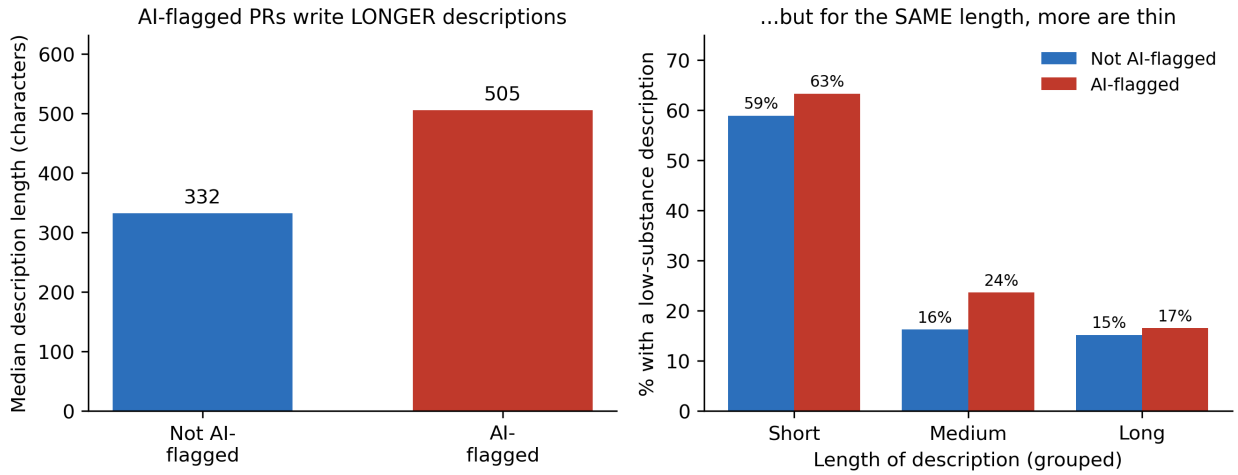


Figure 11: AI-flagged pull requests write *longer* descriptions (left), yet for the *same length* their descriptions are more often thin on substance (right). Length looks like effort but is not.

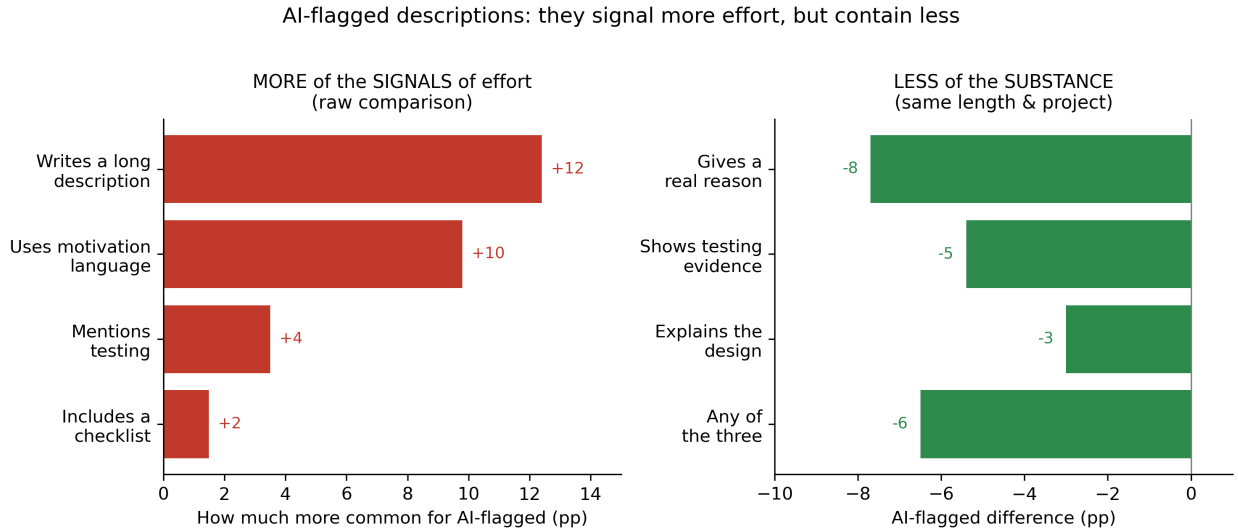


Figure 12: More signal, less substance. *Left:* AI-flagged descriptions show more of every surface sign of effort (raw comparison). *Right:* yet for the same length and project they contain less genuine substance—fewer real reasons, less testing evidence, less design detail (all differences statistically significant).

concern is not a single alternative explanation, but a family of related threats. AI-predicted pull requests may be lower quality, more speculative, more likely to be abandoned, more likely to come from newcomers, routed to stricter projects or collaborators, or submitted by less experienced authors. The penalty may also be specific to volunteer open-source collaboration rather than a broader feature of organized work.

This section subjects the result to these alternatives. The goal is not to turn the observational

	(1)	(2)	(3)	(4)
Panel A. Description effort score (0–2)				
AI-flagged	0.033	0.004	−0.041	−0.071***
	(0.037)	(0.031)	(0.026)	(0.025)
Panel B. Chance the description is low-effort (×100)				
AI-flagged	−0.82	0.29	3.80**	4.99***
	(2.19)	(1.92)	(1.54)	(1.50)
Compare same length			✓	✓
Compare same project		✓		✓
Pull requests	13,614	13,614	13,614	13,614

Notes. All 13,614 pull requests across 166 projects. Description effort scored 0–2 by a language model blind to the AI flag, checked against 5,914 hand-scored descriptions (99.7% agree within one point). "Low-effort" means a score of 0. "Compare same length" adjusts for log(characters); "compare same project" adds project fixed effects. Standard errors clustered by author in parentheses. $^{*}p < 0.10$, $^{**}p < 0.05$, $^{***}p < 0.01$.

Table 12: PR-description effort and AI-flagged code. A raw comparison hides the gap; comparing like-for-like reveals it.

estimate into a causal effect of actual AI use. Rather, the goal is to ask whether the field pattern predicted by effort opacity survives the most direct observable explanations. Across checks, the answer is yes. The AI-predicted penalty attenuates under some conservative specifications, but it remains large, statistically precise, and substantively consistent with the theory.

A.2.1 Do Human Developers Align With The Classifier On Which Code is AI-assisted vs Not?

The observational study shows that AI-flagged pull requests face higher rejection rates on GitHub. But this evidence rests on a key assumption: the classifier must be capturing AI code features that human reviewers could also notice. If the classifier were detecting features that are invisible to developers, then the rejections in the observational study could reflect an unobserved confound rather than maintainer responses to AI-like signals.

I test this assumption directly. In a pre-registered between subjects study, I ask whether developers can distinguish AI-generated code from human-written code when evaluating a contribution in isolation, and whether their classifications are tied to perceived human effort. This design mirrors the observational setting. The study has two goals. First, it tests whether developers have above-chance ability to detect AI-generated code. Second, it tests whether effort is the cue through which detection works. The theory does not require developers to run a conscious "AI detector." It requires only that AI-generated code sometimes disrupts the effort signals developers use when judging work.

	(1)	(2)	(3)	(4)
AI co-author credit	−0.063*	−0.053*	−0.053	−0.001
	(0.037)	(0.029)	(0.034)	(0.020)
Newcomer		0.191***	0.191***	0.043***
		(0.024)	(0.024)	(0.011)
Credit × Newcomer			−0.002	
			(0.046)	
Size controls	Yes	Yes	Yes	Yes
Repository fixed effects	Yes	Yes	Yes	—
Contributor fixed effects	—	—	—	Yes
Observations	21,563	21,563	21,563	17,848

Notes: Linear probability models; outcome is rejection, self-merged pull requests excluded. The AI co-author credit is an attribution line that AI coding tools add to commits by default and that contributors may remove; it is visible in the contribution record. Newcomer indicates the contributor's first observed pull request in the repository (contributors active before the sample window may be misclassified as newcomers). Size controls are log lines added and log files changed. Column (4) is estimated within contributor (contributor fixed effects; sample restricted to contributors with two or more pull requests; repository fixed effects omitted). Standard errors in parentheses, clustered by contributor. $^{*}p < 0.10$, $^{**}p < 0.05$, $^{***}p < 0.01$.

Table 13: A visible AI co-author credit does not predict rejection

I recruited 150 developers through Prolific for this study. Subjects were screened for recent coding experience and code comprehension. After applying pre-registered quality filters for speeding, invariant responding, and positional responding, the final sample contains 126 participants and 630 trial-level observations. Each participant completed five trials. The stimuli consisted of five matched pairs of code snippets. Each pair contained a human-written snippet drawn from pre-AI GitHub pull requests and an AI-generated equivalent produced by a large language model. These snippets came from the known-source corpus used to build the classifier. This design is deliberate. The GitHub pull requests do not contain verified information about whether the contributor used AI. This study therefore does not prove the true source of the GitHub pull requests. Instead, it tests the assumption needed for the interpretation of a possibly reason why AI-predicted code is rejected at higher rates: the code distinction used by the classifier is also visible enough for developers to use. More details on the experiment are in the Appendix.

Figure 4 Panel A reports classification accuracy. Developers distinguish AI-generated from human-written code at above-chance rates. Overall accuracy was 64.0%, significantly above the 50% chance baseline, $t(125) = 6.23$, $p < .001$, $d = 0.55$, 95% CI $[0.37, 0.74]$. Developers distinguish AI-generated from human-written code at above-chance rates. Overall accuracy was 64.0%, significantly above the 50% chance baseline, $t(125) = 6.23$, $p < .001$, $d = 0.55$, 95% CI $[0.37, 0.74]$.

	(1) First observed PR in repository	(2) Prior merge in repository	(3) Collaborator familiarity	(4) Collaborator familiarity
AI-predicted PR	0.2896***	0.3229***	0.3454***	0.1866***
	(0.0195)	(0.0188)	(0.0163)	(0.0148)
Is Newcomer (N)	0.1002***	0.0045	−0.1818***	−0.1004***
	(0.0090)	(0.0072)	(0.0097)	(0.0078)
AI-predicted PR × N	0.0445†	−0.0085	−0.1702***	−0.0366
	(0.0266)	(0.0232)	(0.0304)	(0.0269)
Repository FE	Yes	Yes	Yes	No
Responding-collaborator FE	No	No	No	Yes
Month FE	Yes	Yes	Yes	Yes
Controls	Yes	Yes	Yes	Yes
Observations	13,614	13,614	13,614	13,574
R^2	0.248	0.235	0.282	0.529

Notes: Linear probability models. The dependent variable is rejection. The standing measure M differs by column. Column 1 uses an indicator for whether the pull request is the contributor's first observed eligible pull request to the repository. Column 2 uses an indicator for whether the contributor has at least one prior merged pull request in the repository. Columns 3 and 4 use collaborator familiarity, an indicator for whether the responding collaborator handling the pull request has previously merged work from the same contributor. Column 4 replaces repository fixed effects with responding-collaborator fixed effects. Controls are initial additions, deletions, changed files, mean and maximum cyclomatic complexity, and mean maintainability index. Standard errors clustered by author in parentheses.

Significance: $^{\dagger}p < 0.10$, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

Table 14: The AI-Predicted Penalty and Contributor Status

Detection is meaningful but imperfect. Participants correctly identified human-written code 72.3% of the time, compared with 56.6% for AI-generated code. The difference is statistically significant, paired $t(119) = 3.22$, $p = .002$, $d = 0.29$. Participants also showed a modest human-response bias, choosing "Human" on 57% of trials. The second goal of the study is to check the correlation between perceived effort and perceived AI-assisted code. The theory predicts that developers should be more likely to classify code as AI-generated when the code appears to contain less human effort. Figure 15b shows this pattern among AI-generated snippets. When code is predicted to be AI-generated code it is rated as lower effort, and vice versa, when code is predicted to be more likely to be human written, it is rated as higher effort.

This study supports the main assumption needed to interpret the observational study results. Developers can detect AI-generated code at above-chance rates, but imperfectly. Overall accuracy is 64.0%, and signal detection analysis confirms moderate discriminability, $d' = 0.632$. The classifier signal is therefore not entirely invisible to human evaluators. Developers are more likely to classify code as low effort when they believe it was AI-assisted. This connects the classifier-based obser-

vational evidence to the theory: AI-predicted code can weaken the effort signals developers use to judge whether a contribution deserves their attention. The moderate detection rate also helps explain the observational study results. If developers detected AI-generated code perfectly, AI-flagged contributions would be uniformly penalized. If they could not detect it at all, the observational study rejection penalty would be difficult to interpret as a response to AI-like signals. The evidence falls between these extremes. AI-generated code is partly legible. Some AI-generated work preserves effort signals and passes as human-written. Other AI-generated work disrupts those signals and is classified as AI-generated.

A.2.2 Functional Quality: Automated Test Outcomes

The most direct alternative explanation is that AI-predicted pull requests are rejected because they are lower quality. The main specifications already control for observable code characteristics, including contribution size, cyclomatic complexity, and maintainability. A stronger version of the quality alternative is functional: perhaps AI-predicted pull requests are rejected because they simply do not work.

I examine this possibility using continuous-integration data where available. Continuous-integration systems run automated tests, linters, build checks, and related validation procedures on submitted code. These checks are not a complete measure of quality, but they are among the most concrete quality signals available to project collaborators at review time.

Among the 635 pull requests across 70 repositories with observed CI/CD outcomes, AI-predicted pull requests are less likely to pass all automated checks. As Table **??** shows, 42.7% of AI-predicted pull requests pass all tests, compared with 51.9% of non-AI-predicted pull requests ($p = .026$). This difference is important: there is some functional-quality gap in the subset where automated tests are observed.

However, this quality gap explains little of the rejection penalty. Table 15 conditions directly on whether a pull request passed all observed tests. The CI/CD results show that functional quality differences exist in the observed-test subset. They therefore rule out an overly strong version of the argument in which AI-predicted and non-AI-predicted work are identical. However, conditioning on test passage explains little of the rejection penalty, suggesting that observable functional failure is not sufficient to account for the field pattern. In Column 1, the AI-predicted rejection penalty is 30.1 percentage points. Adding the test-passage indicator in Column 2 moves the coefficient only to 29.6 percentage points. With repository and month fixed effects in Column 4, the coefficient remains 19.7 percentage points. The interaction between AI-predicted status and passing all tests is statistically indistinguishable from zero. In the raw data, even among pull requests that pass all automated checks, AI-predicted pull requests are rejected 57.0% of the time, compared with 29.3% for non-AI-predicted pull requests.

These results do not imply that AI-predicted and non-AI-predicted pull requests are equal on every unobserved dimension of quality. Automated tests do not capture architectural fit, project

	(1) Baseline	(2) + Test	(3) + Repo FE	(4) + Repo, Month FE	(5) Interaction
AI-predicted PR	0.301***	0.296***	0.190***	0.197***	0.217***
	(0.039)	(0.040)	(0.038)	(0.038)	(0.049)
Passed all tests		−0.057	−0.061*	−0.086**	−0.069*
		(0.038)	(0.035)	(0.036)	(0.042)
AI-predicted PR × Passed all tests					−0.047
					(0.070)
Repository FE	No	No	Yes	Yes	Yes
Month FE	No	No	No	Yes	Yes
Observations	635	635	635	635	635
R^2	0.086	0.090	0.557	0.591	0.591

Notes: Linear probability models. The dependent variable is an indicator for rejection. Test outcomes are measured at initial submission. Heteroskedasticity-robust standard errors are reported in parentheses. In Column 5, the AI-predicted penalty is 21.7 percentage points when tests fail and 17.1 percentage points when tests pass. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

Table 15: Rejection Probability Conditioning on Automated Test Outcomes

priorities, long-run maintainability, or whether the contributor can explain and maintain the change. But the CI/CD evidence rules out the simplest functional-quality account: the rejection penalty is not absorbed by whether the submitted code passes automated checks.

A.2.3 The Copilot-Trailer Natural Experiment

In April 2026, a bug in a Visual Studio Code update caused the editor to automatically append a "Co-authored-by: Copilot" line to commits made through its interface. For about two weeks (April 22 to May 6, 2026), pull requests from affected contributors carried a visible AI-attribution label that the contributors had not chosen to add. Whether a given contributor's pull requests were labeled depended on when their editor happened to install the update — timing unrelated to the contributor, the project, or the code. When the bug was reverted, the labels stopped. This accident does what no observational comparison can: it attaches the AI label to contributions independently of anyone's choice, so any change in how labeled contributions were treated can be read as the effect of the label itself.

The affected population is a different set of repositories from the main sample. The main analysis covers 166 mature, widely used Python projects, whose contributors typically commit from the command line; the buggy editor feature fired mostly in a long tail of smaller and younger projects. From the roughly 59,000 repositories that received at least one buggy trailer, I keep the 617 Python repositories with at least one substantive pull request; exactly one repository overlaps with the main sample. The two analyses are therefore complementary: the main sample provides the observational result in critical infrastructure projects, and the experiment provides the causal test of the label in

an independent population. One further cleaning step matters. A small number of repositories were "agent farms," in which an orchestrator account opened and closed hundreds of automated pull requests; a single such repository accounted for the majority of rejections in the raw window. I flag and exclude 66 such repositories (flagging rule: thirty or more bug-window pull requests with one account opening or closing at least 40% of them), and restrict to pull requests decided by someone other than their author.

The label did not change decisions, but it changed attention. Comparing labeled and unlabeled pull requests within the bug window, with author fixed effects, the estimated rejection difference is small and not statistically distinguishable from zero +0.006 ($p = .83$). Scrutiny, by contrast, rose: labeled pull requests received about 28% more reviews ($p = .02$) and 27% more reviews from people other than the author's usual collaborators ($p = .007$), with review comments up a similar amount. Requests for changes roughly doubled in raw counts, while approvals and final outcomes were unchanged. In short, maintainers who saw the AI label looked harder at the work and then decided the same way they otherwise would have.

If maintainers penalized AI use as such, an exogenously attached AI label is exactly what should have triggered rejections; it did not. This supplementary analysis finds increased review activity and no statistically detectable rejection difference following involuntary AI attribution. Because perceived creator effort is not measured, it does not establish whether the label changed effort inferences. The rejection penalty documented in the main analysis attaches to properties of the submitted code, not to the attribution. Two cautions apply: the experiment's repositories are smaller and younger than the main sample's, so magnitudes need not transfer; and the bug window is short, so longer-run responses to labeling are outside its reach.

A.2.4 Newcomer Composition

A third concern is that AI-predicted pull requests may disproportionately come from contributors with little prior relationship to the project. New contributors are often more likely to submit work that violates project norms, duplicates existing work, or requires more guidance. If AI-predicted work is simply newcomer work, the main estimates may capture contributor standing rather than effort opacity.

To address this concern, I add an indicator for whether the pull request is the contributor's first eligible contribution to the repository. The AI-predicted coefficients remain substantively unchanged. With the first-time-contributor indicator included, AI-predicted pull requests are 30.9 percentage points more likely to be rejected, receive 0.25 fewer inline comments, and receive 0.28 more general comments. The average field pattern is therefore not driven by AI-predicted pull requests being submitted by first-time contributors.

This check is distinct from the heterogeneity analysis below. The robustness question is whether newcomer composition explains away the average penalty. It does not. The heterogeneity question is whether the penalty is stronger for contributors with little prior history, as the theory predicts. I

return to that question in later.

A.2.5 Richer Controls and Fixed Effects

A fourth concern is that the main estimates reflect differences in project strictness, collaborator strictness, or contributor experience that are not captured by repository and month fixed effects. I therefore estimate additional specifications that add richer controls and fixed effects.

The additional controls include repository selectivity, responding-collaborator selectivity, first-time-contributor status, and author experience. Repository selectivity captures the baseline strictness of the project. Responding-collaborator selectivity is a leave-one-out measure of the collaborator's tendency to reject pull requests, excluding the focal pull request. Author experience captures the contributor's prior activity in the sample. These variables address the possibility that AI-predicted pull requests are concentrated in stricter projects, handled by stricter collaborators, or submitted by less experienced contributors.

Table 16 reports the results. Column 1 includes the richer observable controls. Columns 2–5 add author, repository, and responding-collaborator fixed effects in different combinations. The AI-predicted coefficient remains positive and statistically significant in every specification. The magnitude falls relative to the main rejection estimate, especially when rich selectivity controls are included, but the remaining association is still economically large: even the most demanding specifications imply a rejection penalty of 11–16 percentage points.

These estimates should be interpreted conservatively. Responding-collaborator identity can itself be partly determined by the review process, so fixed effects for the collaborator who ultimately closes or merges the pull request may absorb some of the response being studied. The purpose of these specifications is therefore not to define the preferred estimate, but to ask whether the result is an artifact of obvious selection into stricter projects, stricter handlers, or less experienced authors. It is not.

A.2.6 Sensitivity to Selection on Unobservables

The previous checks address observable sources of selection. I also assess how strong selection on unobservables would need to be to explain away the AI-predicted rejection penalty, using the method developed by Oster (2019).

A restricted comparison from no controls to basic code-complexity controls suggests substantial coefficient stability: the coefficient on AI-predicted status changes only from 0.363 to 0.362, while R^2 increases from 0.069 to 0.071. This comparison yields a proportional-selection ratio well above Oster's benchmark of 1. However, this restricted comparison is not the most informative one because the basic controls explain relatively little variation in rejection.

I therefore use the richer observable specification in Column 1 of Table 16 as the controlled model. In that specification, the coefficient is 0.124 and R^2 is 0.511. Using this richer comparison, the proportional-selection ratio remains above 1, though by a narrower margin than in the restricted

	Rejected (0/1)				
	(1)	(2)	(3)	(4)	(5)
AI-predicted PR	0.124***	0.111***	0.148***	0.157***	0.137***
	(0.012)	(0.015)	(0.012)	(0.013)	(0.017)
Additional controls					
Repository selectivity	✓	✓	✓	✓	✓
Responding-collaborator selectivity[a]	✓	✓	✓	✓	✓
First-time contributor	✓	✓	✓	✓	✓
Author experience	✓	✓	✓	✓	✓
Author FE	No	Yes	No	No	Yes
Repository FE	No	No	Yes	No	Yes
Responding-collaborator FE	No	No	No	Yes	Yes
R^2	0.511	0.681	0.515	0.509	0.490
Observations	13,511	10,235	13,511	13,335	8,851

Notes: Linear probability models predicting pull-request rejection. "AI-predicted PR" indicates that the submitted code is classified as AI-predicted by the code-based classifier. Repository selectivity and responding-collaborator selectivity are leave-one-out measures excluding the focal pull request. Author experience captures prior author activity in the sample. Samples differ across columns because fixed effects require within-unit variation. Standard errors are in parentheses.

[a] Leave-one-out measure calculated excluding the focal pull request.

Significance: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

Table 16: Robustness: AI-Predicted Rejection Penalty with Richer Controls and Fixed Effects

comparison. I interpret this evidence cautiously. It suggests that unobserved confounding would need to be large relative to the observed controls to fully explain away the AI-predicted penalty. It does not eliminate the possibility of selection on unobservables, which remains a limitation of the observational design.

A.2.7 Boundary Check: Paid Employees in Company-Owned Repositories

A final concern is external validity. The main sample comes from open-source projects, where contributions are often voluntary and reciprocity norms may be unusually strong. If the AI-predicted penalty reflects something peculiar to volunteer peer production, it should be weaker or absent when contributors are paid professionals working on repositories owned by their own employer.

I examine this possibility in the subset of repositories owned by identifiable corporate entities. I restrict the sample to 21 company-owned repositories, yielding 7,466 pull requests. Within these

	Rejection rate		
Contributor	Non-AI	AI-predicted	AI-predicted penalty
External (n = 5,589)	19.8%	64.9%	+45.1pp
Employee (n = 1,717)	25.6%	72.4%	+46.8pp

Interaction (AI-predicted × Employee), LPM with organization fixed effects: $\beta = 0.006$, $p = .87$.

Notes: The sample is restricted to company-owned repositories with classifiable employee status. Employees are identified using public organization membership and commit-author email domains matching the owning firm. These are public company-owned repositories rather than private internal repositories, so the result should be interpreted as a boundary check rather than a direct estimate of private within-firm collaboration.

Table 17: The AI-Predicted Rejection Penalty Persists for Paid Employees

repositories, I distinguish pull requests authored by employees of the owning firm from those authored by external contributors. I identify employees using two signals: public membership in the owning GitHub organization and commit-author email domains matching the company's domain. Real, non-privacy-masked commit domains are recoverable for 93% of company-repository pull requests; excluding the remainder leaves 7,306 classifiable pull requests, of which 1,717 are employee-authored and 5,589 are externally authored.

Table 17 reports rejection rates by contributor status and AI-predicted status. The penalty is not confined to external volunteers. AI-predicted pull requests are rejected 46.8 percentage points more often than non-AI-predicted pull requests when authored by employees, compared with 45.1 percentage points among external contributors. A linear probability model interacting AI-predicted status with employee status, with organization fixed effects and standard errors clustered by organization, shows a large main effect of AI-predicted status and a precisely estimated null interaction: the penalty is statistically indistinguishable across employees and external contributors.

This boundary check matters for a management and strategy audience. The result suggests that the field pattern is not merely an artifact of unpaid volunteer collaboration. Even employees contributing to their own employer's public repositories face essentially the full AI-predicted penalty. These repositories are still public, company-owned repositories rather than private internal codebases, so they do not fully replicate within-firm collaboration. The employee measure is also conservative because public organization membership and commit-email domains may miss some true employees. Nonetheless, the evidence suggests that effort opacity is relevant beyond volunteer open source.

A.2.8 Operational Consequence: Time to Close

The preceding checks address alternative explanations for the rejection and response-composition results. I also examine an operational consequence: whether AI-predicted pull requests take longer

to resolve. This analysis is not a robustness check for the main effect; it is an auxiliary outcome that connects the field pattern to organizational value.

AI-predicted pull requests take 30.4 additional days to close in the fully specified model ($p < .001$). The delay is concentrated in high-activity repositories: the interaction between AI-predicted status and high-activity repository status is 18.3 days ($p < .05$). This pattern is consistent with the theory's organizational implication. When collaborators return less high-effort response and substitute toward lower-effort handling, work does not simply disappear; it can also take longer to resolve. The cost of effort opacity is therefore not only relational, but operational. Full time-to-close results are reported in Appendix A.4.

A.2.9 Summary of Robustness Checks

Taken together, the robustness checks narrow the set of plausible alternative explanations. The AI-predicted penalty is not fully explained by automated test failures, observable code characteristics, newcomer composition, stricter projects, stricter responding collaborators, author experience, or selection on observables. The penalty also appears among paid employees contributing to company-owned repositories, suggesting that the phenomenon is not confined to volunteer peer production.

These checks do not make the observational study causal. Unobserved differences in contributor motivation, task difficulty, architectural fit, or true production process may still contribute to the field pattern. The purpose of the study is to establish that the pattern predicted by effort opacity appears in a real collaborative system and survives the most direct observable alternatives. A pre-registered between-subjects study tests the key assumption behind the field measure, and the mechanism test experiment tests the mechanism and potential remedy for *effort opacity*.

A.3 Code Detection Study Details

Full details on the pre-registered study are available at osf.io/76nrt.

This section continues from the robustness checks section for the observational study. For each matched pair of code, subjects were randomly assigned to see either the human-written version or the AI-generated version. Subjects therefore saw each coding task only once. This single-stimulus design avoids making detection artificially easy through paired comparison and better approximates how maintainers evaluate contributions in the field. For each snippet, subjects completed two tasks. First, they classified the code as either human-written or AI-generated and reported confidence on a 50–100% slider. Second, they rated how much human effort they believed went into producing the code on a 1–10 scale. The order of these two measures was randomized within each trial. The classification task was incentive-compatible: participants received a $0.25 bonus for each correct classification and were told that each snippet had an equal probability of being human-written or AI-generated.

Because raw accuracy can combine true discriminability with response bias, I also use signal detection theory. Table 18 separates sensitivity from criterion. Sensitivity, d', is significantly greater

Metric	Mean	SD	t	p	d
Sensitivity (d')	0.632	0.986	6.991	$< .001$	0.64
Criterion (c)	0.139	0.467	3.249	.002	0.30

Notes: Signal-detection estimates are computed for participants with estimable hit and false-alarm rates. Positive values of d' indicate greater ability to discriminate AI-generated from human-written code. Positive values of c indicate a bias toward choosing "Human."

Table 18: Signal Detection Results

than zero, confirming genuine discriminability. Criterion, c, is also positive, confirming a modest bias toward labeling code as human-written. The bias does not explain away the detection result.

The same conclusion appears in a mixed-effects linear probability model with participant random intercepts. Actually AI-generated code increases the probability of choosing "AI" by 29.7 percentage points, $b = 0.297$, SE = 0.038, $z = 7.89$, $p < .001$. This result rules out the strongest version of the identification threat. The classifier is not only detecting features that exist in statistical space. The known-source AI signal is also visible enough for human developers to detect, though imperfectly.

The regression evidence confirms the visual pattern. In a mixed-effects linear probability model controlling for actual source and button order, perceived effort significantly predicts AI classification. The coefficient is $b = -0.046$, SE = 0.011, $z = -4.37$, $p < .001$. A one-point decrease in perceived effort is associated with a 4.6 percentage point increase in the probability of choosing "AI."

This relationship holds within both actual source conditions. Among AI-generated snippets, perceived effort is negatively correlated with choosing "AI," $r = -.205$, $p < .001$. Among human-written snippets, the relationship is weaker but still present, $r = -.131$, $p = .024$. Developers therefore use effort signals even when the code is actually human-written.

The mechanism test experiment manipulates visible effort directly. The code detection study therefore, shows why the penalty in the observational study is elevated but not universal. AI-generated code is not always perceived as low effort. It is penalized when the visible traces of human effort break down. Detection succeeds where effort signals fail, and fails where effort signals are preserved.

The most revealing result comes from jointly modeling actual source and perceived source. AI-generated code is not simply rated as lower effort. In fact, actual AI-generated snippets receive higher effort ratings once perceived source is included in the model. But code perceived as AI-generated receives lower effort ratings, conditional on actual source.

This suppression pattern is central to the argument. Actual AI-generated snippets can preserve or even amplify visible effort signals. That is why many AI-generated snippets pass as human-written. But when those effort signals fail, developers detect the code as AI-generated and attribute less human effort to it.

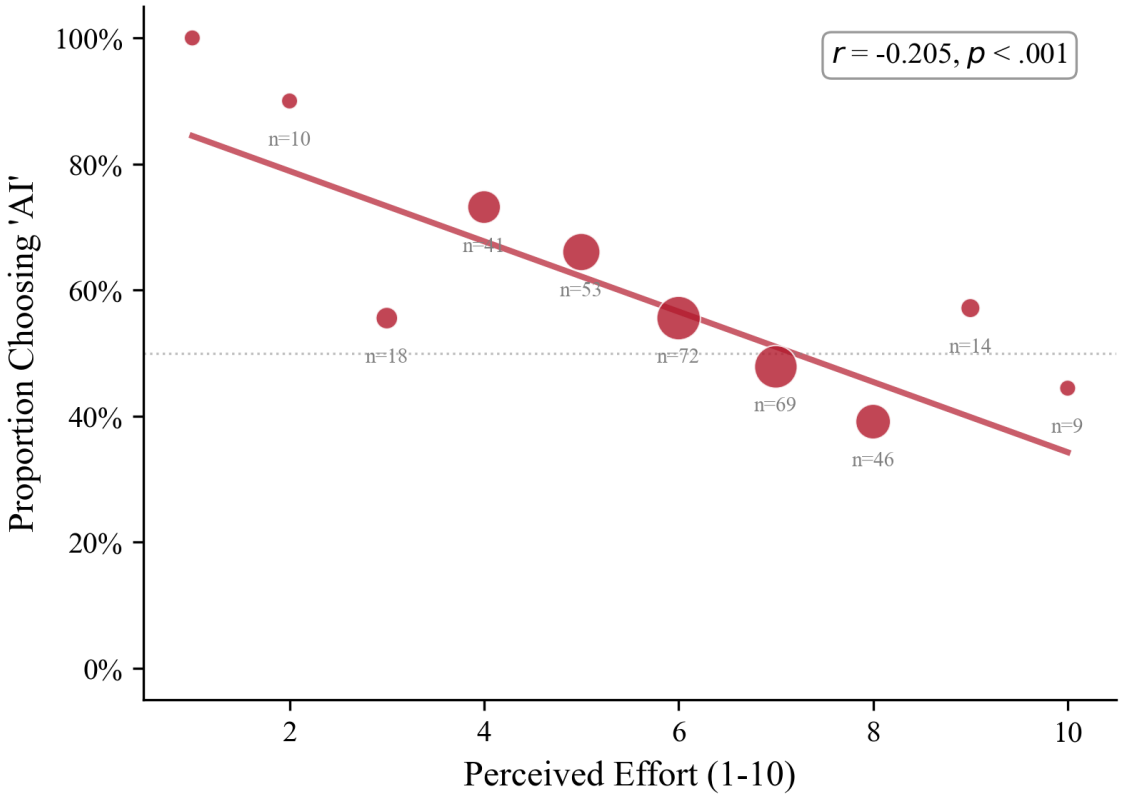


Figure 13: Perceived effort and AI detection among AI-generated snippets. The figure plots the proportion of participants choosing "AI" by perceived effort rating. Dot size reflects the number of observations in each effort bin. Lower perceived effort is associated with a higher probability of classifying the code as AI-generated.

A simple decomposition shows the same pattern. Among AI-generated snippets, those misclassified as human-written receive higher effort ratings than those correctly identified as AI-generated, 6.43 versus 5.67. Among human-written snippets, the same pattern appears directionally: snippets classified as human-written receive higher effort ratings than those classified as AI-generated, 5.68 versus 5.16.

The implication is not that AI-generated code always looks lazy. The sharper point is that AI detection succeeds where effort signals are disrupted and fails where effort signals are preserved.

	Model A Perceived source only	Model B Actual and perceived source
Intercept	5.950***	5.688***
	(0.118)	(0.129)
AI-generated, actual		0.670***
		(0.137)
Chose "AI," perceived	−0.389**	−0.605***
	(0.136)	(0.141)
Random intercept SD	0.913	0.917

Notes: Dependent variable is perceived human effort on a 1–10 scale. Standard errors are in parentheses. $^{**}p < .01$, $^{***}p < .001$.

Table 19: Actual Source, Perceived Source, and Effort Ratings

A.4 Work Efficiency

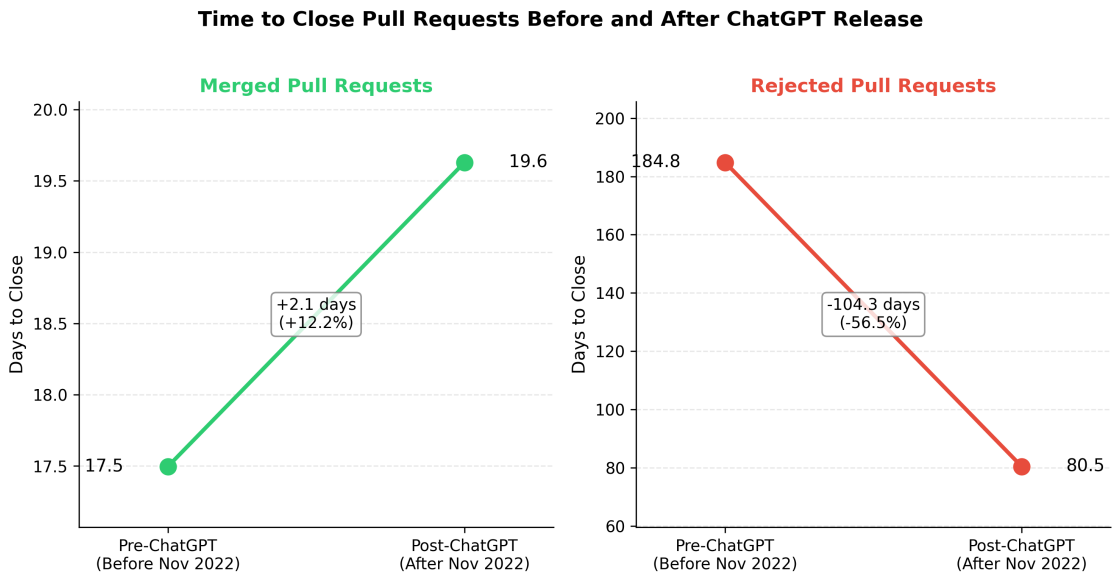


Figure 14: Average Number of Days Taken to Close a PR Before and After ChatGPT

(a) Developers classify AI-generated and human-written code above chance but imperfectly.

(b) Code perceived as AI-generated is attributed to lower contributor effort.

Figure 15: Perceived AI Involvement and Contributor Effort. Panel A shows that software developers distinguish AI-generated from human-written code above chance, but imperfectly. Panel B shows that perceived AI involvement is associated with lower perceived contributor effort, even though actual AI-generated code is not necessarily rated as lower effort.

	Main Effect Models			**Interaction Models**		
	(1)	(2)	(3)	(4)	(5)	(6)
AI PR	14.46***	21.45***	30.35***	1.881	10.07	19.59**
	(3.666)	(5.180)	(5.071)	(5.401)	(6.115)	(6.810)
Large Repo	–	–	–	-17.01***	–	–
				(2.369)		
AI PR × Large Repo	–	–	–	23.18**	19.56**	18.32*
				(7.362)	(7.278)	(7.248)
Repo FE	No	Yes	Yes	No	Yes	Yes
Month FE	No	No	Yes	No	No	Yes
Observations	13,614	13,614	13,614	13,614	13,614	13,614
R^2	0.0016	0.0848	0.0992	0.0055	0.0854	0.2412

Notes: All models include controls for initial deletions, initial additions, initial changed files, and code quality metrics (coefficients omitted for brevity). Standard errors clustered at the repository level in FE models. Large Repo is a time-invariant indicator absorbed by repository fixed effects in Columns (5)–(6); its main effect is reported only in Column (4), which omits repository fixed effects. The interaction AI PR × Large Repo is identified because AI status varies within repositories. Significance levels: *** p<0.001, ** p<0.01, * p<0.05, . p<0.1.

Table 20: Cross-section of AI-predicted PRs and Repo Size on Time to Close PRs

Validation source	*N*	Correct classification rate	Generator family in training
AI-generated code			
phi3:mini	151	100.00%	Yes
gemma3:1b	195	99.49%	Yes
qwen2.5-coder:1.5b-instruct	1,731	98.90%	Yes
deepseek-coder:6.7b-instruct	2,102	98.81%	Yes
qwen2.5-coder:7b-instruct	4,306	98.70%	Yes
gpt-4o-mini	22,263	98.68%	Yes
llama3.2:3b	428	97.90%	Yes
claude-3.5-sonnet	1,095	97.72%	Yes
deepseek-coder-v2:16b	1,190	97.31%	No
openai:gpt-3.5-turbo	164	96.95%	Yes
openai:gpt-4o-mini	320	96.88%	Yes
qwen3:8b	506	96.25%	Yes
anthropic:claude-3-5-haiku	578	95.85%	Yes
qwen2.5-coder:32b-instruct	949	94.84%	No
deepseek-r1:32b	1,584	94.63%	No
llama3.1:8b	794	93.95%	Yes
gpt-oss:20b	10,813	93.63%	Yes
claude-haiku-4-5	2,290	92.75%	No
qwen3-coder:30b	3,113	92.16%	Yes
openai:gpt-4o	782	91.18%	Yes
google:gemini-2.5-flash	7,339	90.97%	No
google:gemini-3-flash	4,210	90.88%	No
codellama:34b	373	90.62%	Yes
google:gemini-2.5-flash-lite	4,662	90.58%	Yes
anthropic:claude-3-7-sonnet	1,230	90.08%	Yes
codellama:7b-python	232	88.79%	Yes
mistral:7b	553	87.34%	Yes
deepseek-coder:33b	22	86.36%	Yes
codegemma:2b	77	81.82%	Yes
google:gemini-2.0-flash-lite	1,470	80.61%	Yes
claude-opus-4-5	1,408	79.76%	No
claude-sonnet-4-5	9,340	73.61%	No
google:gemini-2.0-flash	3,941	69.70%	No
openai:gpt-5.2	593	68.97%	No
starcoder2:7b	249	66.27%	Yes
claude-sonnet-4-6	882	61.56%	No
claude-opus-4-6	1,549	60.10%	No
starcoder2:15b	172	56.40%	No
Human-written code			
Human code, v4 test set	83,680	98.12%	—
Human code, January 2026 validation set	19,378	98.79%	—

Notes: For AI-generated validation samples, the correct classification rate is the percentage classified as AI-generated. For human-written validation samples, the correct classification rate is the percentage classified as human-written. "Generator family in training" indicates whether the model family was represented in the v4 training distribution. The January 2026 human-code sample is an out-of-time validation sample used to assess specificity on human-written code after the observational study window.

Table 21: Model-by-Model Validation of the AI Code Classifier

Alternative explanation	Why it matters	How the paper addresses it
Lower code quality	Maintainers may reject AI-like code because it is technically worse, not because contributor effort is harder to infer.	This study controls for observable code characteristics, complexity, maintainability-related metrics, and automated test outcomes. The mechanism test study holds the underlying code constant.
Competence inference	AI-like work may make contributors appear less capable, which could reduce willingness to review or integrate their work.	The mechanism test experiment measures perceived competence separately from perceived effort and code quality.
Spam or low commitment	Maintainers may treat AI-like submissions as careless, drive-by, or spam-like contributions.	The experiment tests perceived contributor effort directly, separating effort attributions from the mere presence of AI-like output.
AI stigma	Reviewers may dislike AI use regardless of the contributor's effort or the quality of the work.	The mechanism test experiment separates AI disclosure from effort legibility by crossing whether AI is disclosed with whether the contributor makes their own effort visible.
Contributor reputation	Prior experience with a contributor may substitute for effort cues in the submitted work.	The observational study examines heterogeneity by contributor status; the penalty for AI-like work should be stronger for first-time or less familiar contributors.

Table 22: Alternative Mechanisms and Empirical Tests

A.5 Additional Tables

Period	Outcome	Has Tests/Docs	Tests Add Ratio	Docs Add Ratio
Pre-AI	Accepted	0.194	0.339	0.031
Pre-AI	Rejected	0.149	0.337	0.034
Post-AI	Accepted	0.107	0.192	0.050
Post-AI	Rejected	0.051	0.167	0.054

Table 23: Quality Indicators by Period (Pre- vs. Post-AI) and Outcome (Accepted vs. Rejected Pull Requests)

Table 24 examines whether newcomers become regular contributors within one year. Before considering what happened to the first pull request, newcomers who submitted AI-predicted work were estimated to be 1.8 percentage points less likely to become regular contributors. This difference is small and not statistically reliable (p=.46). Column 3 then accounts for whether the first pull request was rejected. Rejection is associated with an 18.5-percentage-point decrease in the likelihood of becoming a regular contributor (p¡.001). Once rejection is included, there is no longer a negative relationship between submitting AI-predicted work and returning to the project.

	Became a regular contributor within one year		
	(1)	(2)	(3)
AI-predicted first pull request	−0.027	−0.018	0.044*
p-value	(0.270)	(0.460)	(0.067)
First pull request rejected			−0.185***
p-value			(< 0.001)
Repository fixed effects	Yes	Yes	Yes
Entry-year fixed effects	Yes	Yes	Yes
Contributor controls	No	Yes	Yes
First-PR rejection included	No	No	Yes
Observations	3,960	3,960	3,960

Notes: The sample contains one observation for each newcomer whose first pull request can be classified as AI-predicted or not AI-predicted. The dependent variable equals one if the newcomer submits at least three pull requests to the same repository within one year of the first contribution. Contributor controls are the logged age of the contributor's GitHub account and logged prior pull-request activity elsewhere on GitHub. The reported values are coefficients from linear probability models. Thus, −0.018 represents a 1.8-percentage-point difference. Parentheses report p-values calculated using standard errors clustered by repository.
Significance: $^{*}p < 0.10$, $^{**}p < 0.05$, $^{***}p < 0.01$.

Table 24: AI-Predicted First Contributions, Rejection, and Newcomer Retention

	Dependent variable: ln(Reviewer Effort)
Contributor Effort (*Effort*)	0.036^{***} (0.002)
Post-AI (author-level)	-0.206^{**} (0.081)
Contributor Effort × Post-AI	-0.010^{***} (0.003)
log(Lines Changed + 1)	−0.023 (0.017)
log(Files Changed + 1)	0.030^{**} (0.012)
Constant	0.061 (0.095)
Observations	62,235
R^2	0.198
Adjusted R^2	0.198
F-statistic	92.54
Covariance Type	Clustered by author

Notes: All variables are double-demeaned by author and repo×month to absorb fixed effects.
Reviewer effort is measured as the log of total non-author review comments per pull request.
Robust standard errors clustered at the author level are in parentheses.
$^{***}p < 0.01$, $^{**}p < 0.05$, $^{*}p < 0.10$.

Table 25: Effect of Author AI Adoption on the Relationship Between Contributor and Reviewer Effort, Size-Controlled

	DV: Maintainer Inline Comments (log)
Contributor Effort (pre-AI centered)	0.0289^{***}
	(0.0018)
Post–AI (Author-level)	-0.1404^{**}
	(0.0629)
Contributor Effort × Post–AI	-0.0114^{***}
	(0.0021)
Controls	Yes
Author FE	Yes
Repo × Month FE	Yes
Clusters	Author
Observations	74,007
R^2	0.084

Notes: OLS with author and repo×month fixed effects absorbed via double-demeaning. Contributor Effort is commits after PR open, centered by the author's pre-AI mean. Robust SEs clustered by author. $^{***}p < 0.01$, $^{**}p < 0.05$.

Table 26: Effort Reciprocity Before vs. After Author AI Adoption

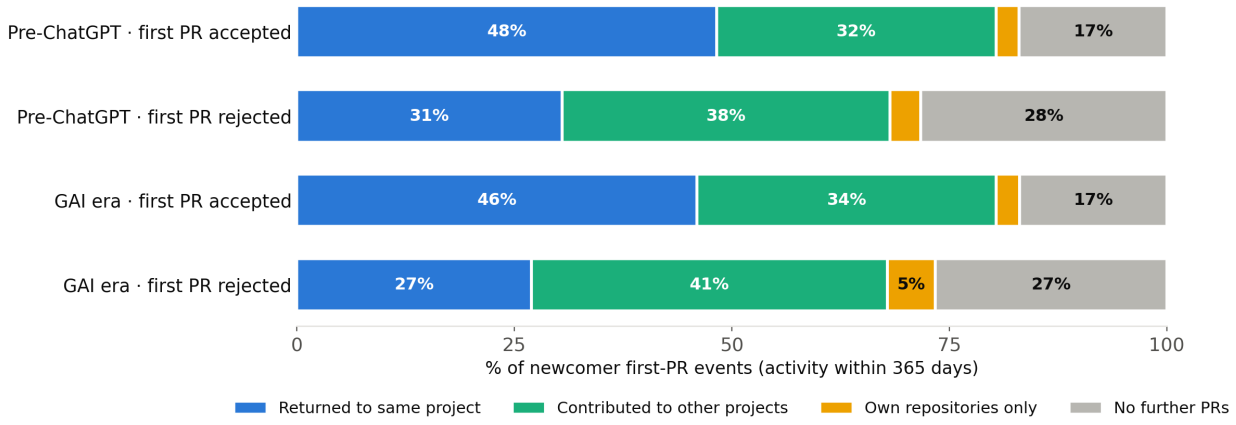


Figure 16: **Where do newcomers go after their first pull request?** Because the measure is based on public pull requests, commits pushed directly to a contributor's private repositories are not observed — the own-repository share is a lower bound — and "no further PRs" does not imply leaving GitHub.

		Pre-AI Period			Post-AI Period		
AI Status	Visible Rework	Accepted	Rejected	N	Accepted	Rejected	N
Human	Yes	0.924	0.076	24,571	0.925	0.075	20,193
Human	No	0.713	0.287	10,677	0.665	0.335	7,950
AI	Yes	0.281	0.719	2,007	0.554	0.446	4,039
AI	No	0.116	0.884	1,711	0.184	0.816	2,966

Table 27: Rejection rates by AI status and visible rework, before and after AI diffusion

A.6 Experimental Stimuli

Figures 17 and 18 show two experimental conditions applied to representative code works, as displayed to participants. Figure 17 shows the high-effort, AI-disclosed condition: the pull re-

AI Flagged	Outcome	Issue Comments	Review Decisions	Inline Comments	Timeline Events
No	Merged	5.43	7.32	13.62	30.39
No	Rejected	2.93	5.92	11.05	15.71
Yes	Merged	6.53	8.52	18.77	54.22
Yes	Rejected	2.68	4.12	8.52	14.89

Table 28: Average reviewer actions per PR by AI flag and outcome. Reviewer actions exclude author activity. "Inline Comments" reflect code-level feedback.

quest description includes diagnostic reasoning, iterative problem-solving, and codebase-specific references, and an "AI-assisted" badge appears in the PR header. Figure 18 shows the low-effort, no-disclosure condition: the description is sparse and generic, and no AI disclosure cue appears. Note that each condition is applied to a different fictional repository; the two figures illustrate different codebases.

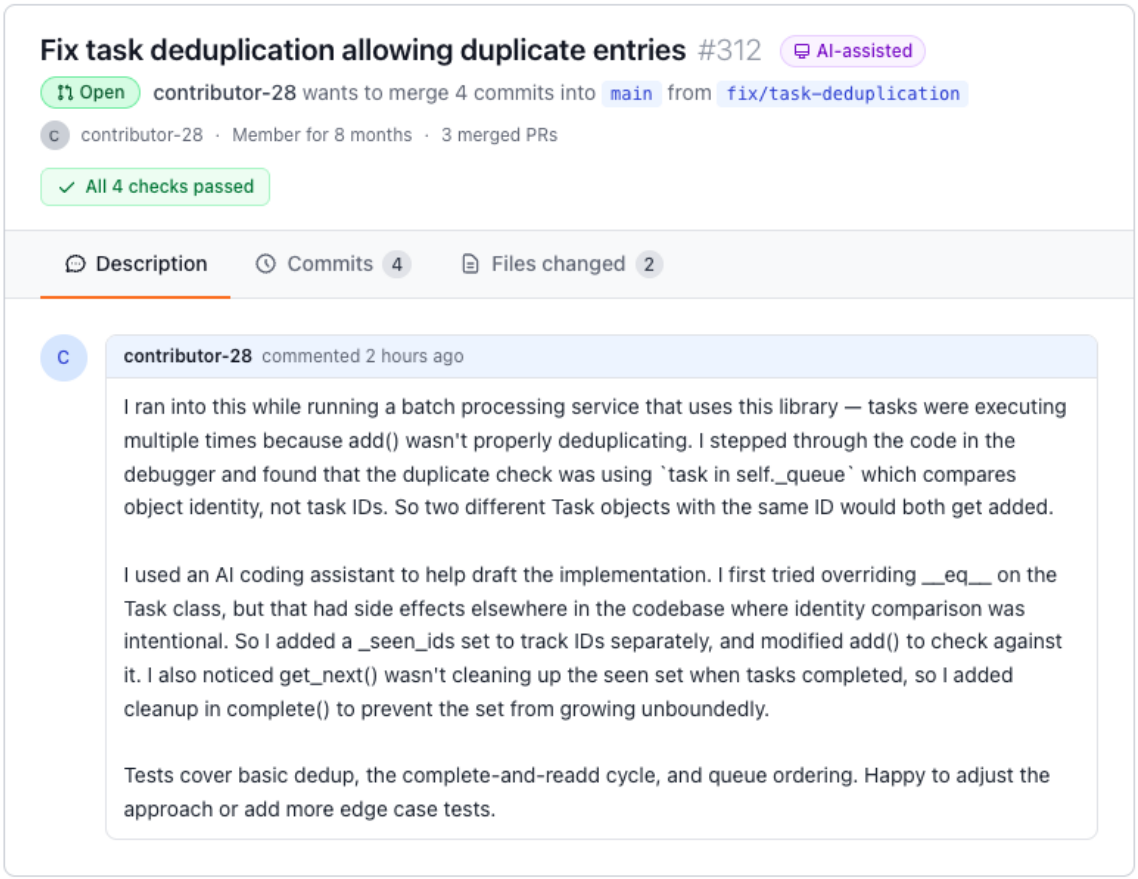


Figure 17: High-effort, AI-disclosed condition (Stimulus A: task deduplication). The PR description displays diagnostic reasoning, iteration, and codebase-specific references. An "AI-assisted" badge appears in the pull request header.

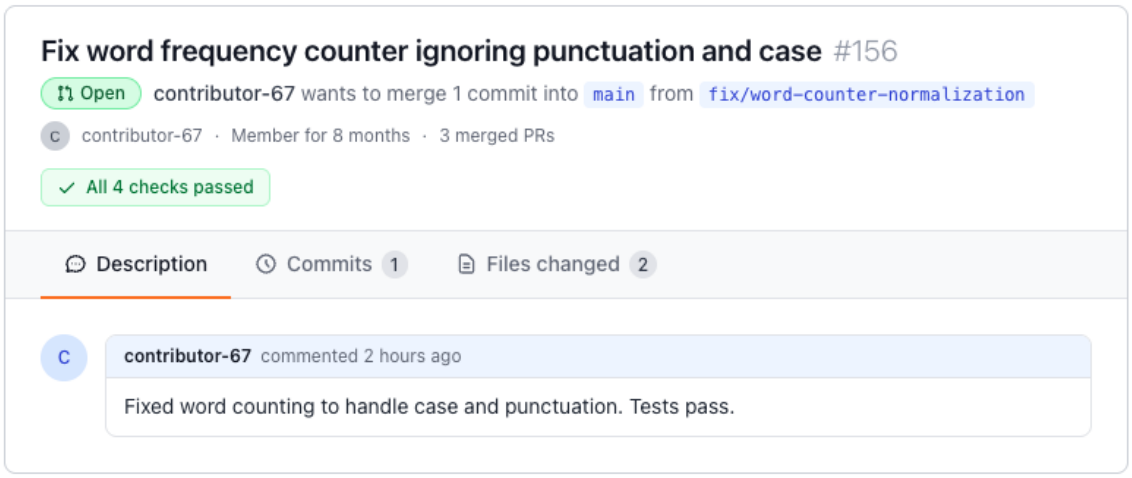


Figure 18: Low-effort, no-disclosure condition (Stimulus B: word frequency counter). The PR description is sparse and generic, and no AI disclosure cue appears.